



A K T U

B.Tech II-Year



CS IT & CS Allied

DATA STRUCTURE

Unit-3 : Searching & Sorting

in ONE SHOT



By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified



Pdf Notes



AKTU PYQs



Imp. Questions



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-3 Lecture-1

Today's Target

- Introduction to Sorting
 - Bubble Sort ✓
 - Insertion Sort ✓
 - Selection Sort ✓
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified



CS IT & CS Allied III-Sem

Subject : DATA STRUCTURE



Unit-3 : Syllabus

Searching: Concept of Searching, Sequential search, Index Sequential Search, Binary Search. Concept of Hashing & Collision resolution Techniques used in Hashing. **Sorting:** Insertion Sort, Selection, Bubble Sort, Quick Sort, Merge Sort, Heap Sort and Radix Sort.

SORTING

- Sorting refers to arranging data in a particular order. (Imp)
- **In-place sorting**: Do not require any extra space for sorting the array.
 - Example: Bubble sort, Selection Sort, Insertion Sort, Heapsort, *Quicksort*
- **Not-in-place Sorting**: The program requires space which more than or equal to the elements being sorted. Example: Merge-sort ✓

➤ **Internal sorting**:

- Sorting data can fit entirely in main memory of a computer.
- Commonly used when sorting smaller datasets.

➤ **External sorting**:

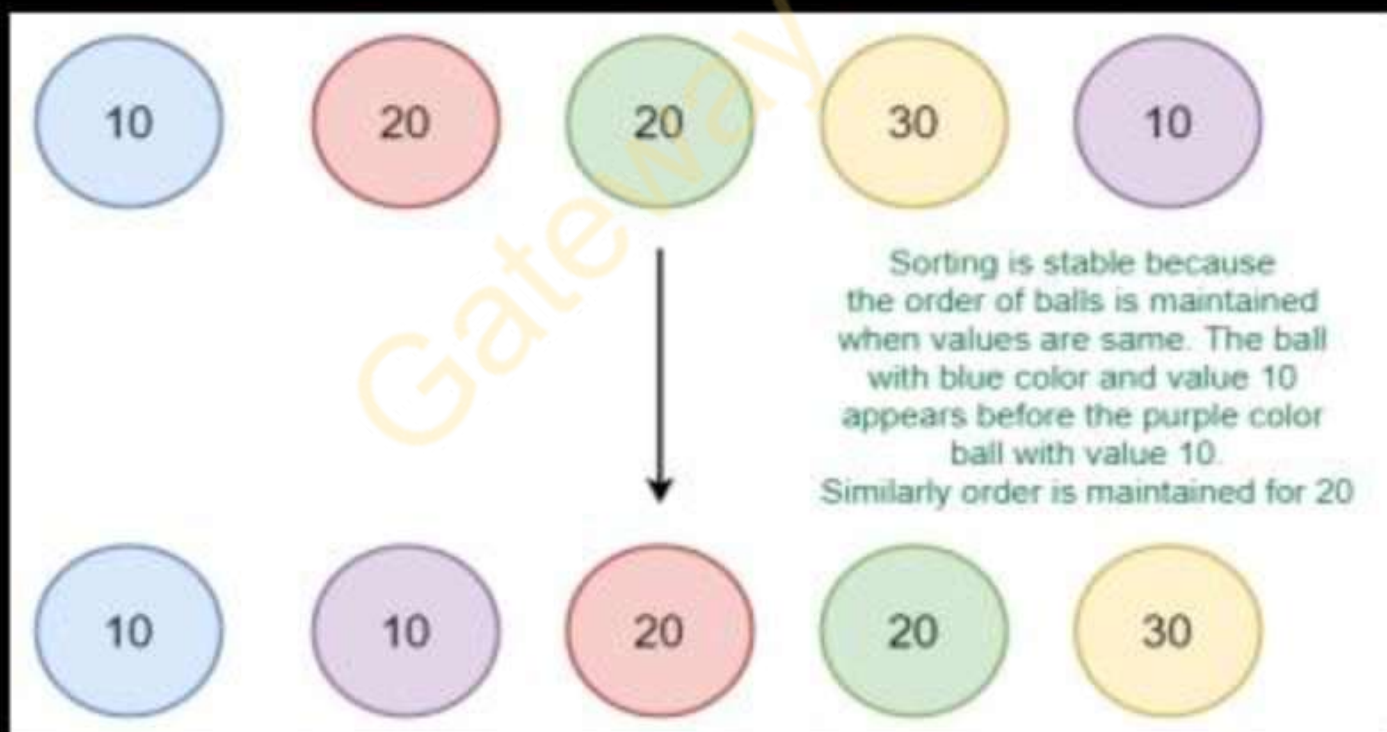
- Sorting data is too large to fit in main memory and must be stored on hard drives.
- Commonly useful in sorting significantly large datasets that exceed the available memory.

What do you understand by stable and unstable sorting?

Stable sorting

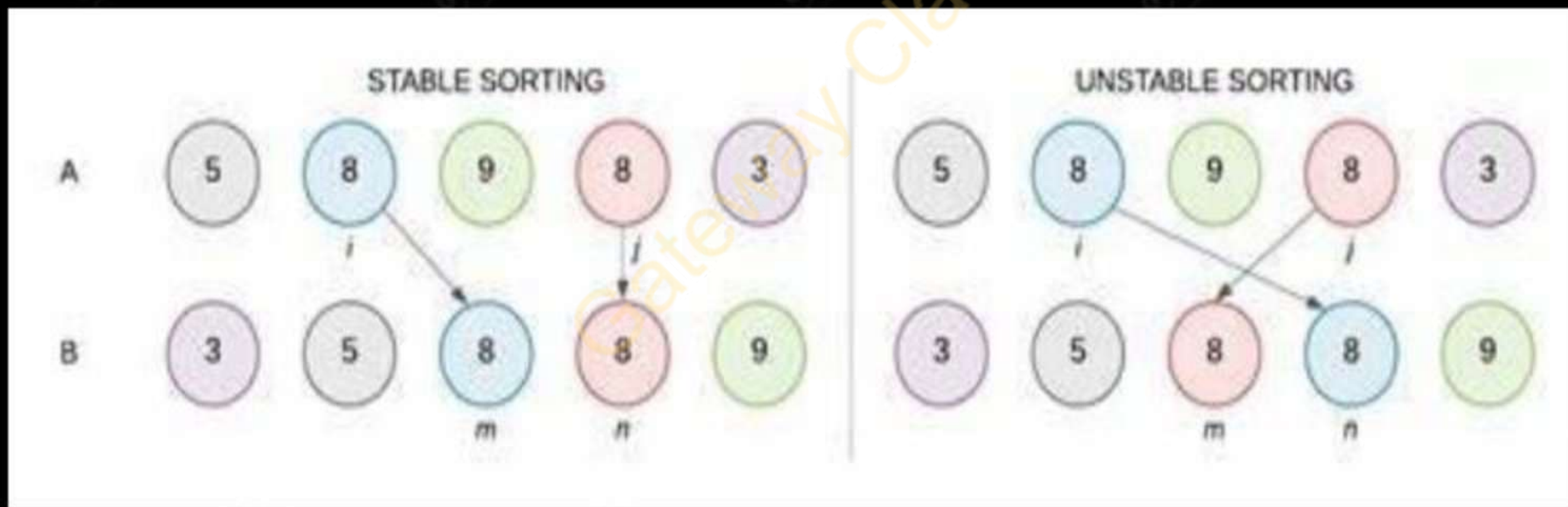
- A sorting algorithm is said to be stable if two objects with equal keys appear in the same order in sorted output as they appear in the input ~~sorted~~ array means initial order of equal items is preserved.
- Stability means that equivalent elements retain their relative positions, after sorting.

Example: bubble sort, insertion sort, merge sort, ~~counting sort~~ etc.



Unstable sorting

- In an unstable sorting algorithm, the ordering of the same values is not necessarily preserved after sorting.
- Example: selection sort, shell sort, Quicksort, Heapsort.



Bubble Sort

No of elements = n
Max no. of passes = $(n-1)$ ✓

int arr[] = {40,20,50,60,30,10};

<u>i=0</u> ✓ <u>pass-1</u> $m-1-i$						<u>i=1</u> <u>pass-2</u>				<u>i=2</u> <u>pass-3</u>			<u>i=3</u> <u>pass-4</u>		<u>pass-5</u> <u>i=4</u>		Sorted Array
	j=0	j=1	j=2	j=3	j=4		j=0	j=1	j=2	j=3		j=0	j=1	j=2		j=0	
0	40	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	10
1	20	40	40	40	40	40	40	40	40	40	40	30	30	30	30	10	20
2	50	50	50	50	50	50	50	50	30	30	30	40	10	10	10	30	30
3	60	60	60	60	30	30	30	50	10	10	10	40	40	40	40	40	40
4	30	30	30	30	60	10	10	10	50	50	50	50	50	50	50	50	50
5	10	10	10	10	10	60	60	60	60	60	60	60	60	60	60	60	60
	EX			EX	EX			EX	EX		EX	EX		EX	EX		

C Program to sort an array using bubble sort

```
#include <stdio.h>

void BubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++) // passes
    {
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arr[j] > arr[j+1])
            {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

Handwritten notes:
 $n = 6$
 $i = 0 \rightarrow 4$
 $j = 0 \rightarrow 4$
 n^2

```
int main()
{
    int arr[] = {40, 20, 50, 60, 30, 10};
    int n = sizeof(arr) / sizeof(arr[0]);
    BubbleSort(arr, n);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Handwritten note:
 $24 \text{ bytes} / 4 \Rightarrow 6$

Bubble Sort

Time Complexity

- Best-case — $O(n)$ $\Rightarrow$ In case of nearly sorted array
- Avg-case — $O(n^2)$
- worst-case — $O(n^2)$

Space Complexity

$$\underline{O(1)}$$

Selection Sort (n)

int arr[] = {40,20,50,60,30,10};

max passes = $n-1$

<u>$i=0$</u>	<u>pass-1</u>					<u>$i=1$</u>	<u>pass-2</u>				<u>$i=2$</u>	<u>pass-3</u>			<u>$i=3$</u>	<u>pass-4</u>		<u>pass-5</u> <u>$i=4$</u>	Sorted Array
	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$		$j=2$	$j=3$	$j=4$	$j=5$		$j=3$	$j=4$	$j=5$		$j=4$	$j=5$	$j=5$	
0	40	20	20	20	20	0	10	10	10	10	0	10	10	10	0	10	10	10	10
1	20	40	40	40	40	1	40	40	40	30	1	20	20	20	1	20	20	20	20
2	50	50	50	50	50	2	50	50	50	50	2	50	50	40	2	30	30	30	30
3	60	60	60	60	60	3	60	60	60	60	3	60	60	60	3	60	50	40	40
4	30	30	30	30	30	4	30	30	30	40	4	40	40	50	4	50	60	60	50
5	10	10	10	10	10	5	20	20	20	20	5	30	30	30	5	40	40	50	60
	EX				EX			EX	EX			EX	EX			EX	EX	EX	

C Program to sort an array using selection sort

```
#include <stdio.h>
```

```
void SelectionSort(int arr[], int n)
```

```
{ for (int i = 0; i < n - 1; i++) // passes
```

```
{ for (int j = i+1; j < n; j++)
```

```
{ if (arr[i] > arr[j])
```

```
{ int temp = arr[i];
```

```
arr[i] = arr[j];
```

```
arr[j] = temp;
```

```
}
```

```
}
```

```
}
```

Time Complexity

Best
Avg } $\rightarrow O(n^2)$
worst

Space comp

$O(1)$ ✓

```
int main()
```

```
{
```

```
int arr[] = {40, 20, 50, 60, 30, 10};
```

```
int n = sizeof(arr) / sizeof(arr[0]);
```

```
SelectionSort(arr, n);
```

```
printf("Sorted array: ");
```

```
for (int i = 0; i < n; i++)
```

```
printf("%d ", arr[i]);
```

```
return 0;
```

```
}
```

Insertion Sort

The simple steps of insertion sort -

Step 1 – If it is the first element, it is already sorted.

Step 2 – Pick next element

Step 3 – Compare with all elements in the sorted sub-list

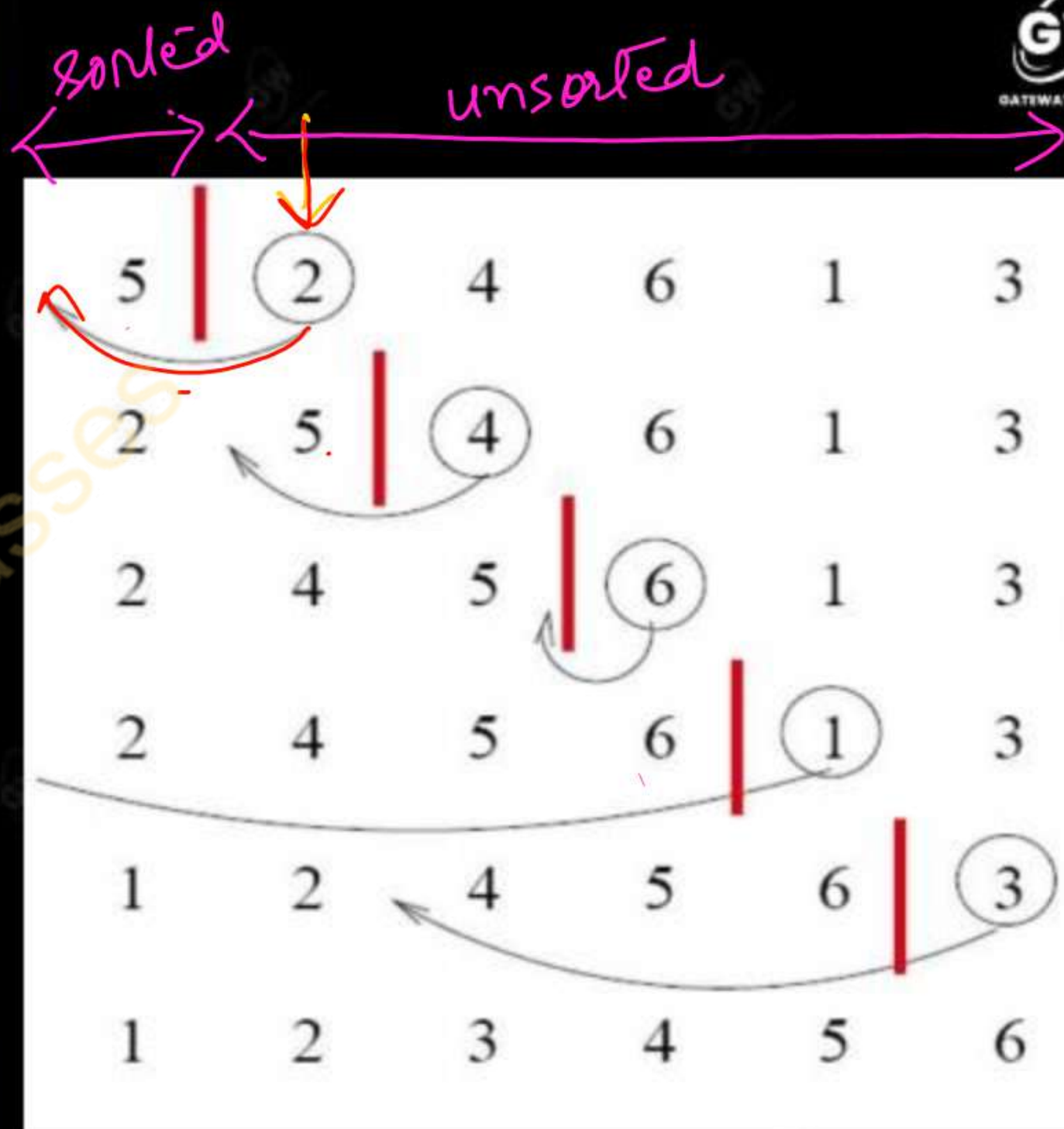
Step 4 – Shift all the elements in the sorted sub-list that is greater than the value to be sorted

Step 5 – Insert the value

Step 6 – Repeat until list is sorted

Points to Remember

- stable sort
- In place



Q: Implement the same on the following numbers; also calculate its time complexity.

13, 16, 10, 11, 4, 12, 6, 7 (AKTU 2021-22)

Insertion-Sort(A)

for $i = 2$ to A.length

key = A[i]

$j = i - 1$

while $j > 0$ and $A[j] > \text{key}$

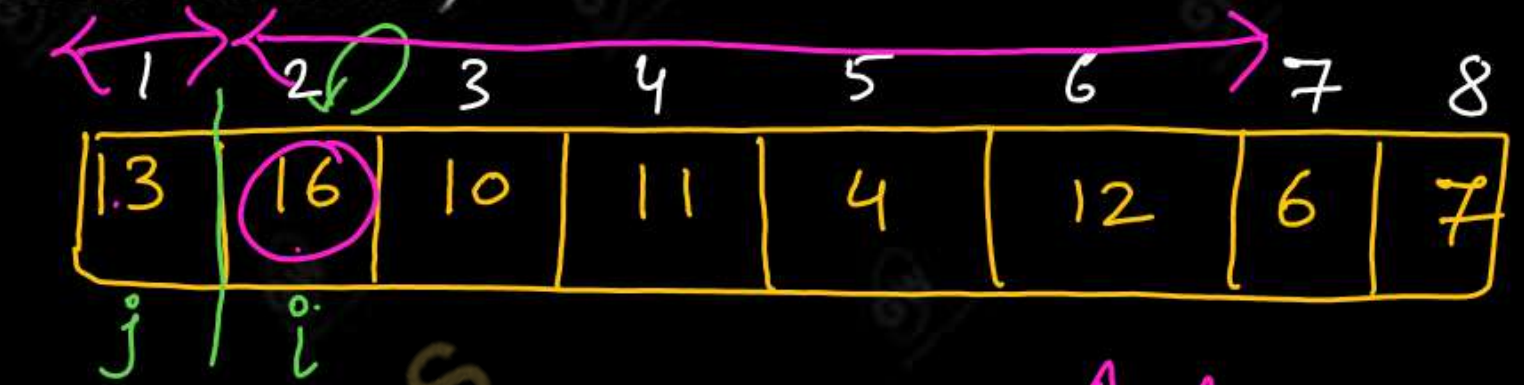
$A[j + 1] = A[j]$

$j = j - 1$

$A[j + 1] = \text{key}$

$i \rightarrow 2 \text{ to } 8$

$i = 2$



A.length = 8

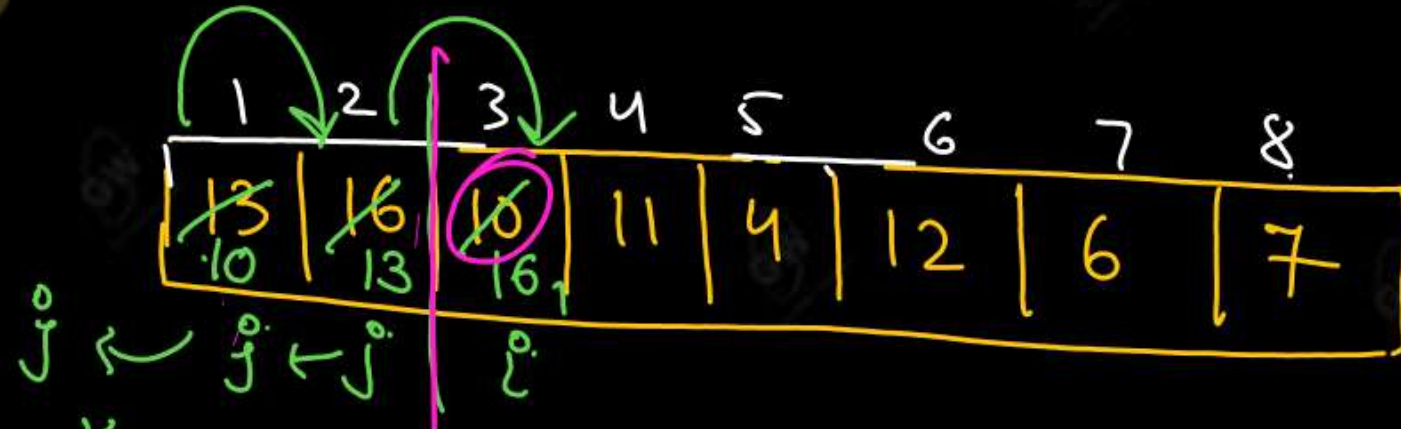
Key = A[i] = 16

$j = 1$

$A[j] < \text{key}$

$A[j+1] = \text{key} \Rightarrow A[2] = 16$

$i = 3$
 $j = 2$



Key = 10

$A[j] > \text{key}$

Q: Implement the same on the following numbers; also calculate its time complexity.

13, 16, 10, 11, 4, 12, 6, 7 (AKTU 2021-22)

Insertion-Sort(A)

for $i = 2$ to A.length

key = A[i]

$j = i - 1$

while $j > 0$ and $A[j] > \text{key}$

$A[j + 1] = A[j]$

$j = j - 1$

$A[j + 1] = \text{key}$

$i \rightarrow 2 \text{ to } 8$

$i = 4$

$j = 3$

1	2	3	4	5	6	7	8
10	13	16	11	4	12	6	7

$j \leftarrow j - 1$

Key = 11 ✓ $A[j] > \text{key}$ ✓

$i = 5$

$j = 4$

1	2	3	4	5	6	7	8
10	11	13	16	4	12	6	7

$j \leftarrow j - 1$

Key = 4 ✓

1	2	3	4	5	6	7	8
4	10	11	13	16	12	6	7

$j=4$

$$A[j] = 16 > 4$$

↓

1	2	3	4	5	6	7	8
4	10	11	13	16	12	6	7

Q: Implement the same on the following numbers; also calculate its time complexity.

13, 16, 10, 11, 4, 12, 6, 7 (AKTU 2021-22)

Insertion-Sort(A)

for $i = 2$ to A.length

key = A[i]

$j = i - 1$

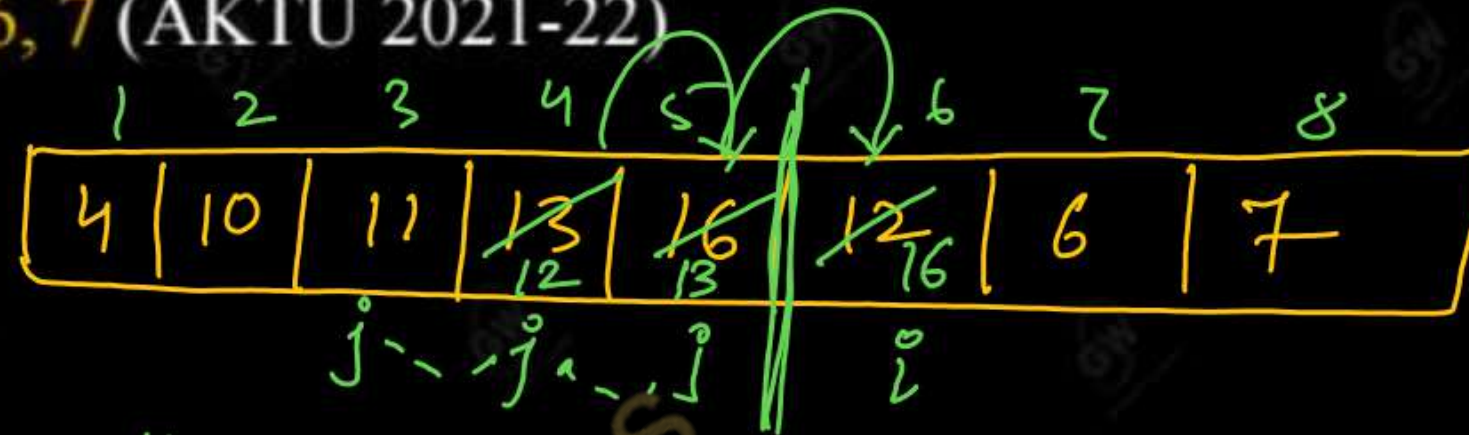
while $j > 0$ and $A[j] > \text{key}$

$A[j + 1] = A[j]$

$j = j - 1$

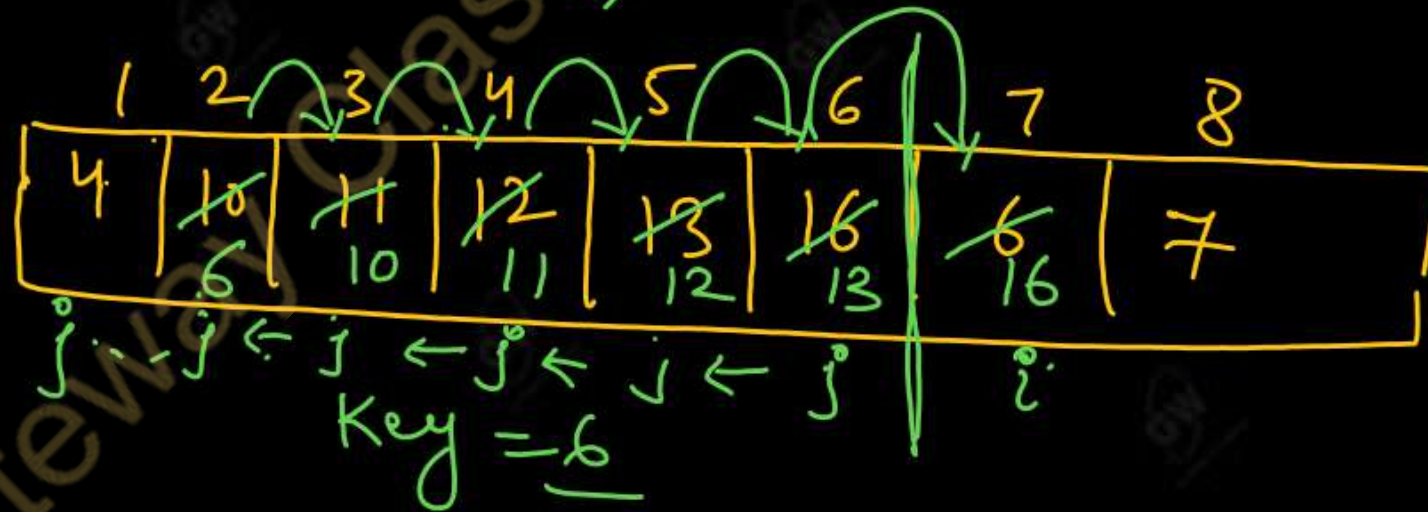
$A[j + 1] = \text{key}$

$i = 6$

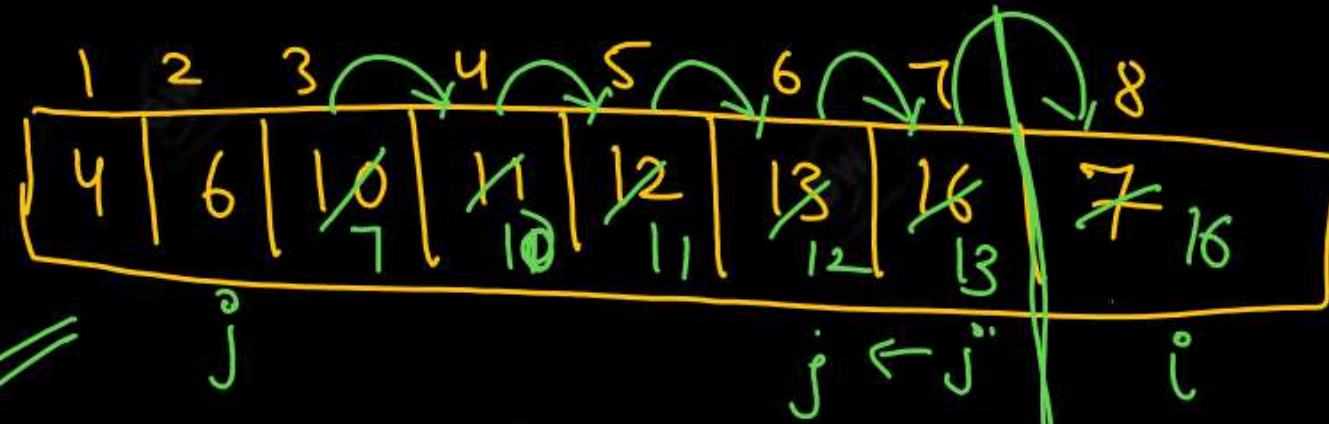


Key = 12

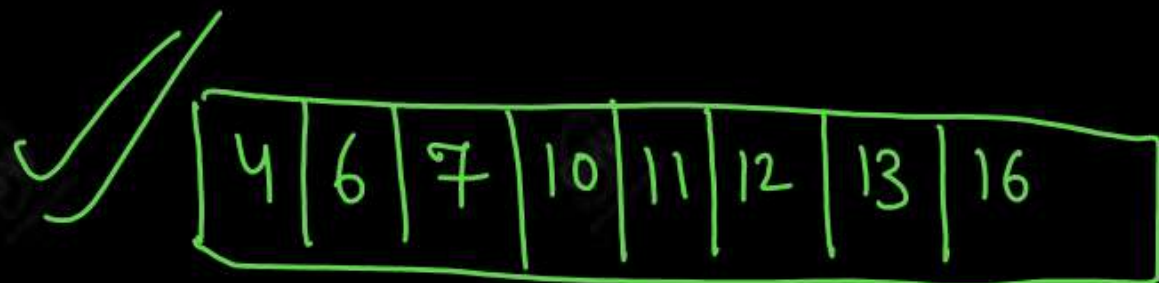
$i = 7$



Key = 6



Key = 7



C Program to sort an array using insertion sort

```
#include <math.h>
```

```
#include <stdio.h>
```

for $N=5$

```
void insertionSort(int arr[], int N)
```

```
{
```

$i=0$
 $i < N$
for (int $i = 1$; $i < N$; $i++$)

```
{
```

$j <= N-1$

```
    int key = arr[i];
```

```
    int j = i - 1;
```

```
    while (j >= 0 && arr[j] > key)
```

```
    {
```

```
        arr[j + 1] = arr[j];
```

```
        j = j - 1;
```

```
    }
```

```
    arr[j + 1] = key;
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
    int arr[] = { 12, 11, 13, 5, 6 };
```

```
    int N = sizeof(arr) / sizeof(arr[0]);
```

```
    insertionSort(arr, N);
```

```
    printf("Sorted array: ");
```

```
    for (int i = 0; i < N; i++)
```

```
    {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

0	1	2	3	4
12	11	13	5	6

worst case
Avg-case $\Rightarrow \underline{\underline{O(n^2)}}$

for n-elements

No. of comparisons $\Rightarrow$

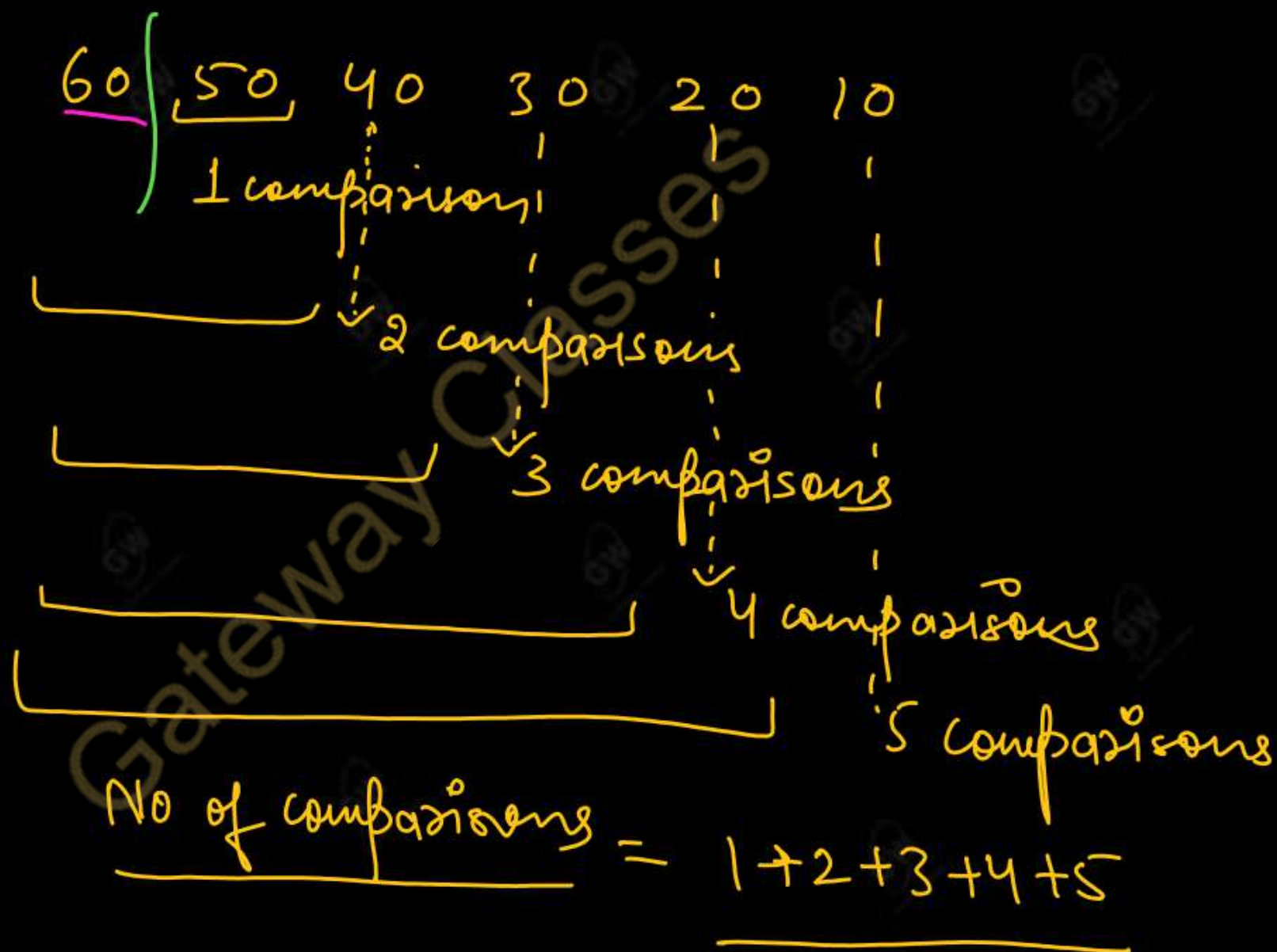
$$1 + 2 + 3 + \dots + (n-1)$$

$$\Rightarrow \frac{n(n-1)}{2}$$

$$\Rightarrow \boxed{O(n^2)}$$



Ex. $\rightarrow$ when list of elements are sorted in reverse order



Best case

Ex. If list is already sorted:



Total comparisons = $n-1$

$\Rightarrow O(n)$

Space complexity

$O(1)$

Data Structure

Unit-3 DPP-1

1. How does bubble sort work? Explain. (AKTU 2019-20)
2. Give example of one each stable and unstable sorting techniques. (AKTU 2021-22)
3. Write algorithms of insertion sort. Implement the same on the following numbers; also calculate its time complexity. 13, 16, 10, 11, 4, 12, 6, 7 (AKTU 2021-22) ✓
4. Write a C program for sorting 100 integer numbers using selection sort procedure. Discuss the worst-case time complexity of the algorithms. (AKTU 2023-24)
5. Differentiate between internal sorting and external sorting. (AKTU 2022-23)
6. Explain stable and in place sorting? (AKTU 2018-19)



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-3 Lecture-2

Today's Target

- Merge Sort
- AKTU PYQs
- DPP



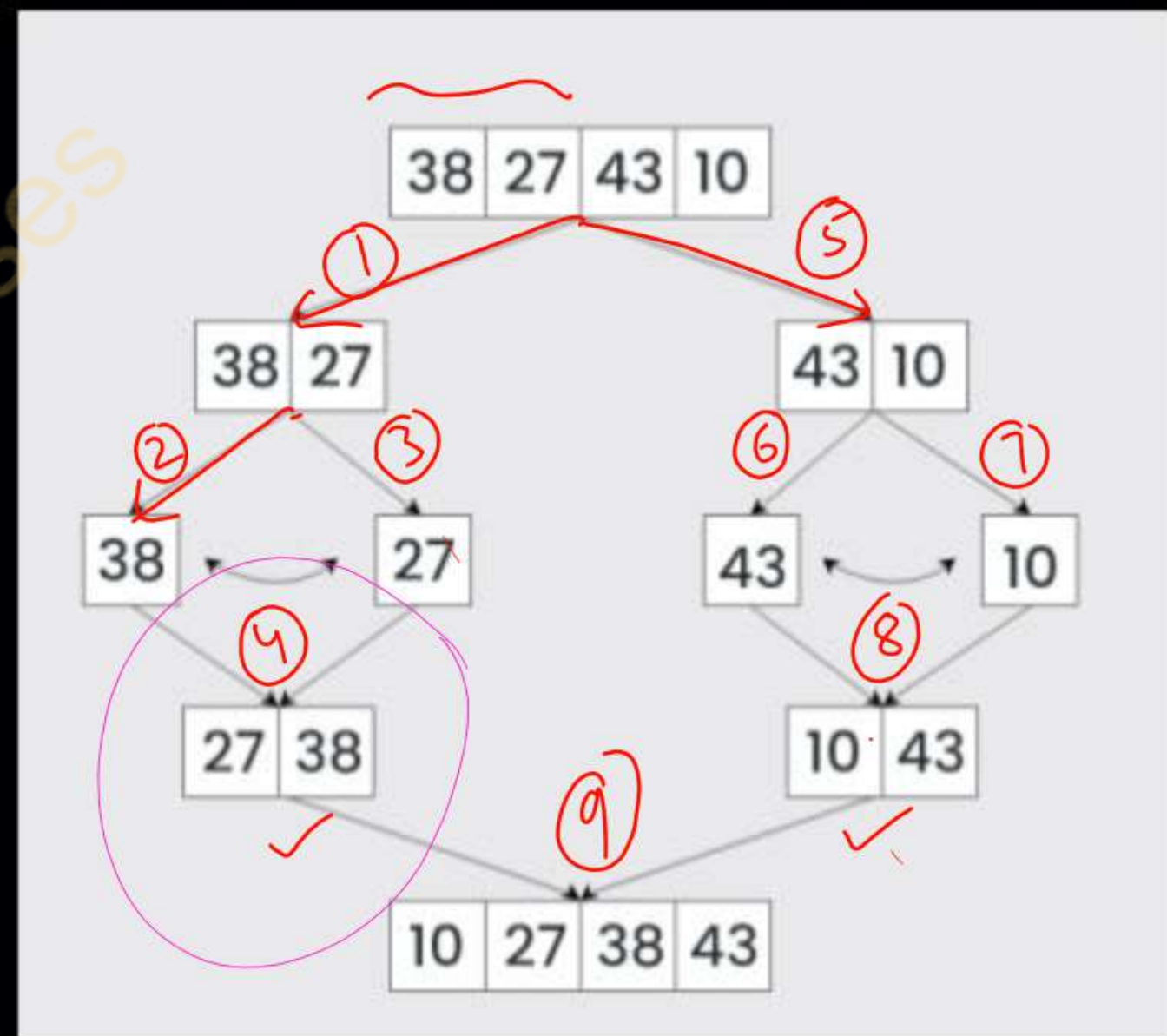
By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

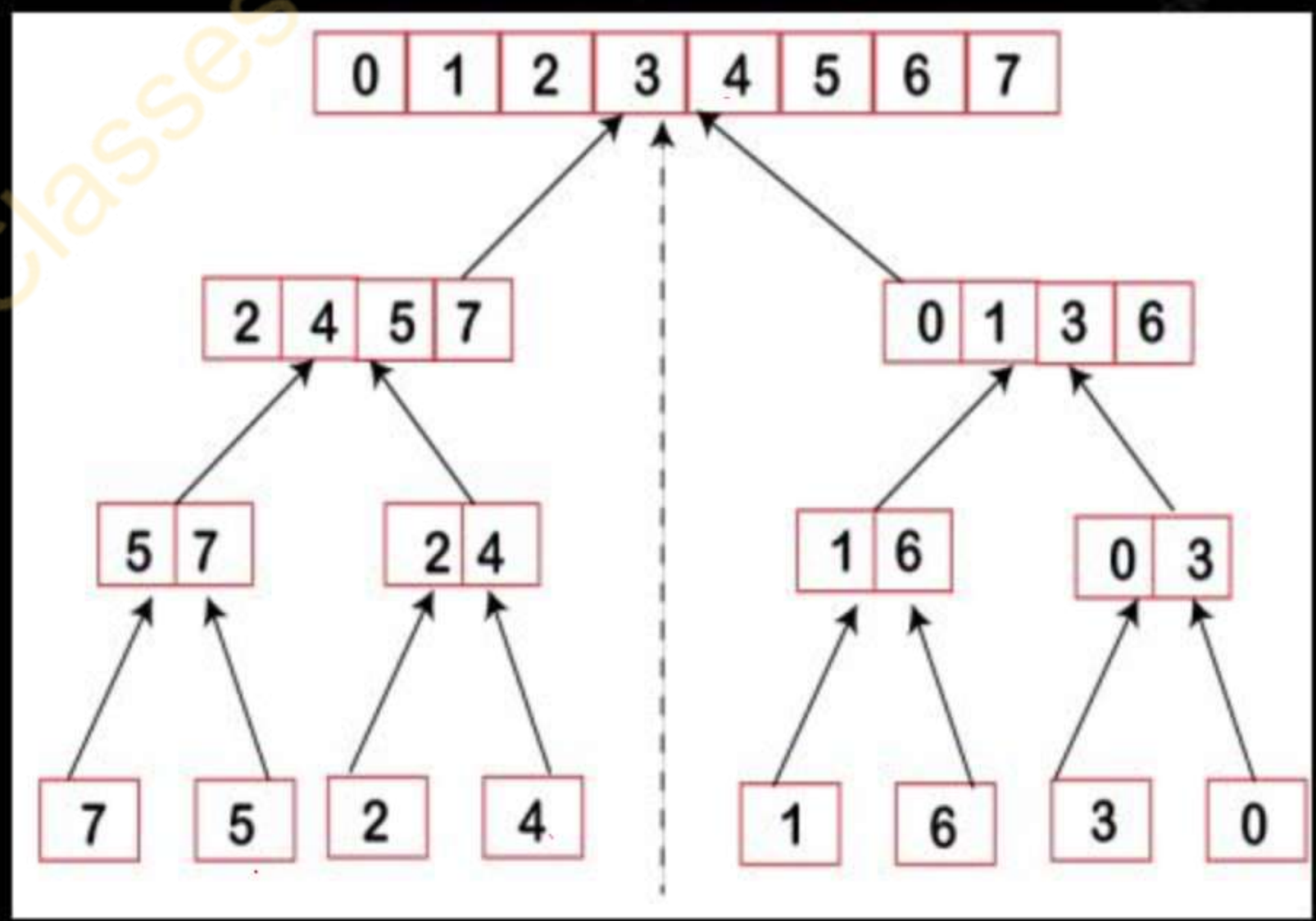
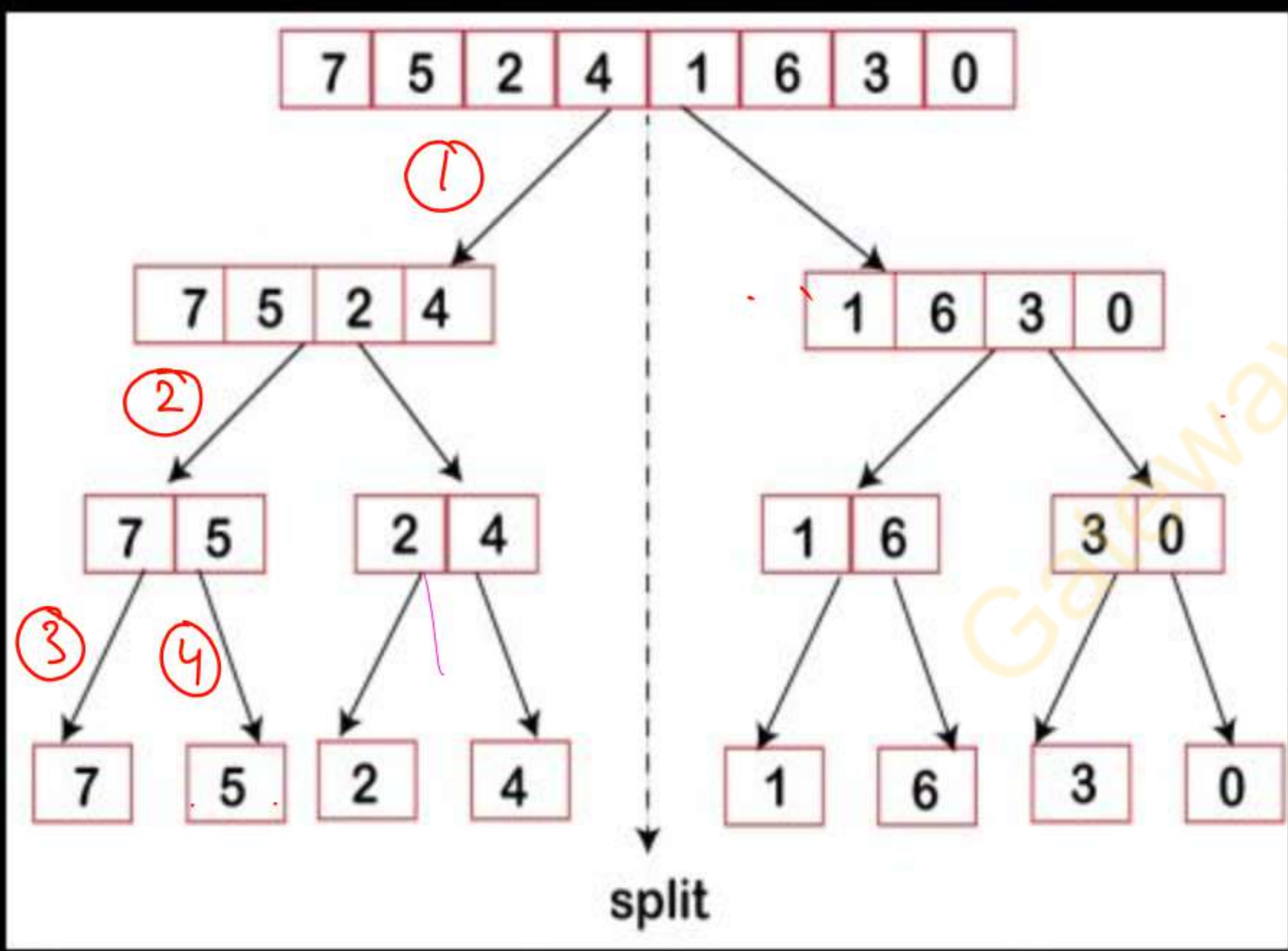
- M.Tech Gold Medalist
- Net Qualified

How merge sort works:

- **Divide:** Divide the list or array recursively into two halves until it can no more be divided.
- **Conquer:** Each subarray is sorted individually using the merge sort algorithm.
- **Merge:** The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

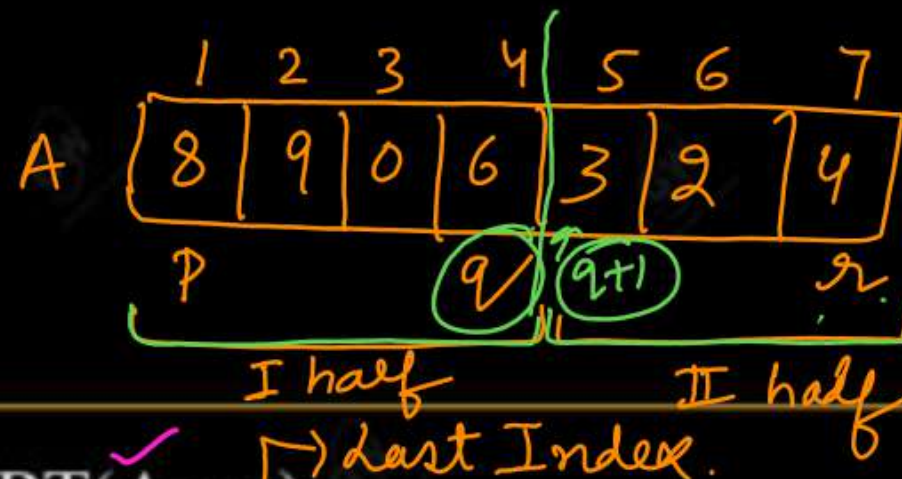


Merge Sort



Merge Sort Algorithm

$$q = \left\lfloor \frac{p+r}{2} \right\rfloor = \left\lfloor \frac{1+7}{2} \right\rfloor = 4$$



Array $A[p..r]$ starts at index p and ends at index r .

Divide: q as central point between p and r , we divide array into two arrays $A[p..q]$ and $A[q+1, r]$.

Conquer:

Individually sort subarrays $A[p..q]$ and $A[q+1, r]$ recursively.

Combine: $A[p..q]$ and $A[q+1, r]$ are merged to form a new sorted array $[p..r]$.

MERGE SORT($\underline{A}, p, r$)

1. If $p \leq r$ // Base condition $\rightarrow$ First Index
2. Then $q \leftarrow \text{floor}[(p+r)/2]$
3. MERGE-SORT ($\underline{A}, \underline{p}, \underline{q}$) // Sort first half
4. MERGE-SORT ($\underline{A}, q+1, r$) // Sort second half
5. MERGE ($\underline{A}, p, q, r$) // Merge two sorted halves

Recursive calls

$$q = \left\lfloor \frac{1+7}{2} \right\rfloor = 4$$

1	2	3	4	5	6	7
10	1	23	50	4	9	-4

p q q+1 r

Example

$$q = \left\lfloor \frac{1+4}{2} \right\rfloor = 2$$

1	2	3	4
10	1	23	50

p r

$$q = \left\lfloor \frac{5+7}{2} \right\rfloor = 6$$

4	9	-4
---	---	----

p r

further
division
is stopped

$$q = \left\lfloor \frac{1+2}{2} \right\rfloor = 1$$

1	2
10	1

p r

23	50
----	----

4	9
---	---

-4

10

p r

1

p r

23

50

4

9

1	10
---	----

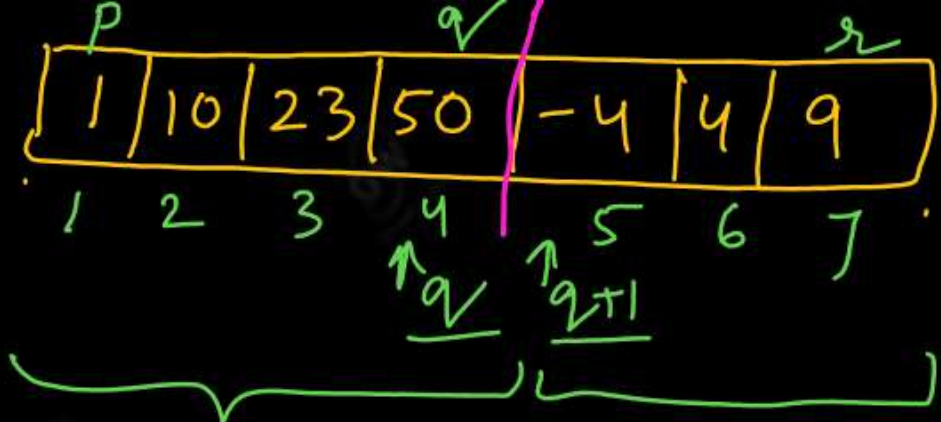
23	50
----	----

4	9
---	---

1	10	23	50
---	----	----	----

-4	4	9
----	---	---

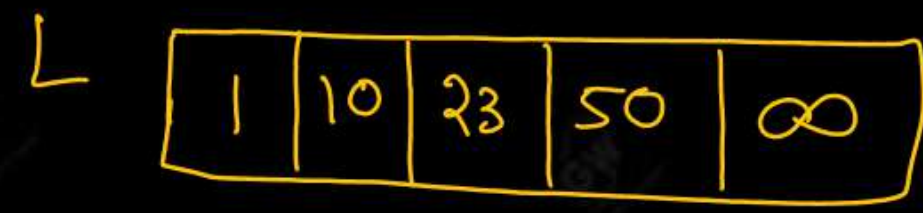
-4	1	4	9	10	23	50
----	---	---	---	----	----	----



I sorted array II sorted array

$$n_1 = q - p + 1 \Rightarrow 4 - 1 + 1 \Rightarrow 4 \checkmark$$

$$n_2 = r - q \Rightarrow 7 - 4 = 3 \checkmark$$



MERGE (A, p, q, r)

$$n_1 = q - p + 1 \checkmark$$

$$n_2 = r - q \checkmark$$

create arrays L[1..... $n_1 + 1$] and
R[1..... $n_2 + 1$]

for i ← 1 to n_1
do L[i] ← A[p + i - 1] }
for j ← 1 to n_2
do R[j] ← A[q + j] }

$$L[n_1 + 1] \leftarrow \infty$$

$$R[n_2 + 1] \leftarrow \infty$$

$$i \leftarrow 1$$

$$j \leftarrow 1$$

For k ← p to r

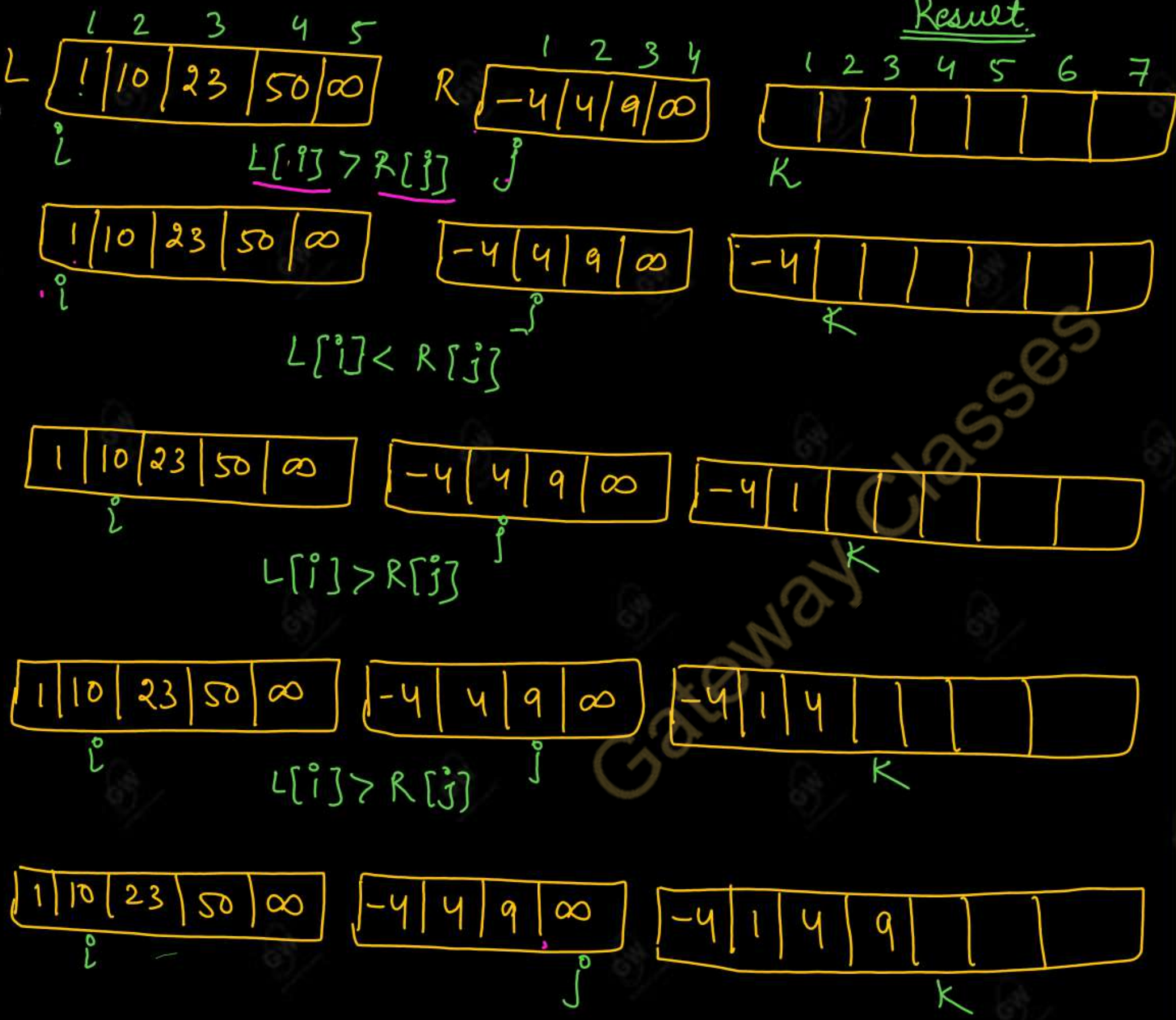
do if L[i] ≤ R[j]

then A[k] ← L[i]

i ← i + 1

else A[k] ← R[j]

j ← j + 1



GW
GATEWAY CLASSES

MERGE (A, p, q, r)

$n1 = q - p + 1$ ——— $O(1)$
 $n2 = r - q$ ——— $O(1)$
 create arrays $L[1 \dots n1 + 1]$ and $R[1 \dots n2 + 1]$
 for $i \leftarrow 1$ to $n1$
 do $L[i] \leftarrow A[p + i - 1]$
 for $j \leftarrow 1$ to $n2$
 do $R[j] \leftarrow A[q + j]$
 $L[n1 + 1] \leftarrow \infty$
 $R[n2 + 1] \leftarrow \infty$
 $i \leftarrow 1$ ✓
 $j \leftarrow 1$ ✓
 For $k \leftarrow p$ to r ✓
 do if $L[i] \leq R[j]$ ✓
 then $A[k] \leftarrow L[i]$
 $i \leftarrow i + 1$
 else $A[k] \leftarrow R[j]$
 $j \leftarrow j + 1$

$O(n)$ ✓

GW
GATEWAY CLASSES

L
1 | 10 | 23 | 50 | ∞
i

R
-4 | 4 | 9 | ∞
j

$L[i] < R[j]$

1 | 10 | 23 | 50 | ∞
i

-4 | 4 | 9 | ∞
j

$L[i] < R[j]$

1 | 10 | 23 | 50 | ∞
i

-4 | 4 | 9 | ∞
j

$L[i] < R[j]$

1 | 10 | 23 | 50 | ∞
i

-4 | 4 | 9 | ∞
j

-4 | 1 | 4 | 9 | 10 | 23 | 50
k

Result.
1 2 3 4 5 6 7
-4 | 1 | 4 | 9 | | | |
k

MERGE (A, p, q, r)

$n1 = q - p + 1$

$n2 = r - q$

create arrays L[1.....n1 + 1] and
R[1.....n2 + 1]

for i $\leftarrow$ 1 to n1
do L[i] $\leftarrow$ A[p + i - 1]

for j $\leftarrow$ 1 to n2
do R[j] $\leftarrow$ A[q + j]

L[n1 + 1] $\leftarrow \infty$

R[n2 + 1] $\leftarrow \infty$

i $\leftarrow$ 1

j $\leftarrow$ 1

For k $\leftarrow$ p to r

do if L[i] $\leq$ R[j]

then A[k] $\leftarrow$ L[i]

i $\leftarrow$ i + 1

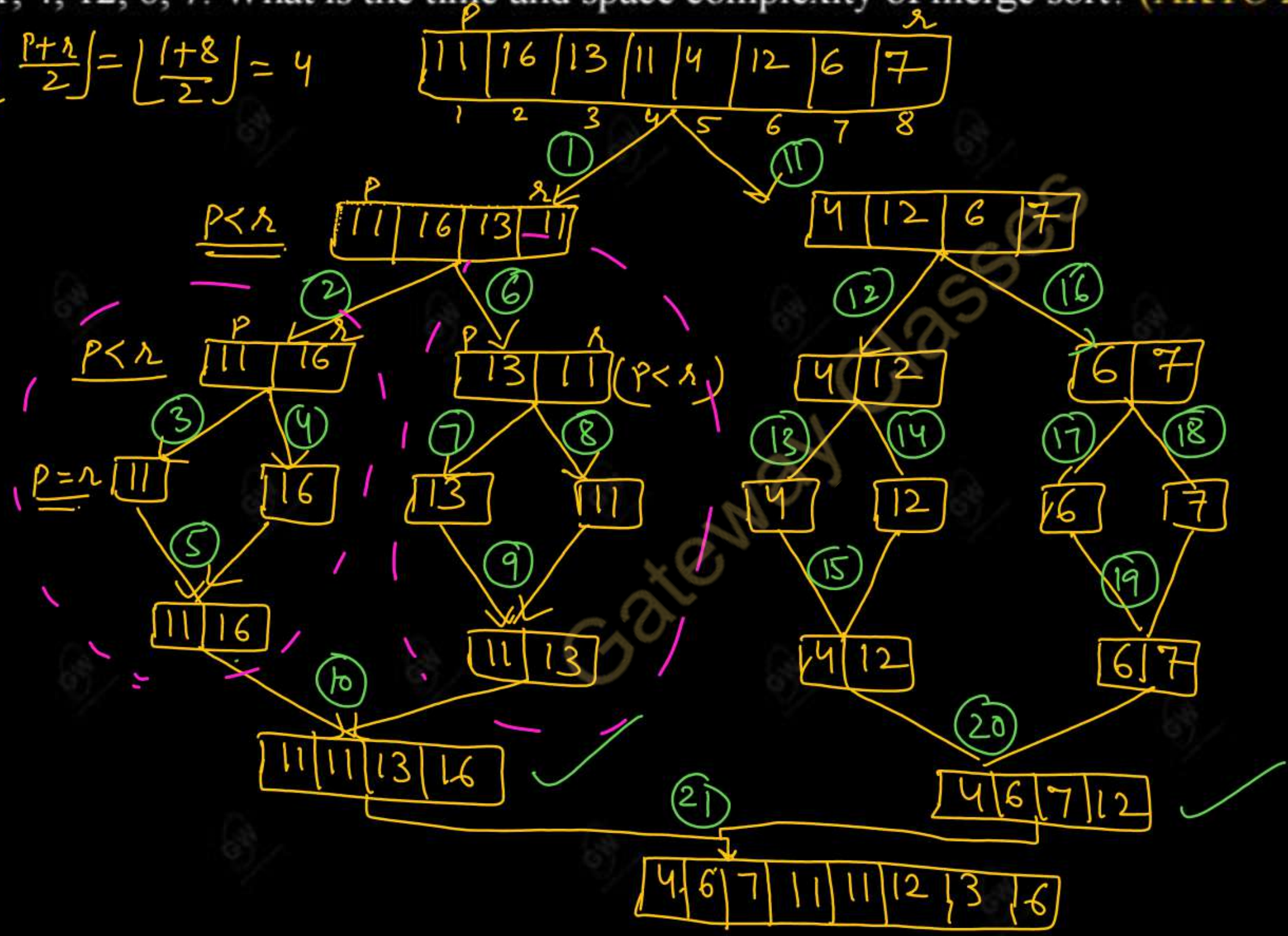
else A[k] $\leftarrow$ R[j]

j $\leftarrow$ j + 1

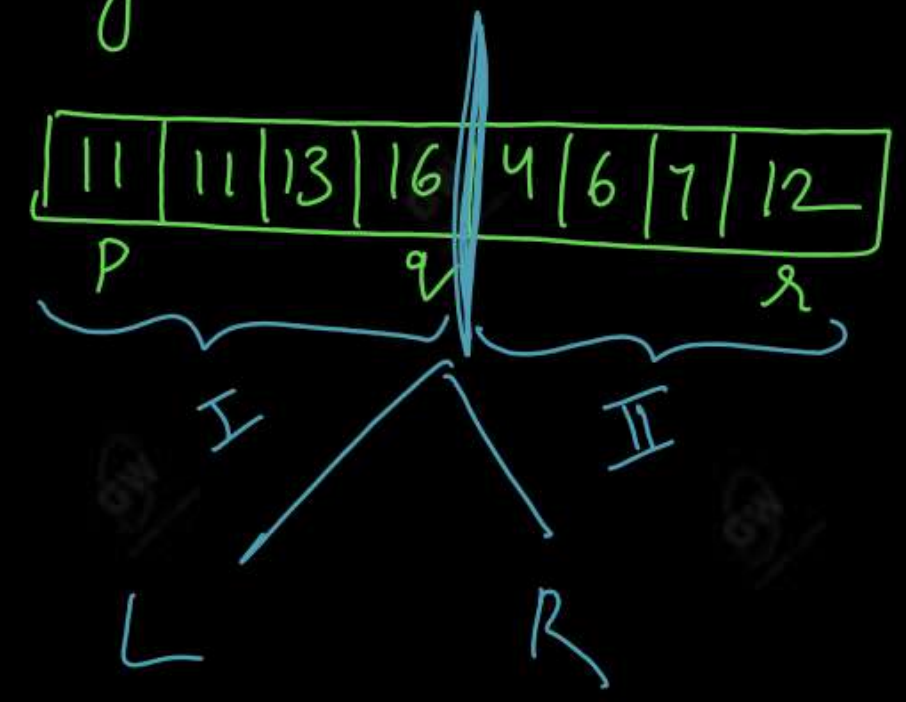
loop ends

Q: Demonstrate merge sort algorithm to sort the following elements in ascending order: 11, 16, 13, 11, 4, 12, 6, 7. What is the time and space complexity of merge sort? (AKTU 2024-25)

$$q = \left\lfloor \frac{p+r}{2} \right\rfloor = \left\lfloor \frac{1+8}{2} \right\rfloor = 4$$



After this trace Merge algorithm on



```
#include <stdio.h>
```

```
void merge(int arr[], int p, int q, int r)
```

```
{ int n1 = q - p + 1;
```

```
int n2 = r - q;
```

```
int L[n1], M[n2];
```

```
for (int i = 0; i < n1; i++)
```

```
    L[i] = arr[p + i];
```

```
for (int j = 0; j < n2; j++)
```

```
    M[j] = arr[q + 1 + j];
```

```
int i, j, k;
```

```
i = 0, j = 0, k = p;
```

```
while (i < n1 && j < n2)
```

```
{ if (L[i] <= M[j])
```

```
    { arr[k] = L[i];
```

```
      i++; k++; } }
```

```
else {
```

```
    arr[k] = M[j];
```

```
    j++; k++;
```

```
}
```

```
}
```

```
while (i < n1)
```

```
{
```

```
    arr[k] = L[i];
```

```
    i++; k++;
```

```
}
```

```
while (j < n2)
```

```
{
```

```
    arr[k] = M[j];
```

```
    j++; k++;
```

```
}
```

```
}
```

} if elements in L array are remaining

} if elements in M array are remaining

```
void mergeSort(int arr[], int p, int r)
```

```
{  
    if (p < r)  
    {  
        int q = (p + r) / 2;  
        mergeSort(arr, p, q);  
        mergeSort(arr, q + 1, r);  
        merge(arr, p, q, r);  
    }  
}
```

```
void printArray(int arr[], int size)
```

```
{  
    for (int i = 0; i < size; i++)  
        printf("%d ", arr[i]);  
}
```

```
int main()
```

```
{  
    int arr[] = {6, 5, 12, 10, 9, 1};  
    int size = sizeof(arr) / sizeof(arr[0]);  
    mergeSort(arr, 0, size - 1);  
    printf("Sorted array: \n");  
    printArray(arr, size);  
}
```

Merge Sort Time Complexity Analysis

Mergesort (A, p, r) $\rightarrow$ $T(n)$

if (p < r) $\xrightarrow{\quad\quad\quad} O(1)$

then find $q \leftarrow \lfloor \frac{p+r}{2} \rfloor \xrightarrow{\quad\quad\quad} O(1)$

Mergesort (A, p, q); $T(n/2)$

Mergesort (A, q+1, r); $T(n/2)$

Merge (A, p, q, r); $\xrightarrow{\quad\quad\quad} O(n)$

$$T(n) = T(n/2) + T(n/2) + O(n)$$

$$T(n) = 2T(n/2) + n$$

Time complexity:

Best case: $O(n \log n)$

Worst case: $O(n \log n)$

Average case: $O(n \log n)$

Space complexity: $O(n)$

Advantages of Merge Sort:

- Stable
- Guaranteed worst-case performance even on large datasets.
- Simple to implement.
- suitable for parallel processing. as we independently merge subarrays.

Disadvantages of Merge Sort:

- Space complexity: Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
- Not in-place as it requires additional memory to store the sorted data.
- Slower than Quick Sort because quick sort works in-place.

1. Write an algorithm for merge sort and apply on following elements 45,32,65,76,23,12,54,67,22,87.
(AKTU 2020-21)
2. Use the merge sort algorithm to sort the following elements in ascending order. 13, 16, 10, 11, 4, 12, 6, 7. What is the time and space complexity of merge sort? (AKTU 2021-22)
3. Demonstrate merge sort algorithm to sort the following elements in ascending order: 11, 16, 13, 11, 4, 12, 6, 7. What is the time and space complexity of merge sort? (AKTU 2024-25)



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-3 Lecture-3

Today's Target

- Quick Sort
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

To sort Array $A[p.. r]$

Divide: Partition $A[p.. r]$, into two (possibly empty) subarrays

$A[p.. q-1]$ and $A[q+1.. r]$, such that

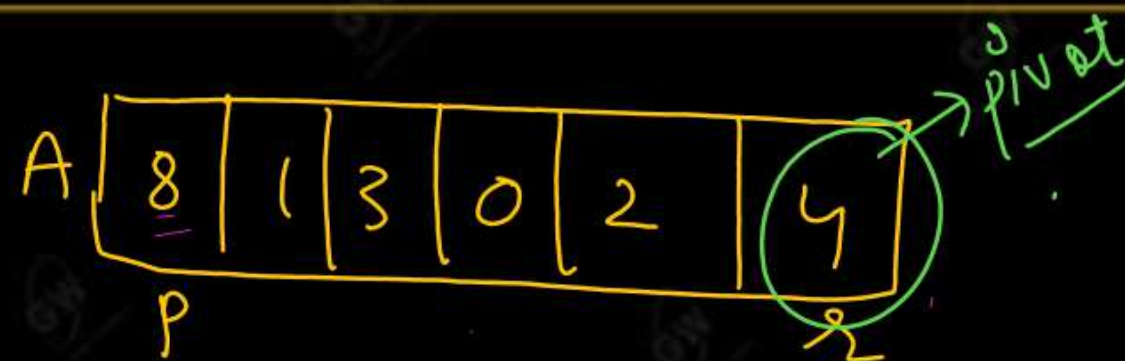
- Each element in the first subarray $A[p.. q-1]$ is $\leq A[q]$, and
- $A[q]$ is $\leq$ each element in the second subarray $A[q+1.. r]$

Conquer: Sort two subarrays by recursive calls to QUICKSORT

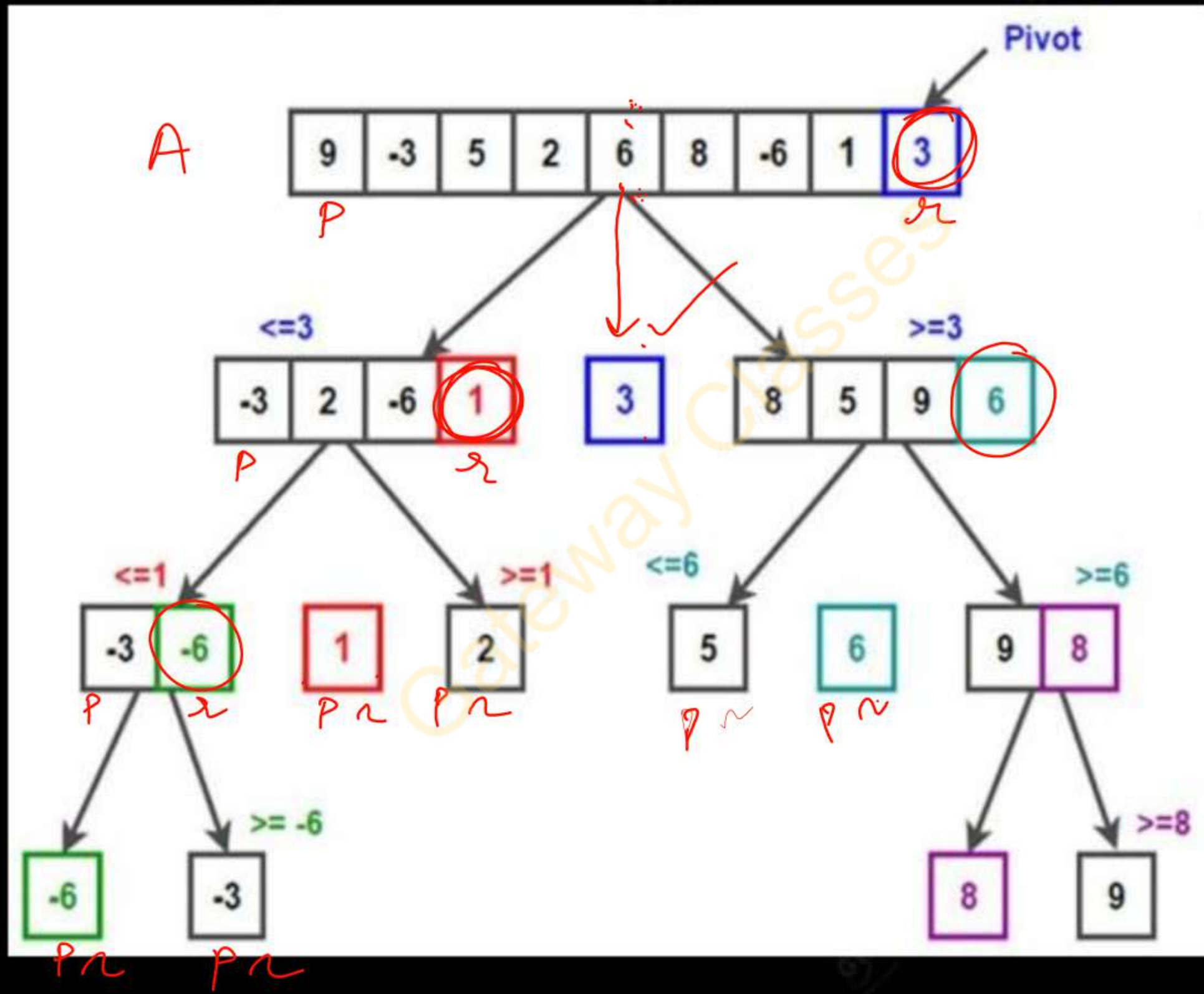
Combine: No work is needed to combine the subarrays, because they are sorted in place.

QUICK SORT(A, p, r)

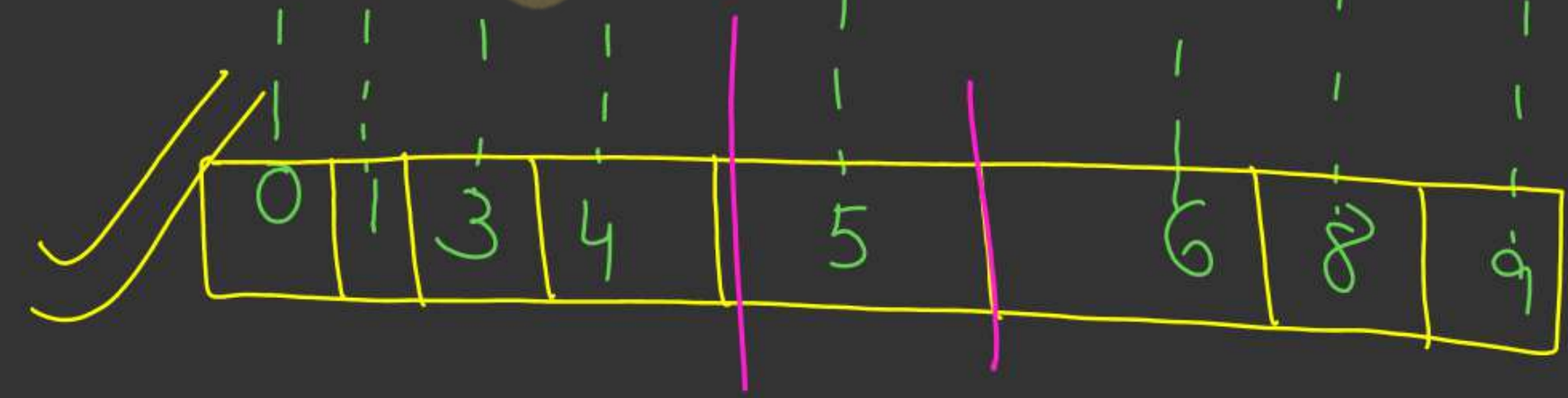
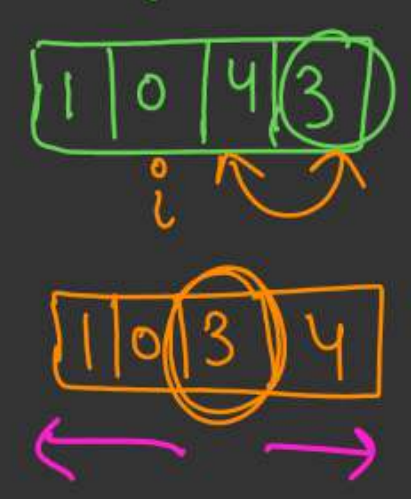
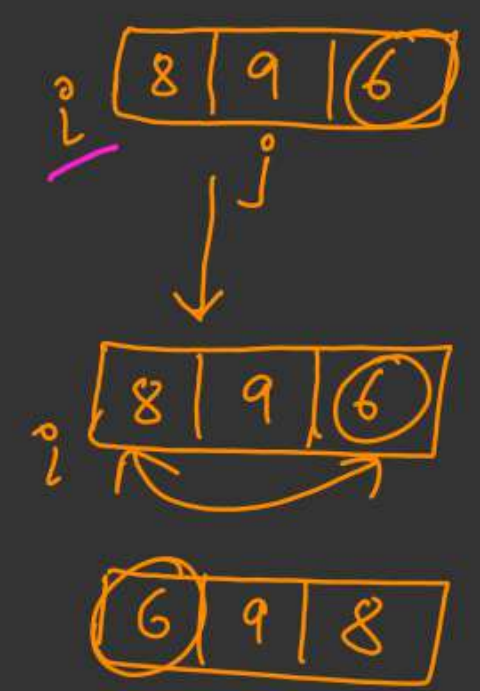
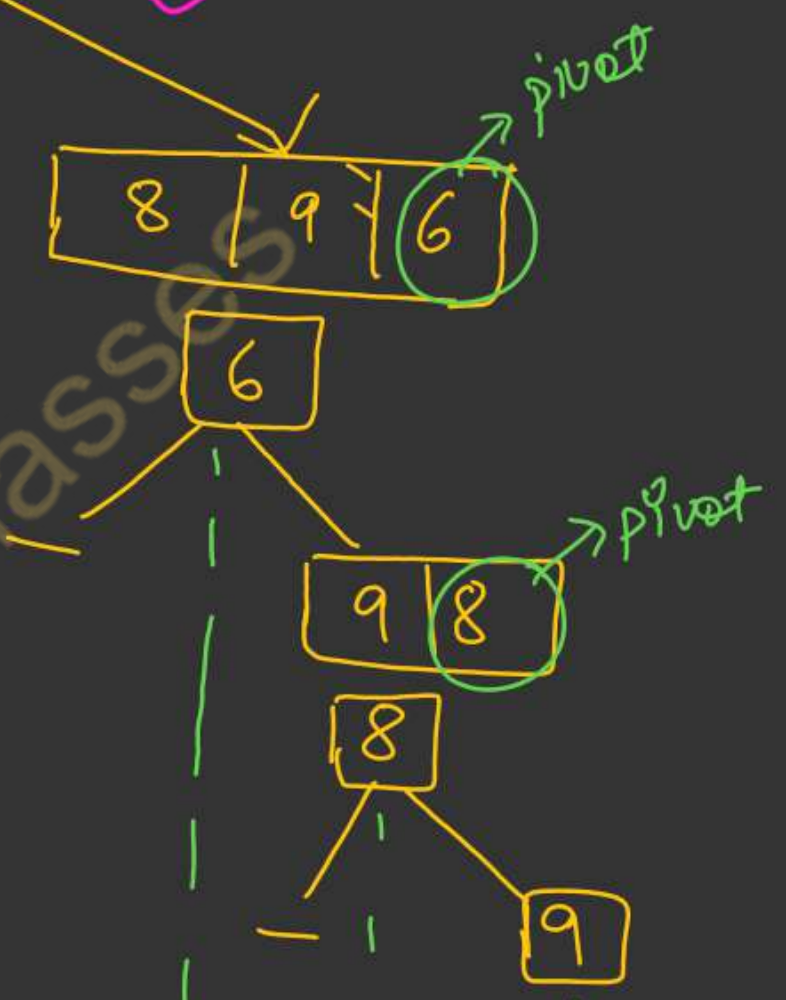
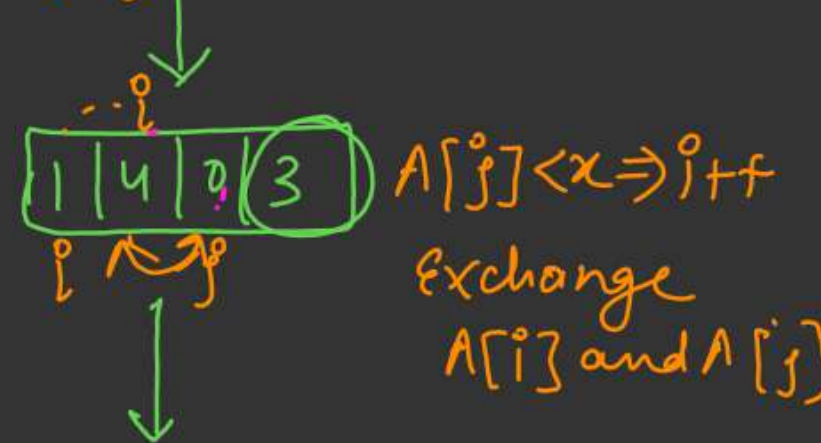
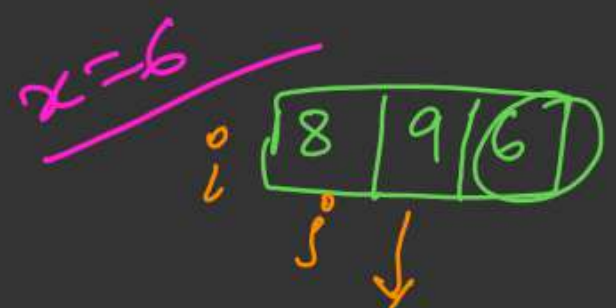
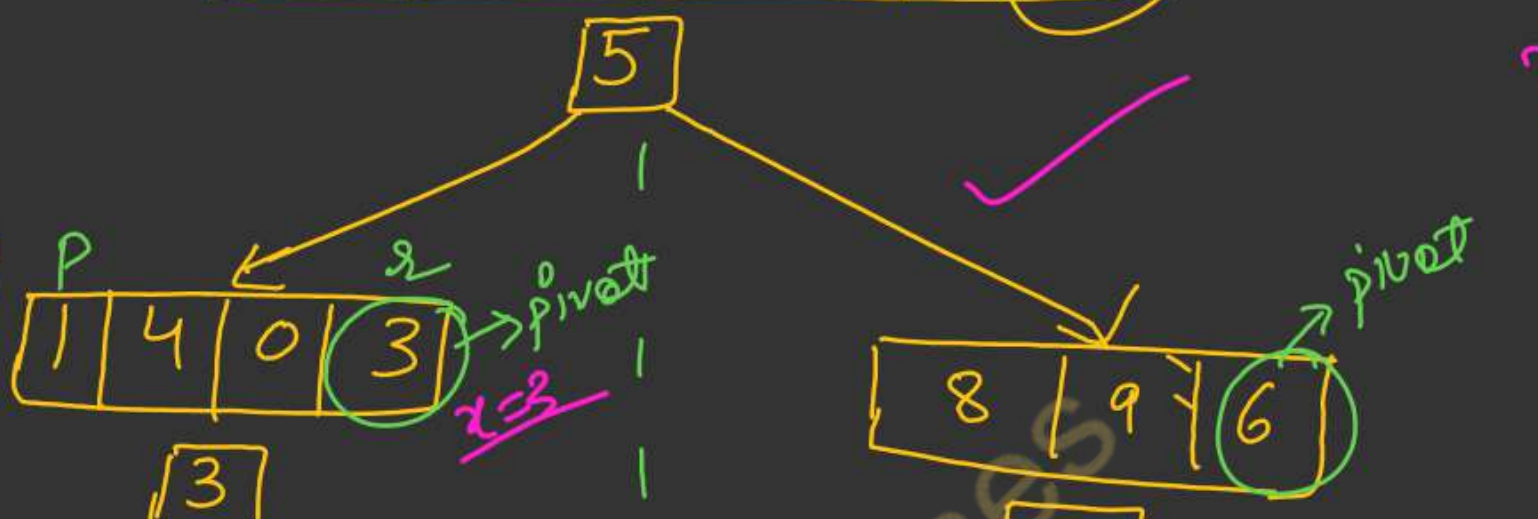
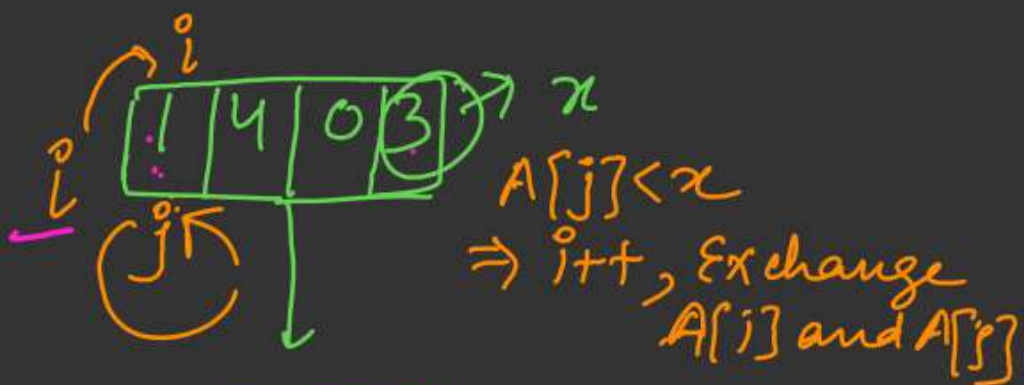
1. If $p < r$ // Base condition
2. Then $q \leftarrow$ PARTITION(A, p, r)
3. QUICK SORT ($A, p, q-1$)
4. QUICK SORT ($A, q+1, r$)



Quick Sort working Example



8 | 1 | 6 | 4 | 0 | 3 | 9 | 5



PARTITION(A, p, r)

 $x = A[r]$ — $O(1)$ $i = p - 1$ — $O(1)$ for $j = p$ to $r - 1$ { if $A[j] \leq x$

{

 $i = i + 1$ Exchange $A[i]$ with $A[j]$

}

}

Exchange $A[i + 1]$ with $A[r]$ return $(i + 1)$

}

Partition part Time Complexity

partition algo Time Complexity

$O(n)$

$\Rightarrow \underline{\underline{O(n)}}$ ✓

Quick Sort Time and space Complexity Analysis

QUICK SORT(A,p,r) $T(n)$

1. If $p < r$ — $O(1)$

2. Then $q \leftarrow \text{PARTITION}(A, p, r)$ — $O(n)$

3. QUICK SORT (A, p, q-1) — $T(n_1)$

4. QUICK SORT (A, q+1, r) — $T(n_2)$

Time complexity:

Best case: $O(n \log n)$

Worst case: $O(n^2)$

Average case: $O(n \log n)$

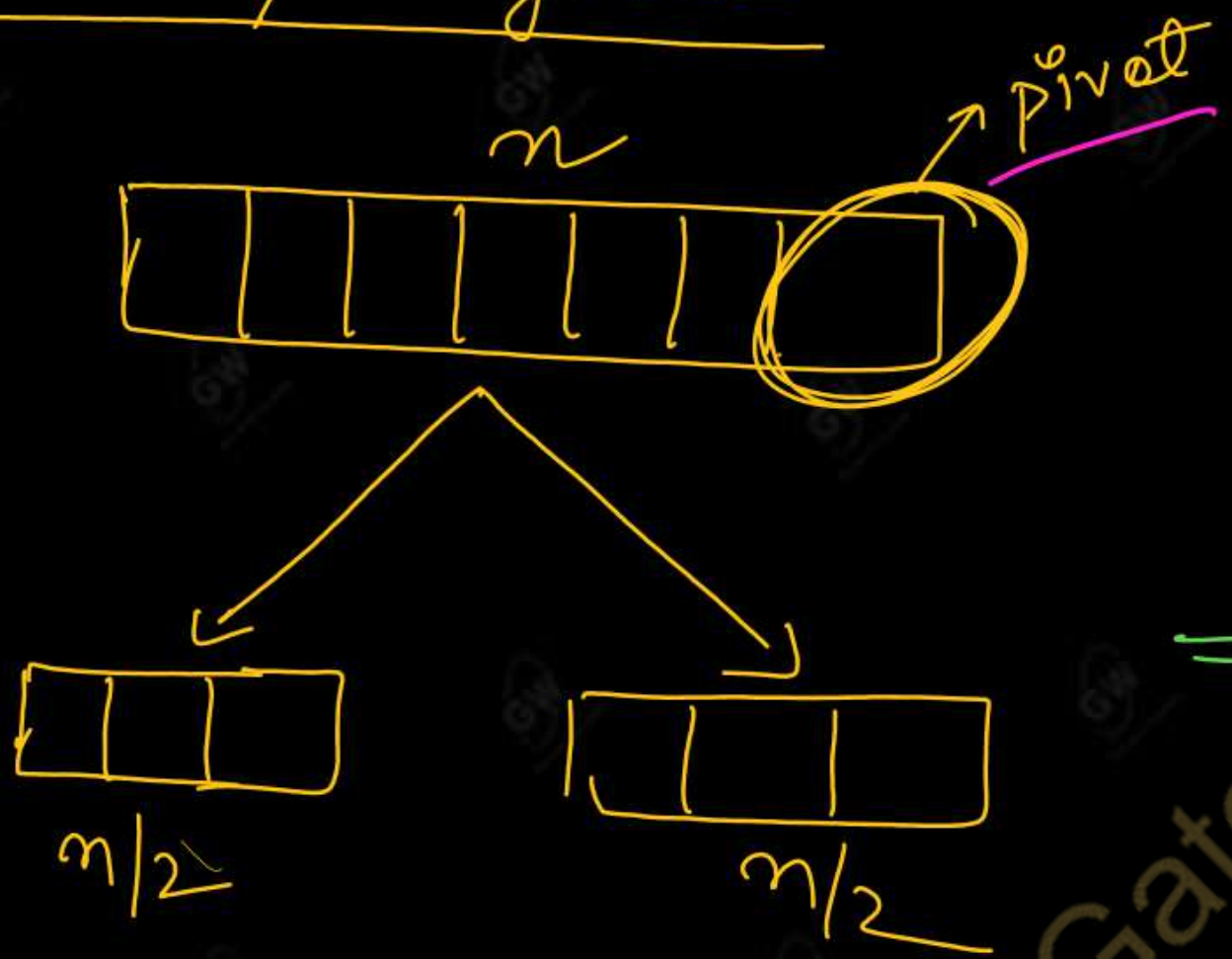
Space complexity:

$O(\log n)$ (best case) ✓

$O(n)$ (worst case) ✓

Quick Sort Time and space Complexity Analysis

Best case / Average case



$$T(n) = T(n_1) + T(n_2) + O(n)$$

$$T(n) = T(n/2) + T(n/2) + O(n)$$

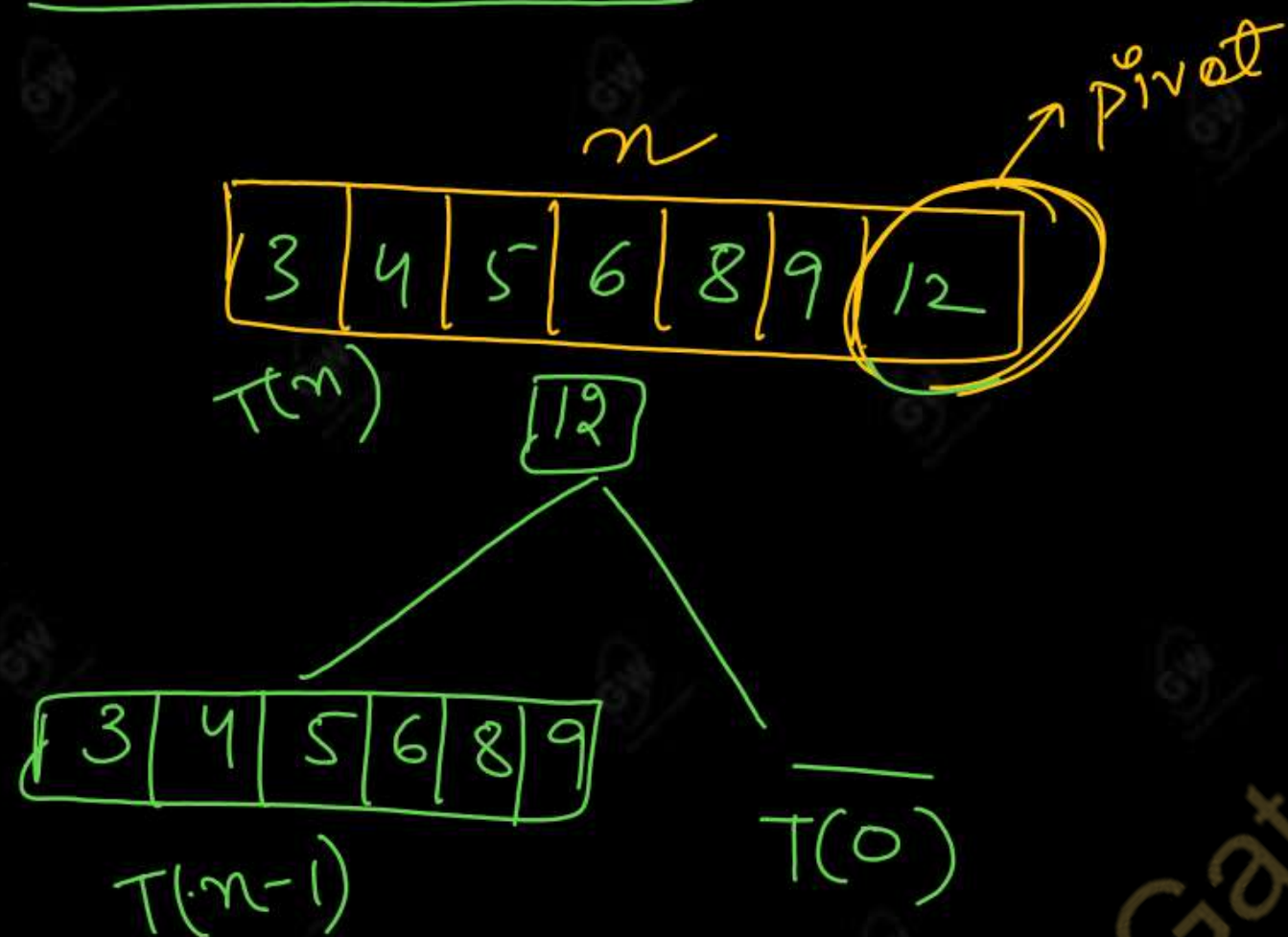
$$T(n) = 2T(n/2) + n$$

$$T(n) = O(n \log n)$$

⇒ Best case / Avg case happens when partition algo divides the array in nearly equal halves.

Quick Sort Time and space Complexity Analysis

Worst case



$\Rightarrow$ worst case happens if array is already sorted.

" It is always better to choose any random number from the array as pivot.

$$T(n) = T(n-1) + T(0) + O(n)$$

$$T(n) = T(n-1) + n$$



$$T(n) = O(n^2)$$

Advantages of Quick Sort:

- Divide-and-conquer approach makes it easier to solve problems.
- Unlike merge sort, it is Cache Friendly as we work on the same array to sort and do not copy data to any auxiliary array(In-Place algo).
- Fastest general purpose algorithm for large data when stability is not required.

Disadvantages of Quick Sort:

- It has a worst-case time complexity of $O(n^2)$, which occurs when the pivot is chosen poorly.
- Not a good choice for small data sets.
- It is not a stable sort
- **In case of already sorted array, if last/first element is chosen as pivot then time complexity is $O(n^2)$. To address this randomized Quick sort is developed**

Why Quick sort is named “quick” ?

It is generally **faster** than many other sorting algorithms, especially for large datasets.

1. **Efficient average performance:**

- Its average time complexity is $O(n \log n)$, which is faster than algorithms like **bubble sort** or **insertion sort** ($O(n^2)$).

2. **Divide and conquer approach:**

- Quickly divides array into smaller parts, reducing the sorting problem size at each step.

3. **In-place sorting:**

- It doesn't require additional memory like merge sort, which makes it faster in practice.

4. **Fewer comparisons on average:**

- By choosing a good pivot, it minimizes the number of comparisons and swaps needed.

So, it's called **Quick Sort** because it can **sort data faster on average** compared to most other sorting methods.

Data Structure

Unit-3 DPP-3

1. Use quick sort algorithm to sort 15,22,30,10,15,64,1,3,9,2. Is it a stable sorting algorithm? Justify.
(AKTU 2021-22)
2. Write an algorithm for Quick sort. Use Quick sort algorithm to sort the following elements: 2, 8, 7, 1, 3, 5, 6, 4. (AKTU 2022-23)
3. How the choice of pivot element effects the running time of quick sort algorithm? (AKTU 2023-24)
4. Illustrate the operation of Quick sort on the array, $A = (19, 4, 7, 24, 52, 56, 87)$ (AKTU 2018-19)
5. Why is quick sort named as quick? Show the steps of quick sort on the 10 following set of elements: 25, 57, 48, 37, 12, 92, 86, 33. (AKTU 2019-20) Assume the first element of the list to be the pivot element.
6. What is quick sort? Sort the given values using quick sort; present all steps/iterations :
38, 81, 22, 48, 13, 69, 93, 14, 45, 58, 19, 72? (AKTU 2016-17)



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-3 Lecture-4

Today's Target

- Heap Sort
- AKTU PYQs
- DPP



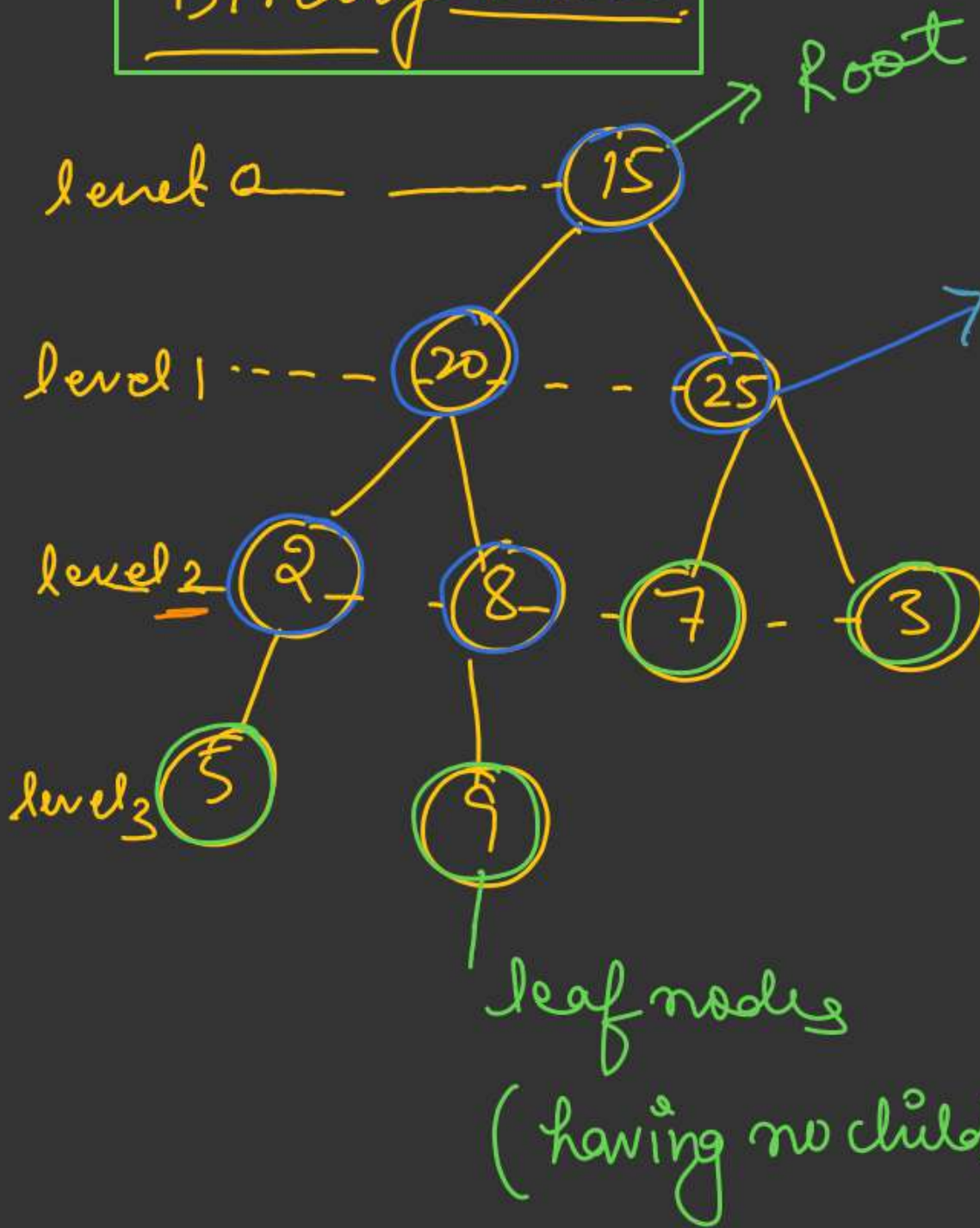
By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

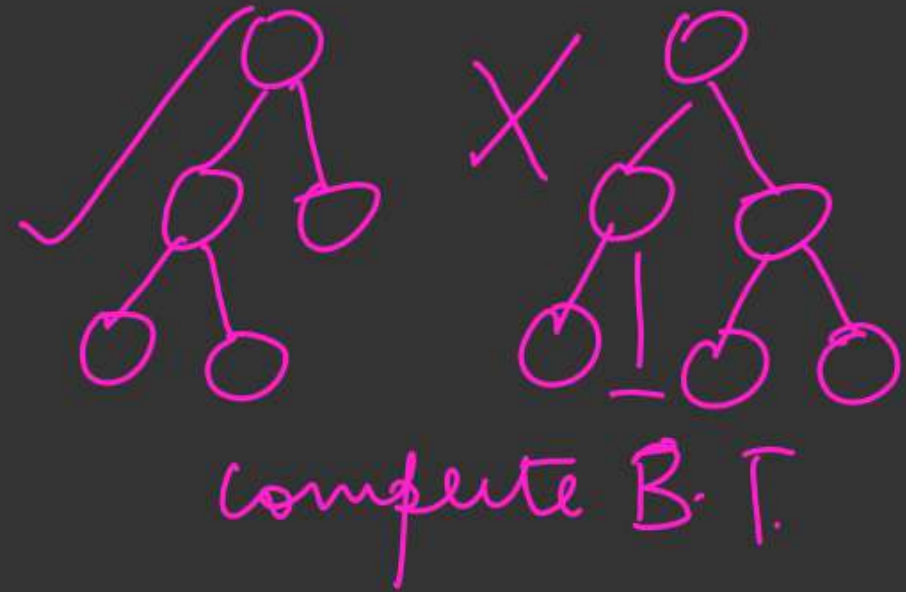
- M.Tech Gold Medalist
- Net Qualified

Binary Tree

→ Each node can have max 2-children
No. of children → 0/1/2



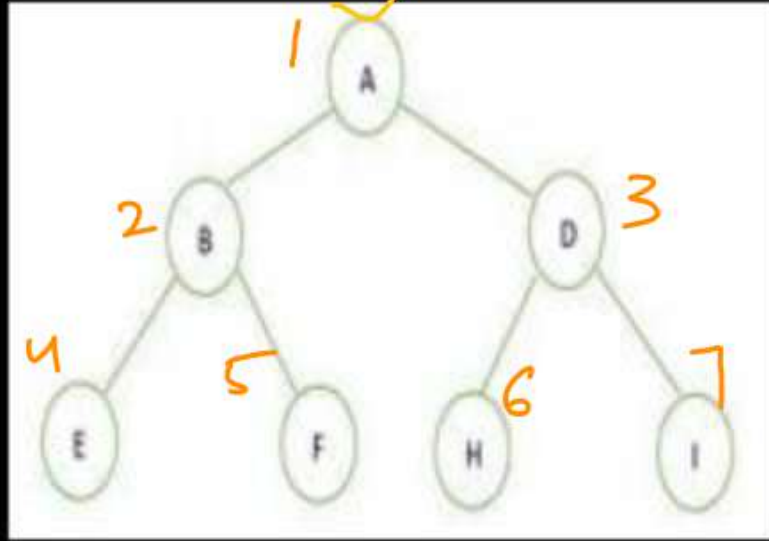
Internal nodes
(at least one child node is present)



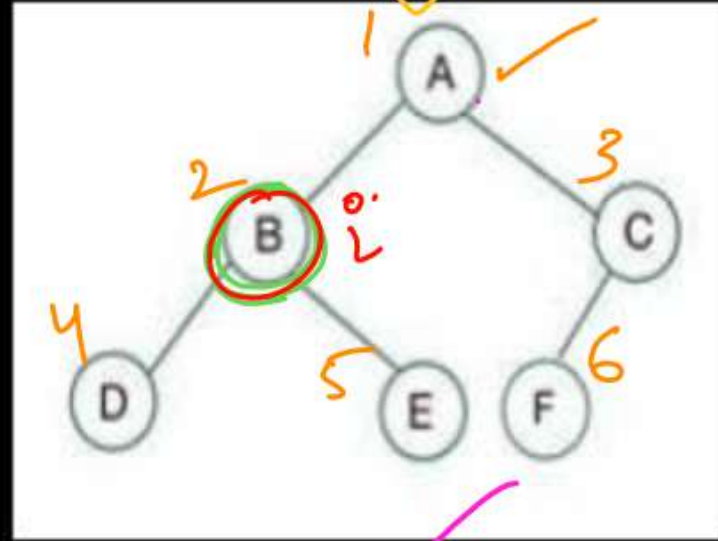
→ Max no. of nodes at each level = $2^{\text{level no.}}$

→ Tree can be represented using array / linked list.

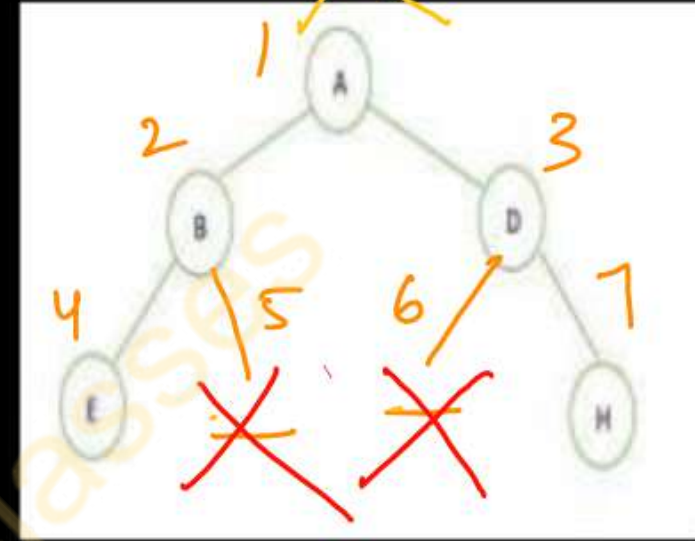
Complete Binary Tree: A tree in which except for the last level all the other level has maximum nodes and the nodes are lined up from left to right side.



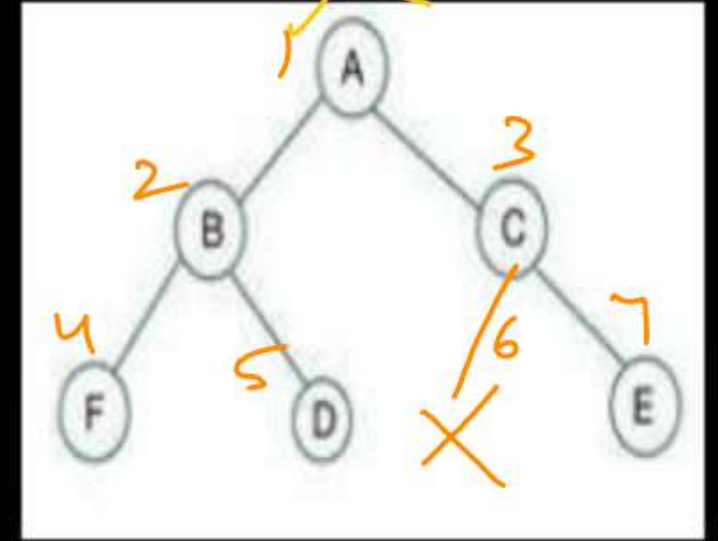
1	2	3	4	5	6	7
A	B	D	E	F	H	I



1	2	3	4	5	6	7
A	B	C	D	E	F	-



1	2	3	4	5	6	7
A	B	D	E	-	-	H



1	2	3	4	5	6	7
A	B	C	F	D	-	E

for Node (B) $i=2$
 Index of parent of B $\Rightarrow \lfloor \frac{i}{2} \rfloor = \lfloor \frac{2}{2} \rfloor = 1$
 Index of left child of B $= 2 \times i \Rightarrow 2 \times 2 = 4$
 Right child $= 2 \times i + 1 = 2 \times 2 + 1 = 5$

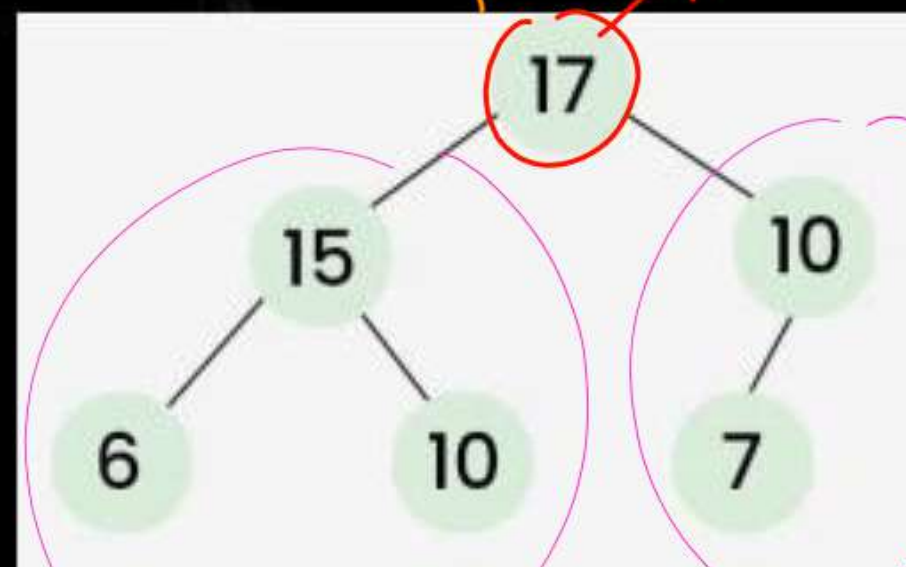
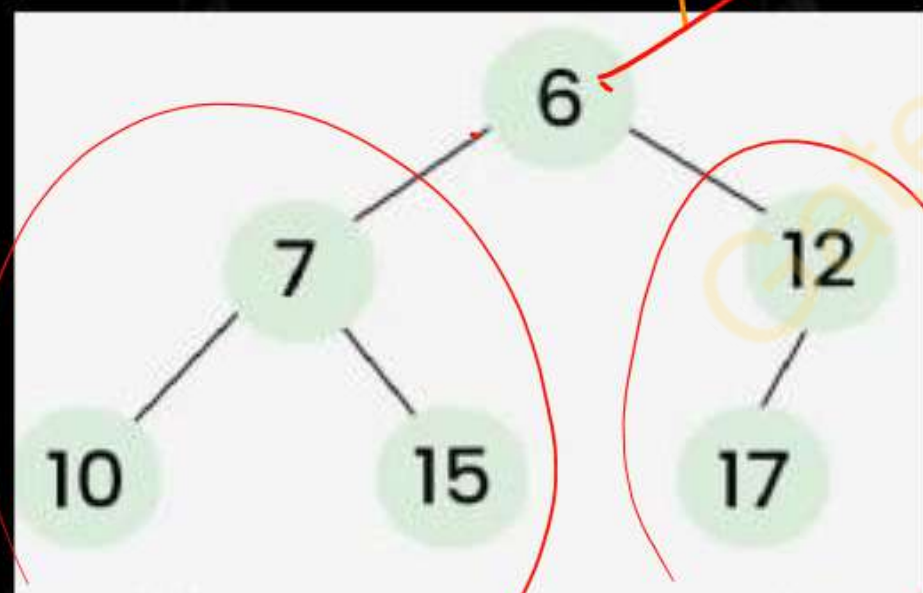
Using arrays, we find parents/children as follows
 (Assuming we start indexing at 1):

Node Index: i
 If parent is it
 index: $\text{floor}(i/2)$
 left child is at: $2 \times i$
 right child is at: $2 \times i + 1$

Binary Heap: It is a complete Binary Tree which is used to store data efficiently to get the max or min element based on its structure.

Min Heap: Key at the root must be minimum among all keys present in Heap. for all nodes i , excluding the root, $A[\text{PARENT}(i)] \leq A[i]$

Max Heap: Key at the root must be maximum among all keys present in Heap. for all nodes i , excluding the root, $A[\text{PARENT}(i)] \geq A[i]$



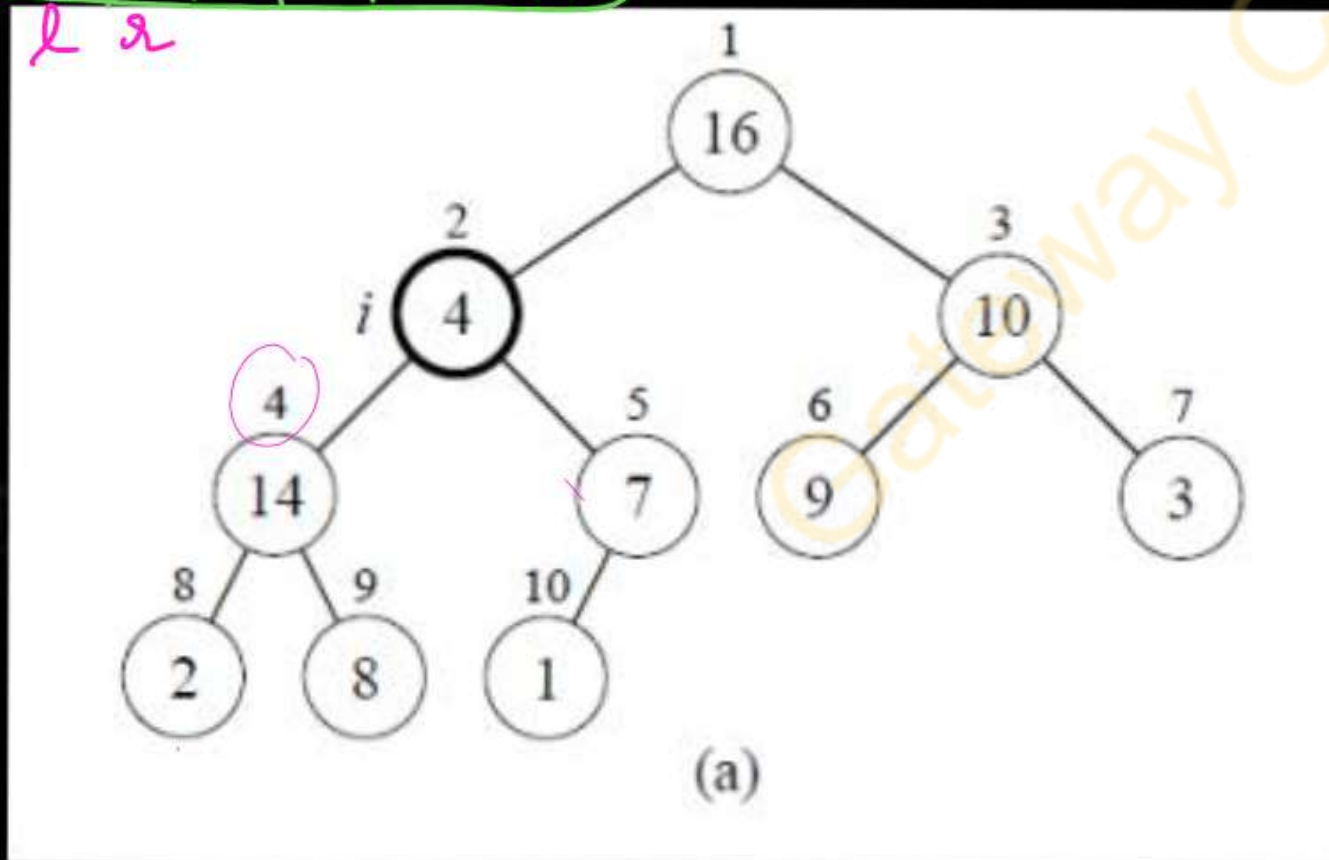
Maintaining the heap property

- MAX-HEAPIFY is used to maintain the max-heap property.
- Before MAX-HEAPIFY, $A[i]$ may be smaller than its children.
- Assume left and right subtrees of i are max-heaps.
- After MAX-HEAPIFY, subtree rooted at i is a max-heap.

A

1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

 $n=10$



MAX-HEAPIFY (A, i, n)

{ $l = \text{Left}(i);$ $2 \times i^0$

$r = \text{Right}(i);$ $2 \times i^0 + 1$

if ($l \leq n \ \&\& \ A[l] > A[i]$) $\top$
if it has a left child.
 $\text{largest} = l;$

else $\text{largest} = i;$

if ($r \leq n \ \&\& \ A[r] > A[\text{largest}]$)
if it has right child.
 $\text{largest} = r;$

if ($\text{largest} \neq i$)

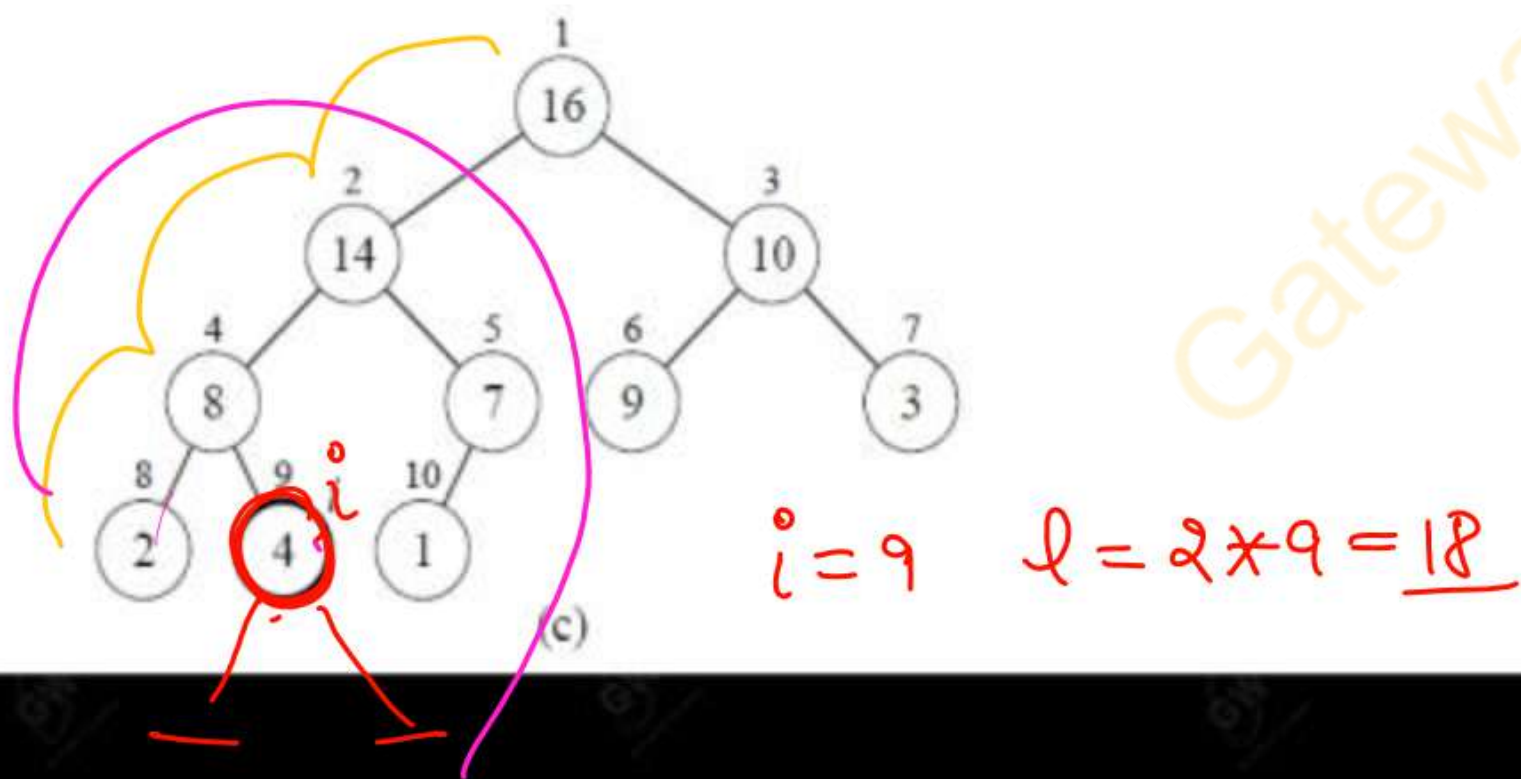
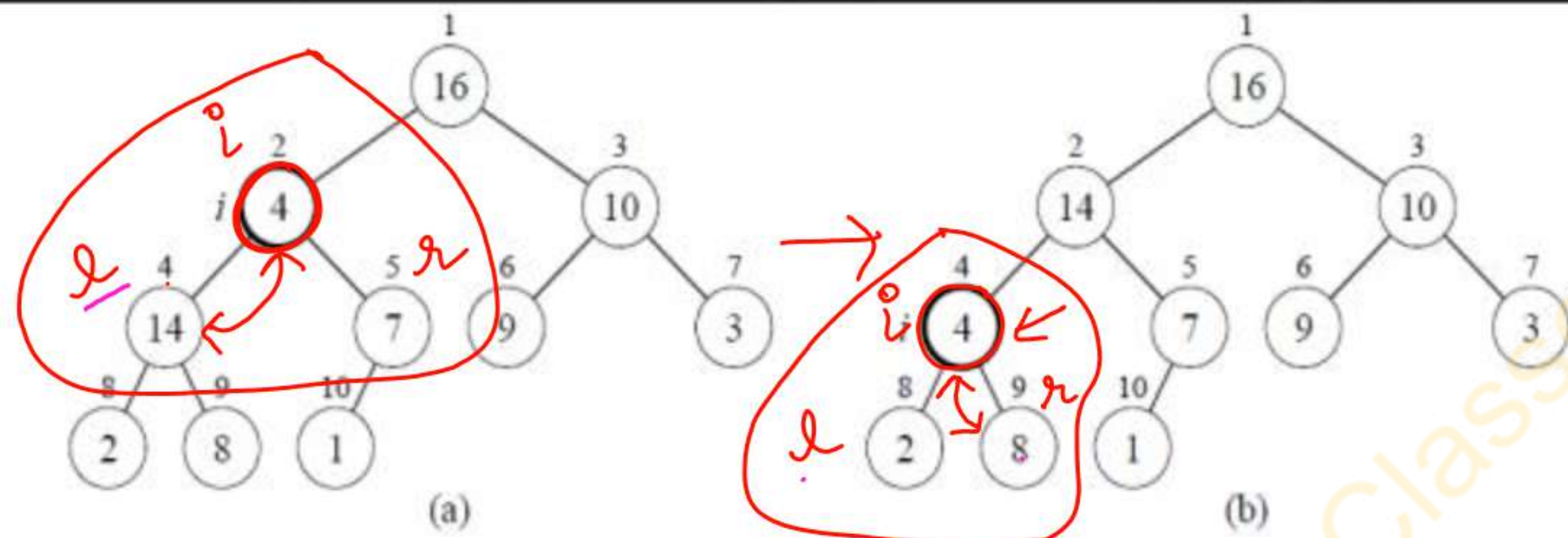
{ Swap($A[i], A[\text{largest}]$)

MAX-HEAPIFY($A, \text{largest}, n$);

}

}

Illustration of MAX-HEAPIFY



Time Complexity of MAX-HEAPIFY

Heap is almost-complete binary tree, so must process $O(\log n)$ levels, with constant work at each level (comparing 3 items and maybe swapping 2).

Time Complexity: $O(\log n)$

Building a Max-heap from list of elements

Time Complexity $\Rightarrow O(n)$

BUILD-MAX-HEAP(A, n)

{

for $i = \text{floor}(n/2)$ down to 1 ✓

MAX-HEAPIFY(A, i, n)

}

$n = 10$

$\text{floor}(\frac{10}{2})$

$\Rightarrow 5$

A

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

INPUT

MAX-HEAP

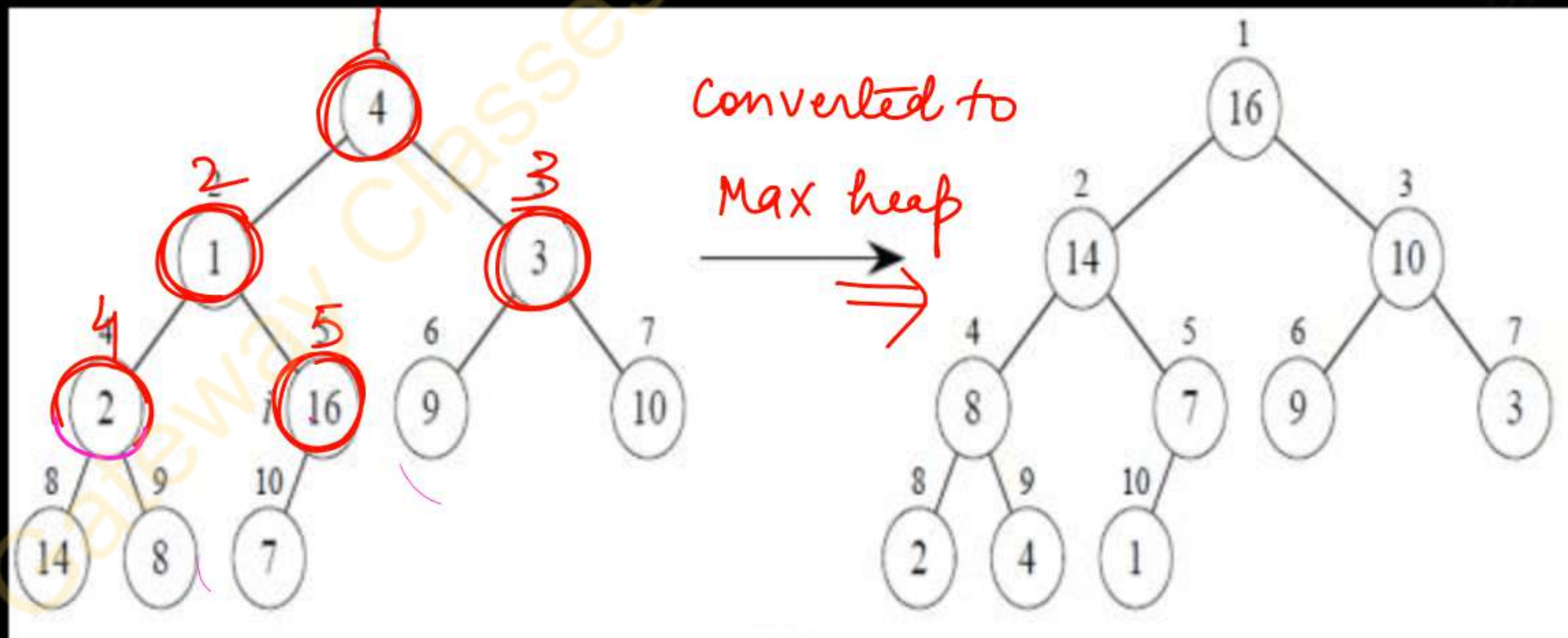


Illustration of Building a Max-heap

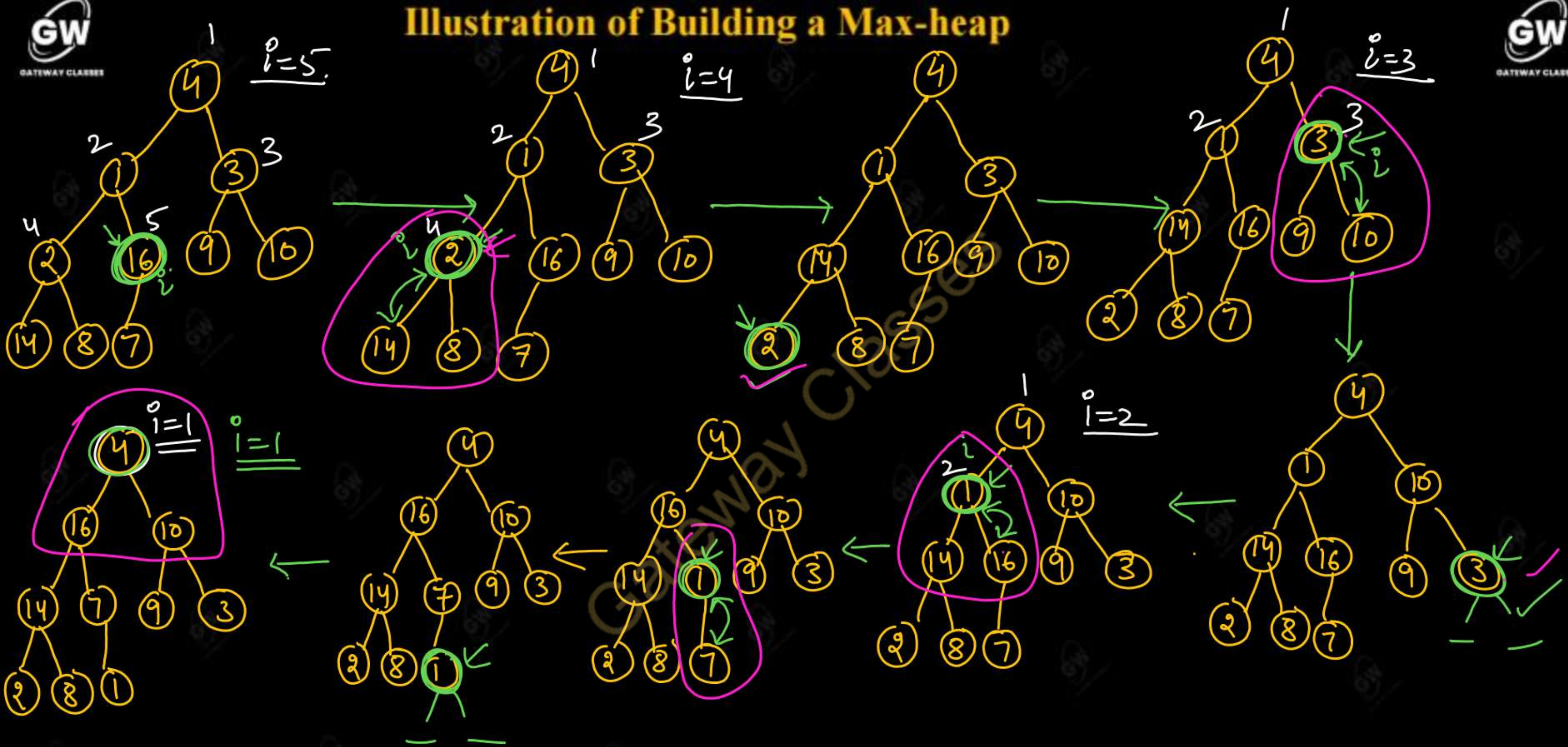
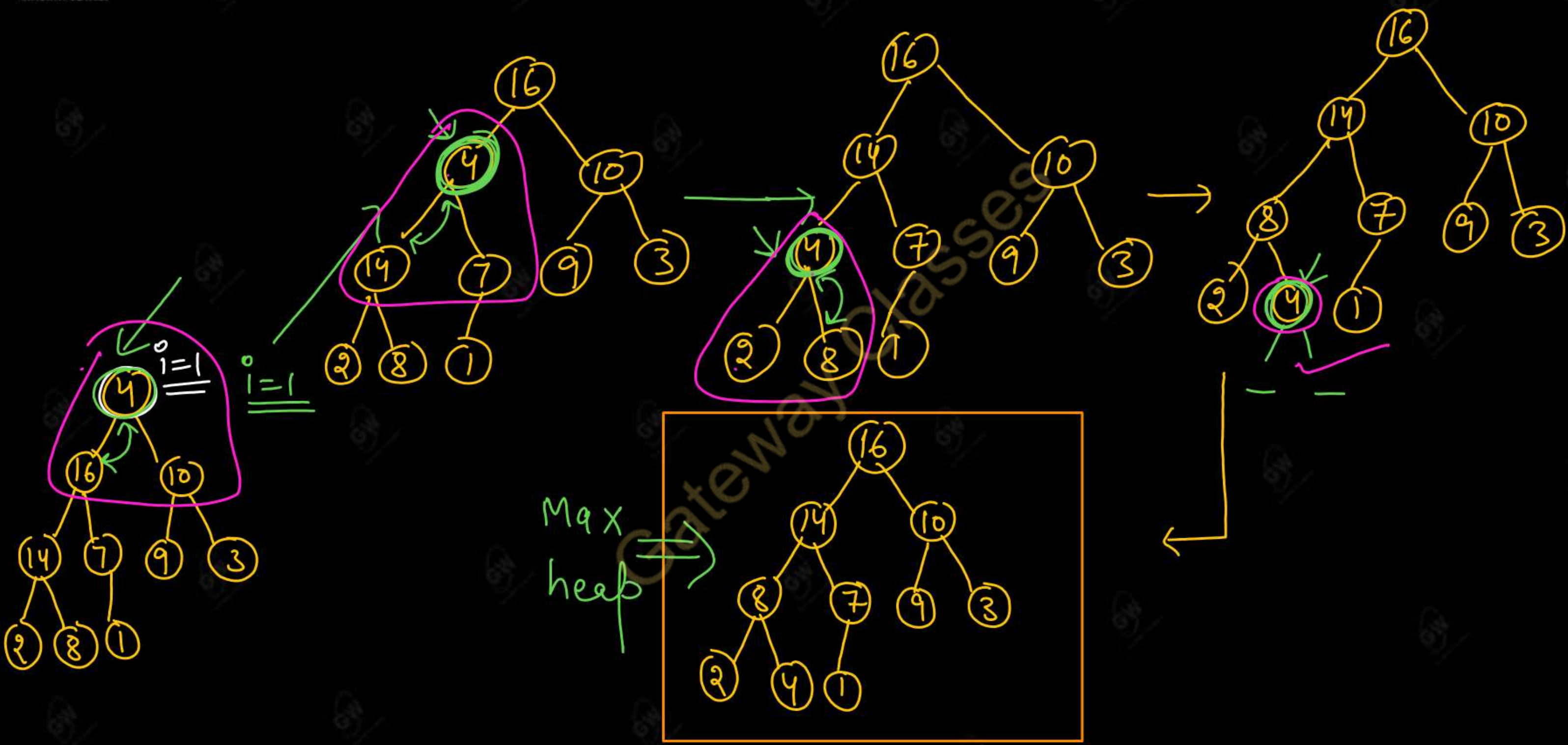


Illustration of Building a Max-heap



Heap Sort

HEAP-SORT(A, n)

{

Build-Max-Heap(A, n) ✓

for $i \leftarrow n$ down to 2

exchange $A[1] \leftrightarrow A[i]$

heap-size[A] = heap-size[A] - 1

MAX-HEAPIFY(A, 1, i-1) ✓

call Max-heapify on the root

➤ Advantages of Heap Sort: In-Place algo

➤ Disadvantages of Heap Sort: It is not a stable sort

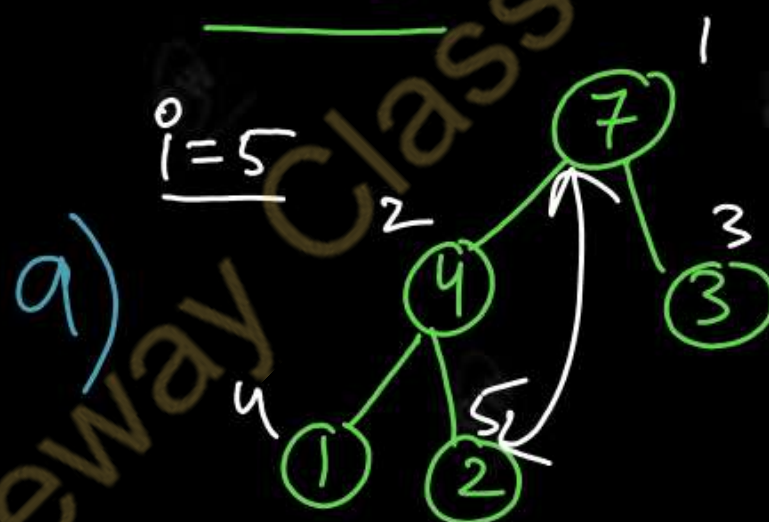
in ascending order

Example: Sort the given array using heap sort.

	1	2	3	4	5
A	7	4	3	1	2

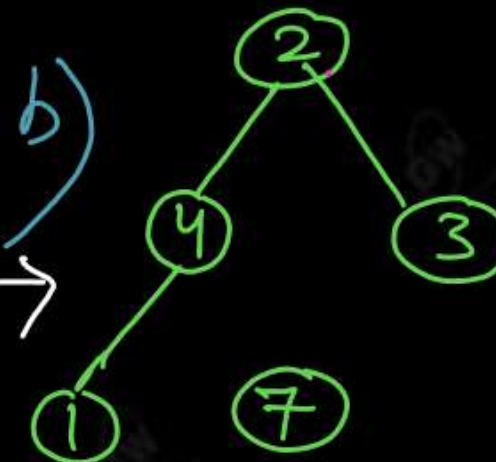
$i = 5$ to 2

$n = 5$



7	4	3	1	2
---	---	---	---	---

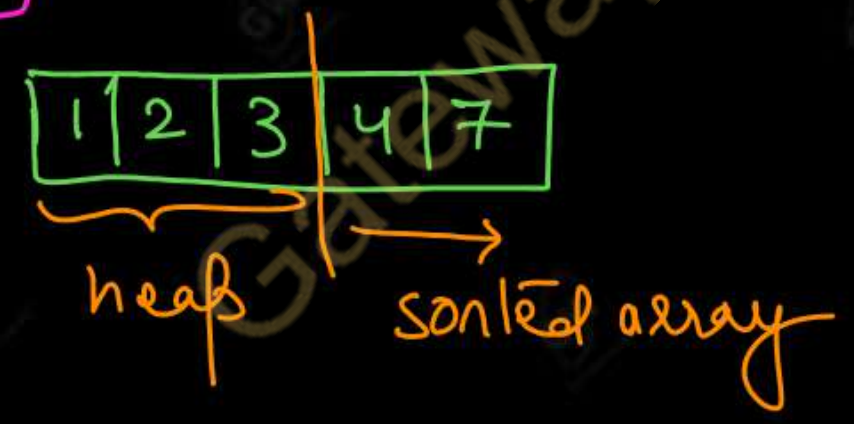
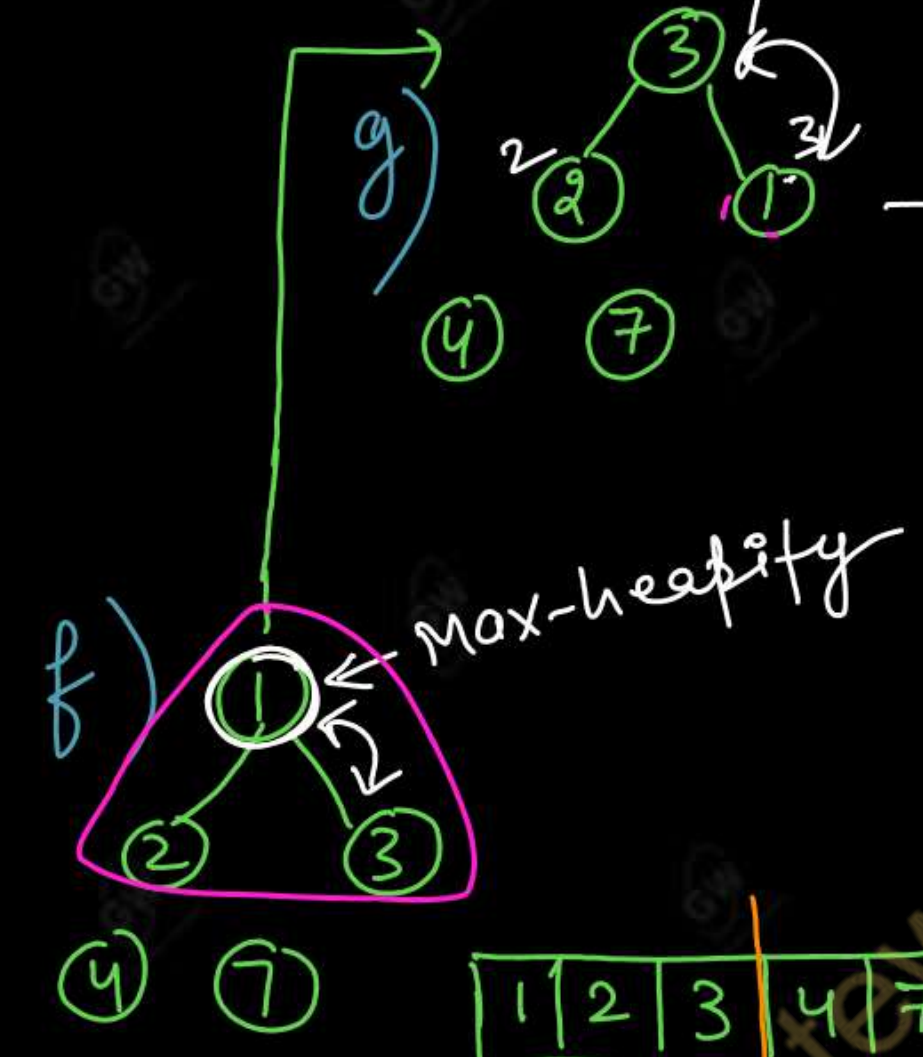
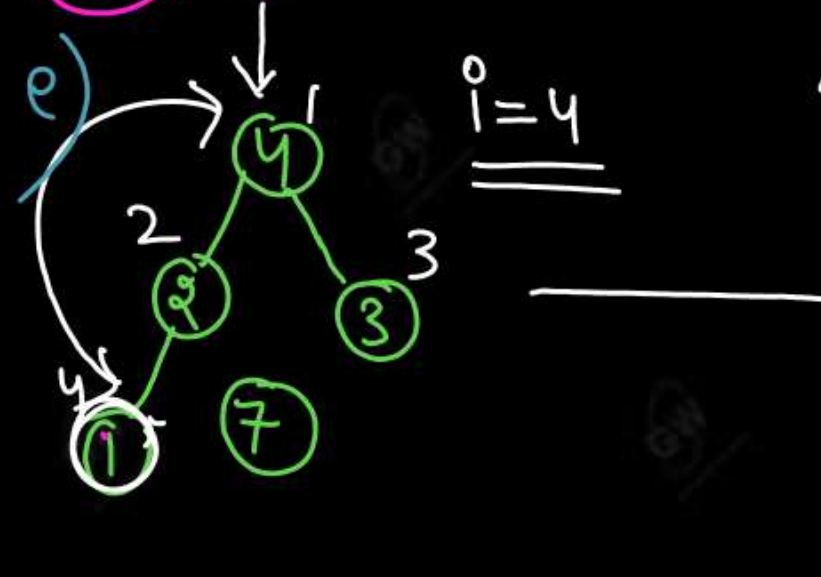
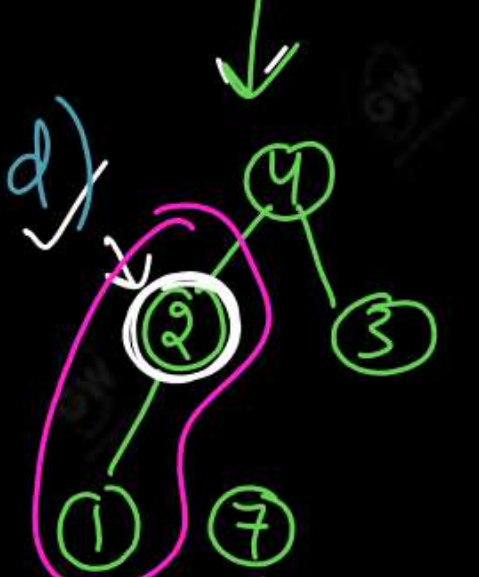
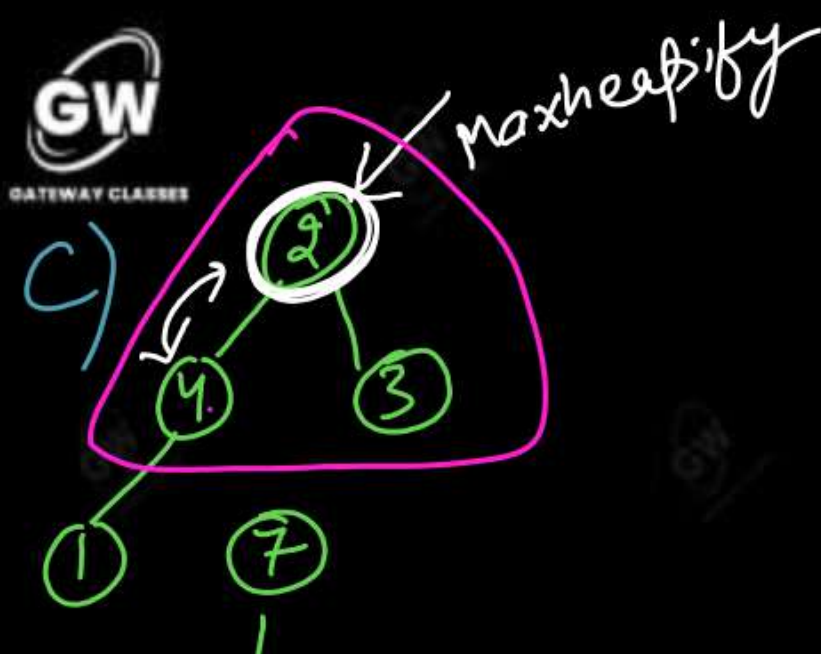
heap



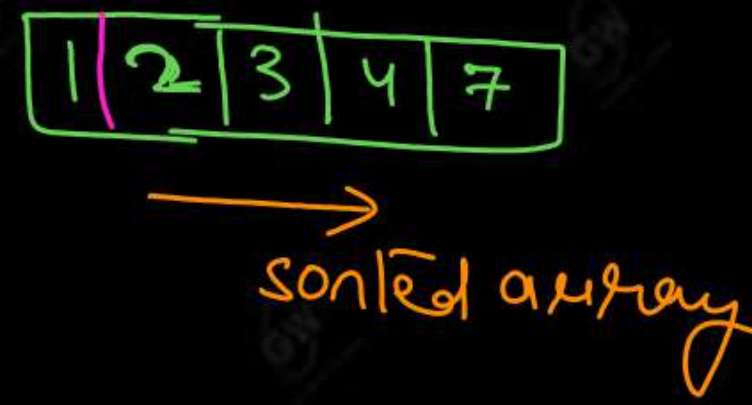
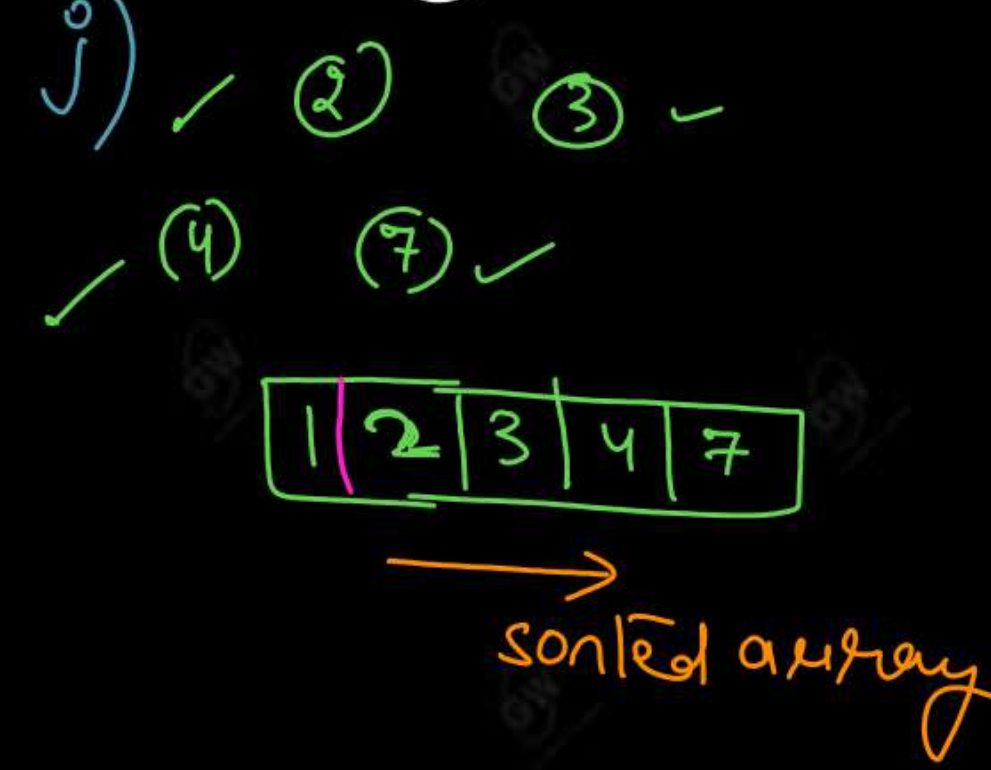
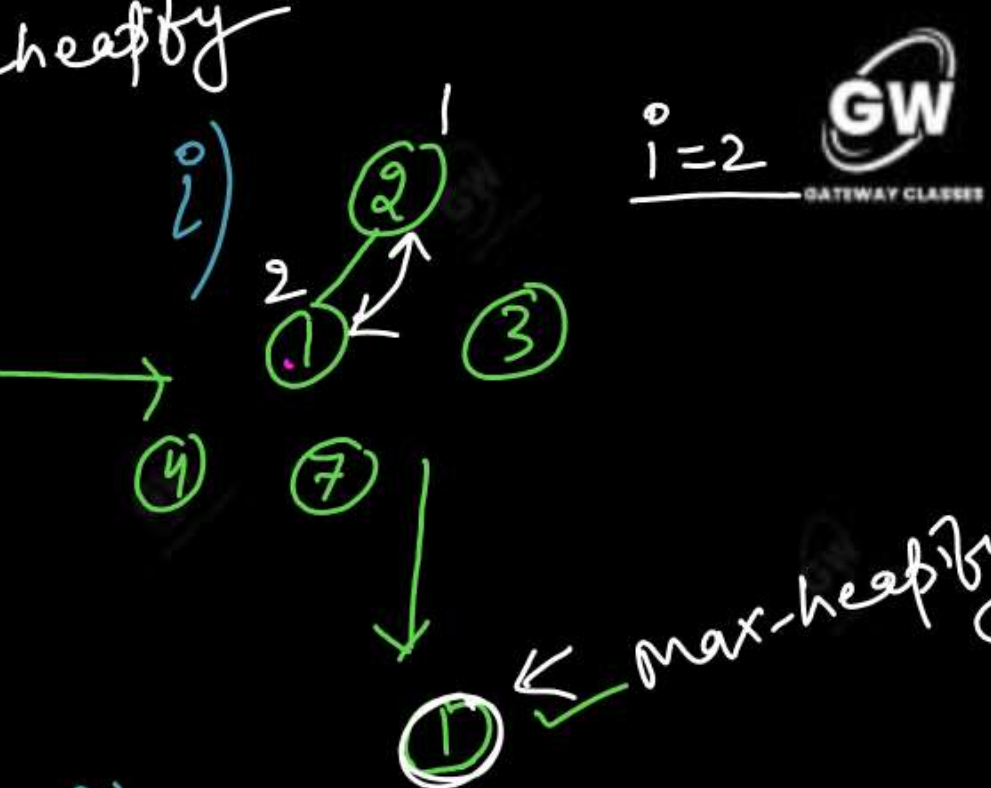
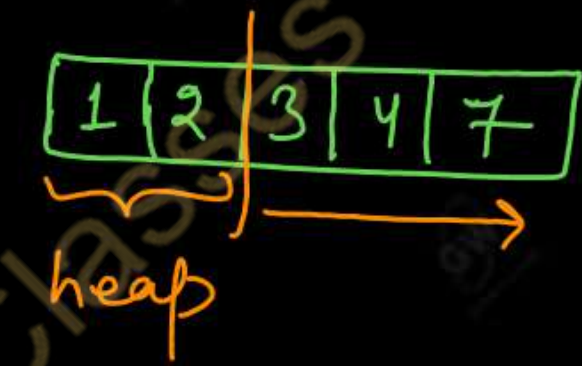
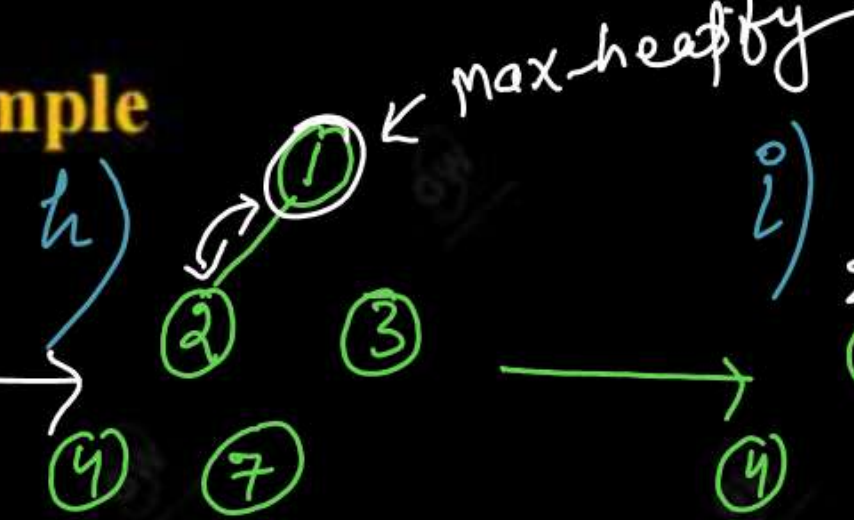
2	4	3	1	7
---	---	---	---	---

heap

Heap Sort Example



$i=3$



Heap sort Time Complexity

HEAP-SORT(A, n)

{

Build-Max-Heap(A, n) $\rightarrow O(n)$

for i $\leftarrow$ n down to 2

exchange A[1] $\leftrightarrow$ A[i]

heap-size[A] = heap-size[A] - 1

MAX-HEAPIFY (A, 1, i-1) $O(\log n)$

}

$$T(n) = O(n) + O(n \log n)$$

$$T(n) = O(n \log n)$$

Time complexity:

Best case: $O(n \log n)$

Worst case: $O(n \log n)$

Ans cor 2
Average case: $O(n \log n)$

Space complexity: heapsort is an in-place designed sorting algorithm so Therefore, we don't use any extra space, so

Space Complexity is: $O(1)$

Data Structure

Unit-3 DPP-4

1. Examine the minimum number of interchanges needed to convert the array
90, 20, 41, 18, 13, 11, 3, 6, 8, 12, 7, 71, 99 into a maximum heap. (AKTU 2020-21)
2. Write short notes on min heap. (AKTU 2020-21)
3. Write an algorithm for Heap Sort. Use Heap sort algorithm, sort the following sequence:
18, 25, 45, 34, 36, 51, 43, and 24. (AKTU 2022-23)



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-3 Lecture-5 (PART-1 + PART-2)

Today's Target

- Problems based on Heap Sort } PART-1
- Radix Sort }
- Linear Search(Sequential Search) } PART-2
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

Q: Examine the minimum number of interchanges needed to convert the array

90, 20, 41, 18, 13, 11, 3, 6, 8, 12, 7, 71, 99 into a maximum heap. (AKTU 2020-21)

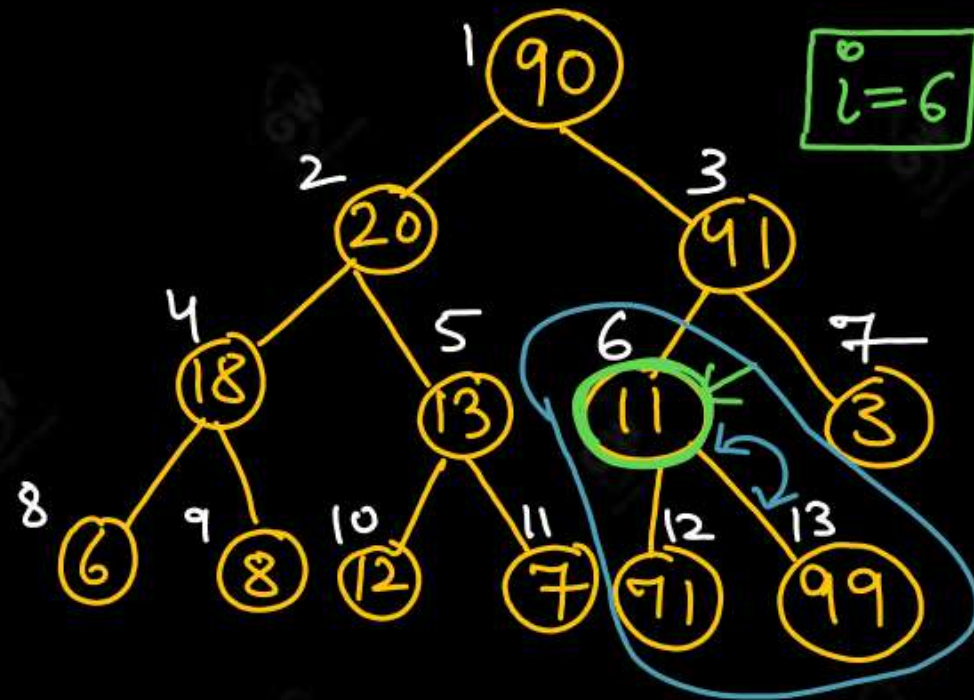
90	20	41	18	13	11	3	6	8	12	7	71	99
----	----	----	----	----	----	---	---	---	----	---	----	----

1 2 3 4 5 6 7 8 9 10 11 12 13

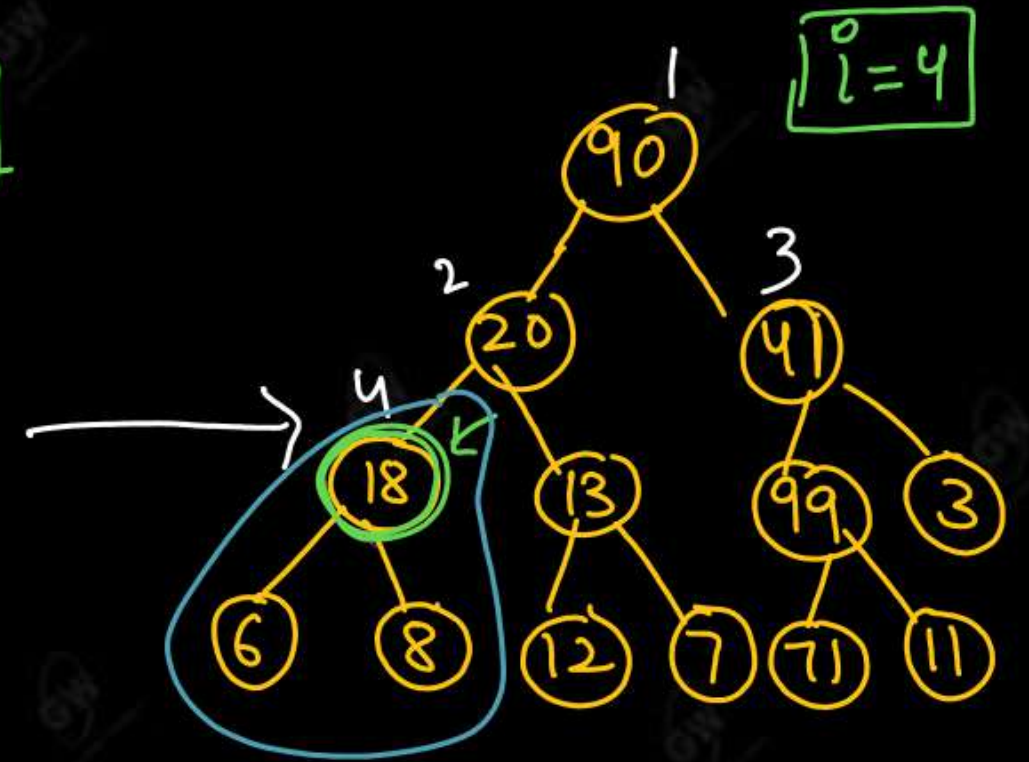
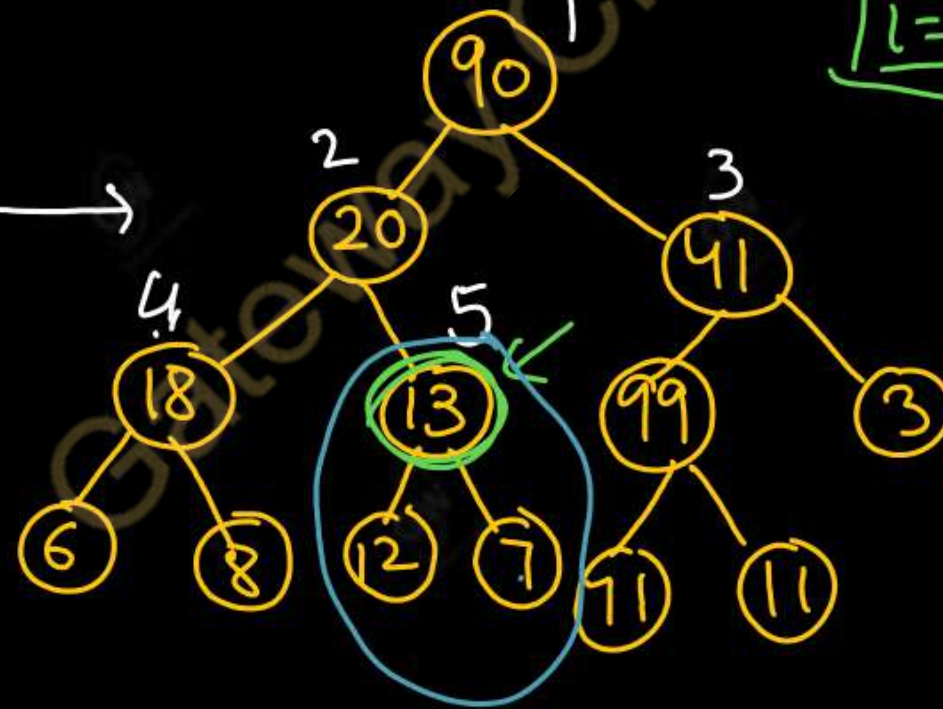
$$n = 13$$

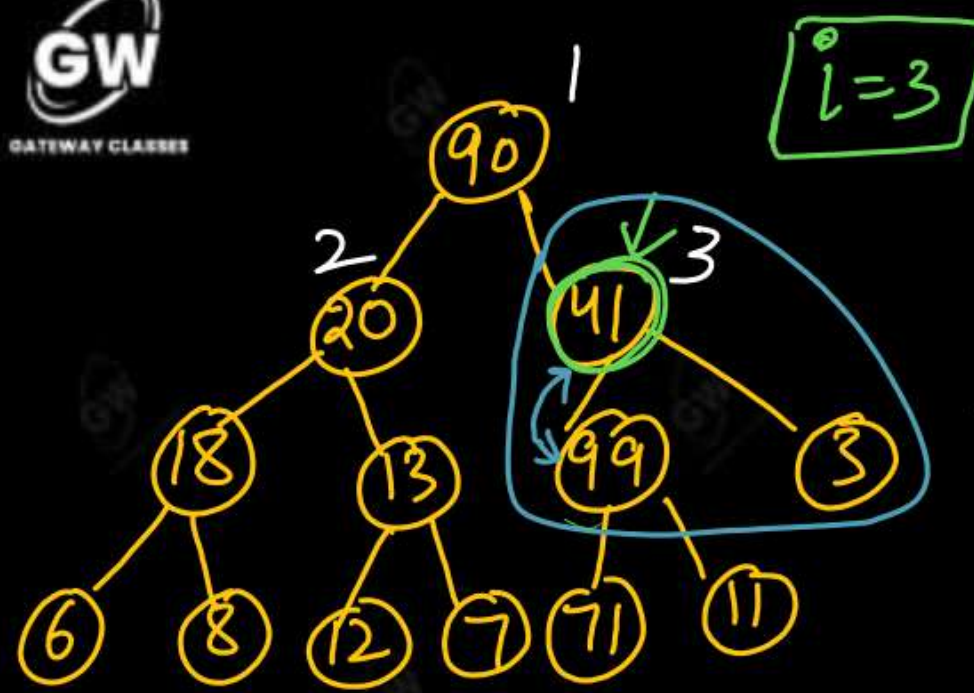
$$i = \lfloor n/2 \rfloor = \frac{13}{2} = 6$$

$$i = 6, 5, 4, 3, 2, 1$$

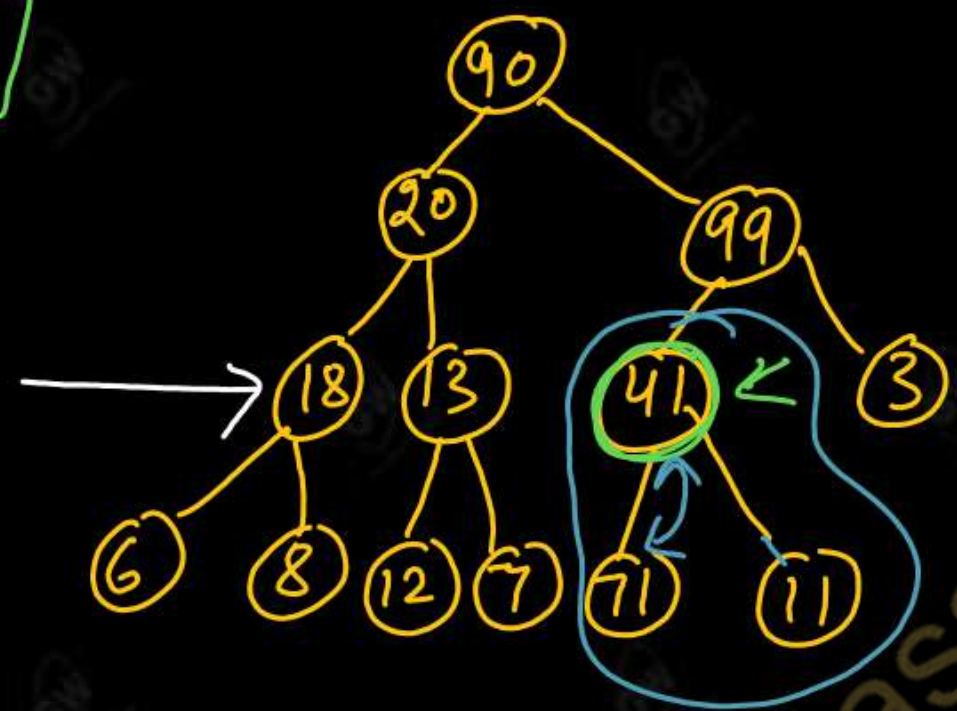


1-swap



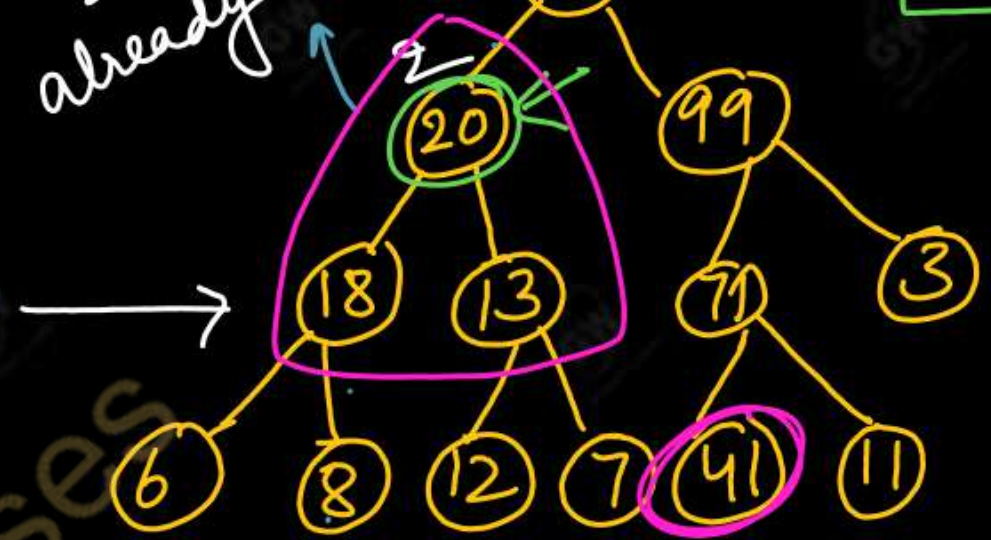


$i=3$



subtree is already a max heap

$i=2$



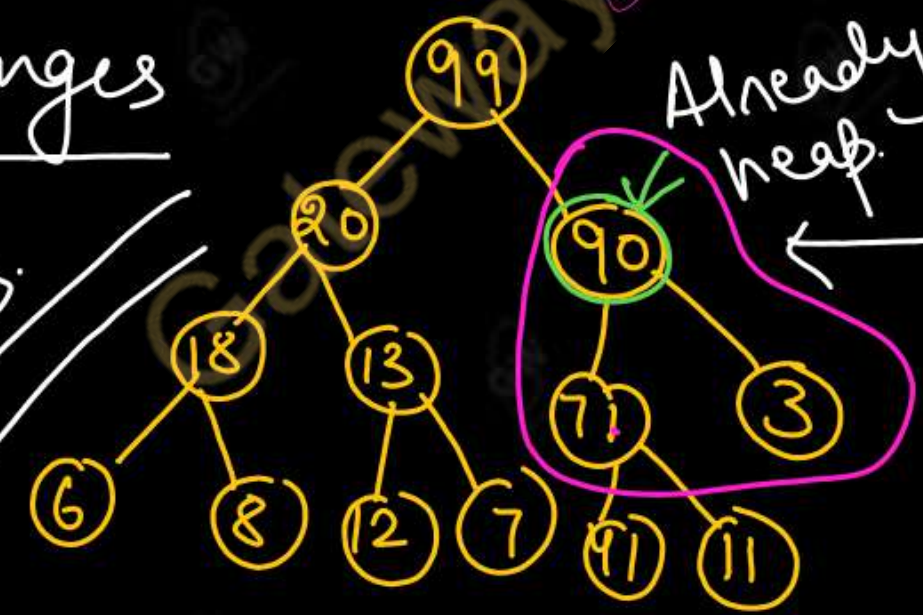
1-swap

1-swap

minimum no. of Interchanges

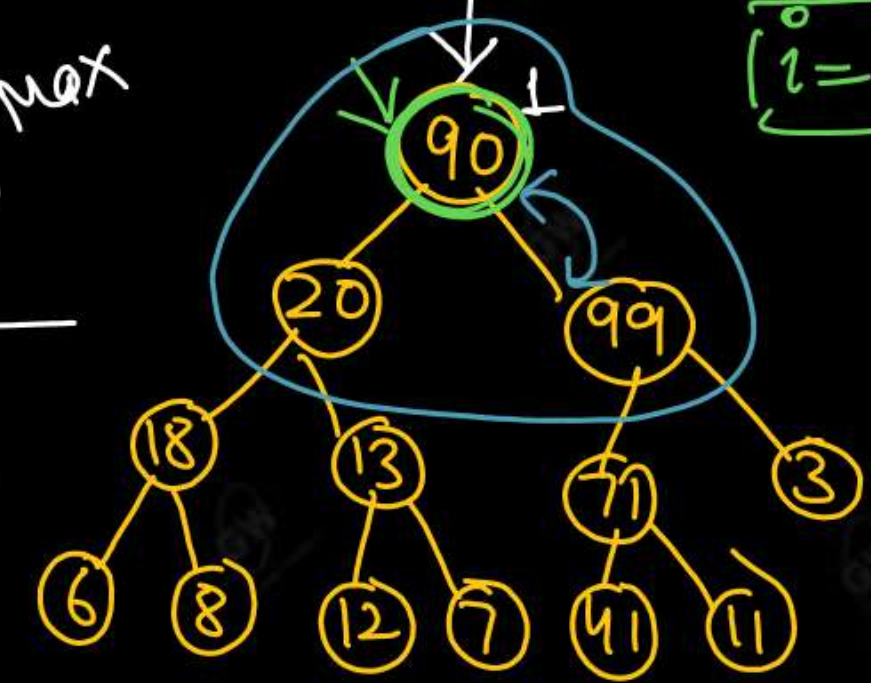
4

Ans.



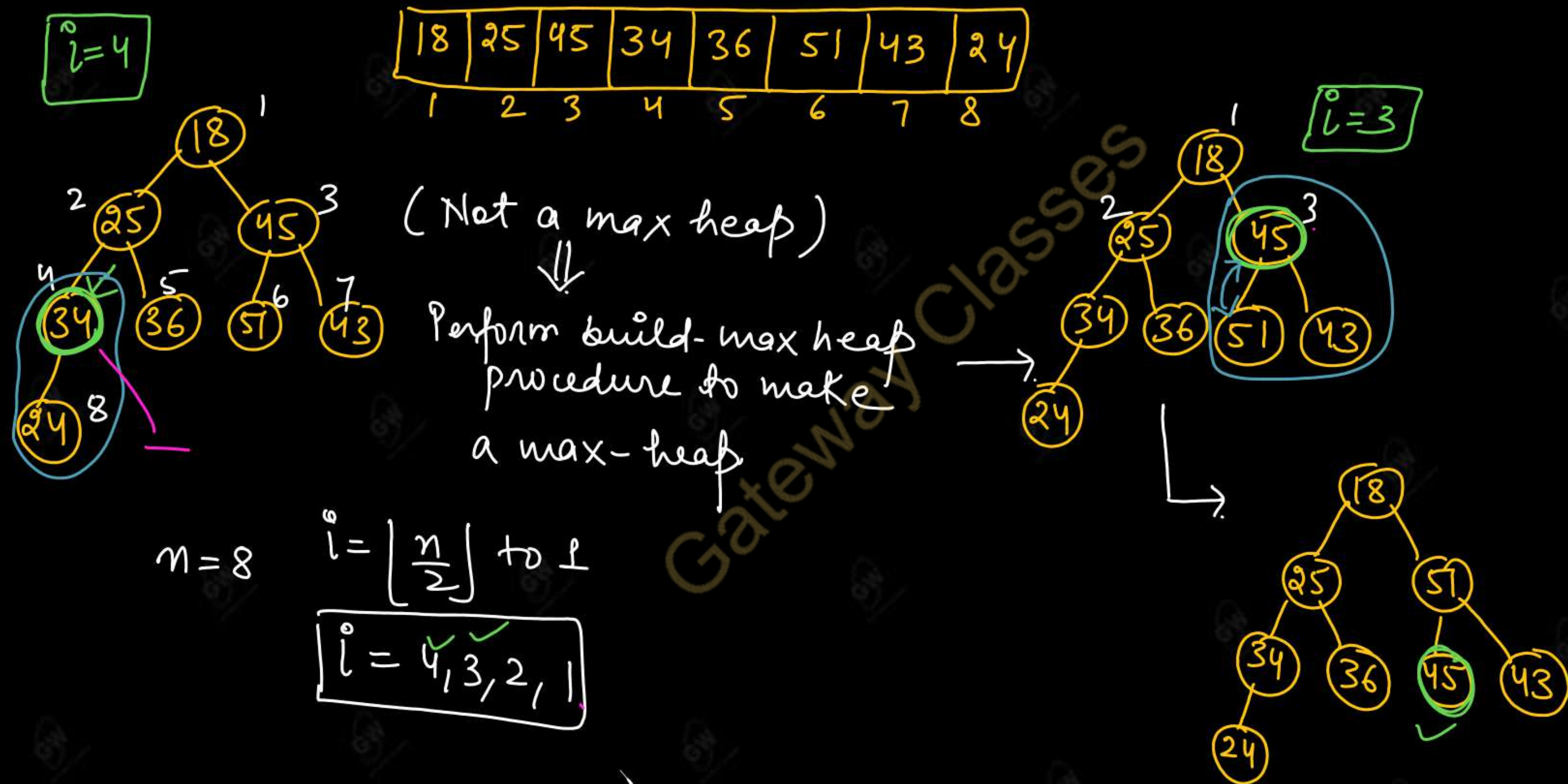
Already max heap

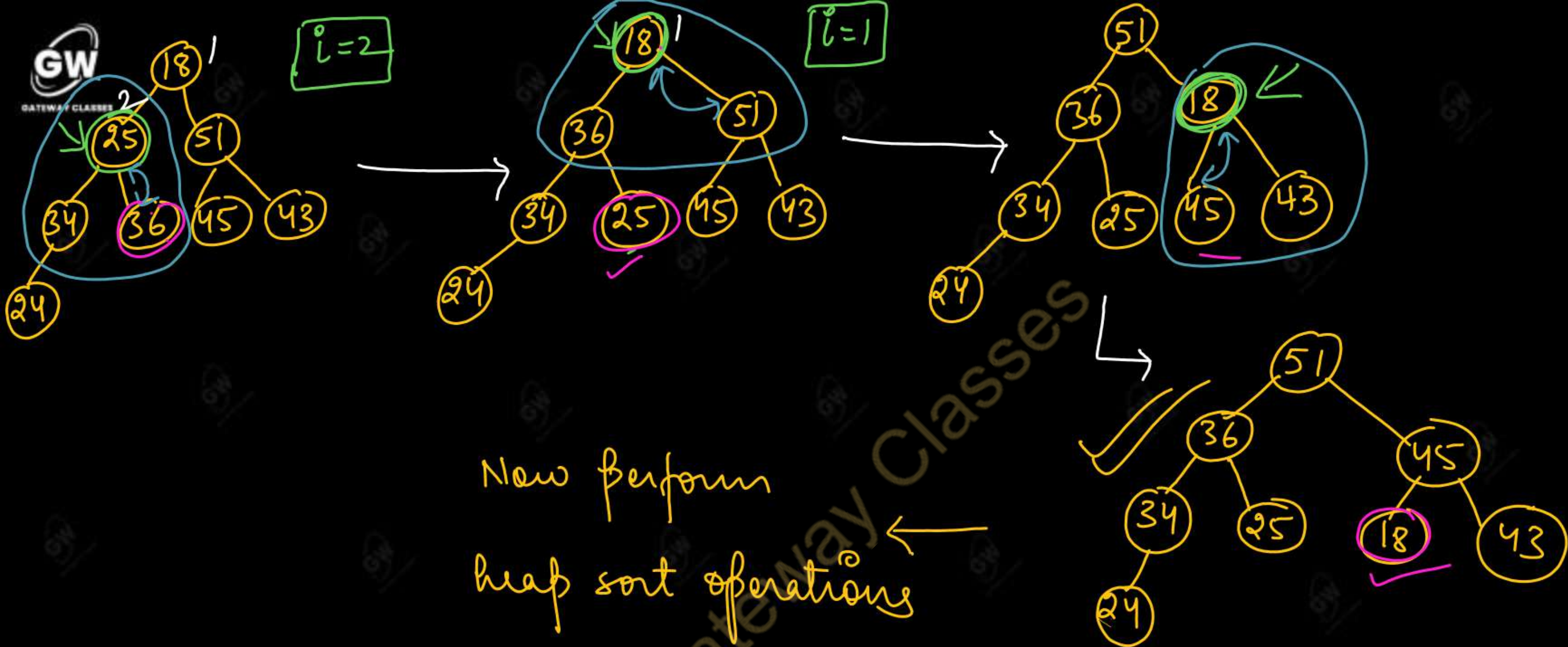
$i=1$



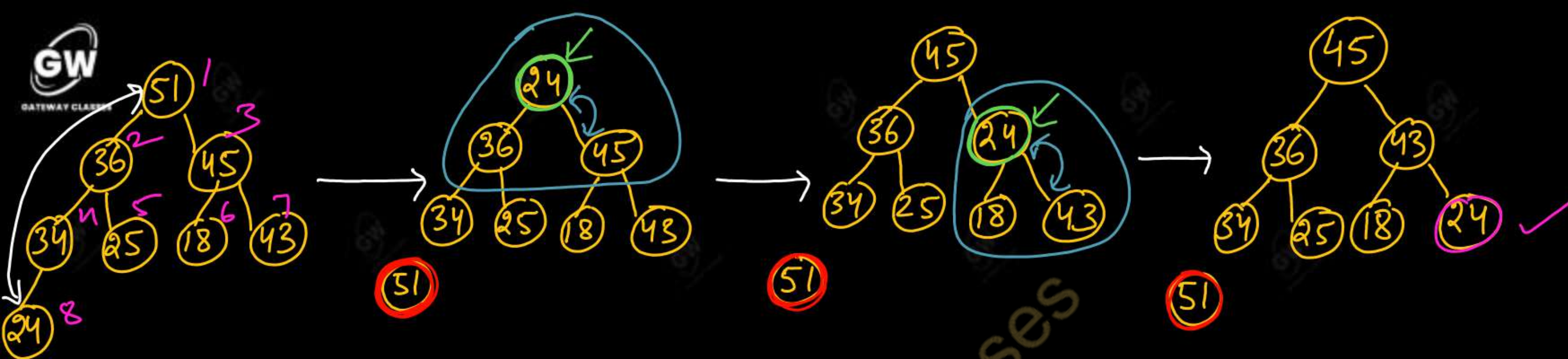
1-swap

Q: Write an algorithm for Heap Sort. Use Heap sort algorithm, sort the following sequence: 18, 25, 45, 34, 36, 51, 43, and 24. (AKTU 2022-23)





Max-heap

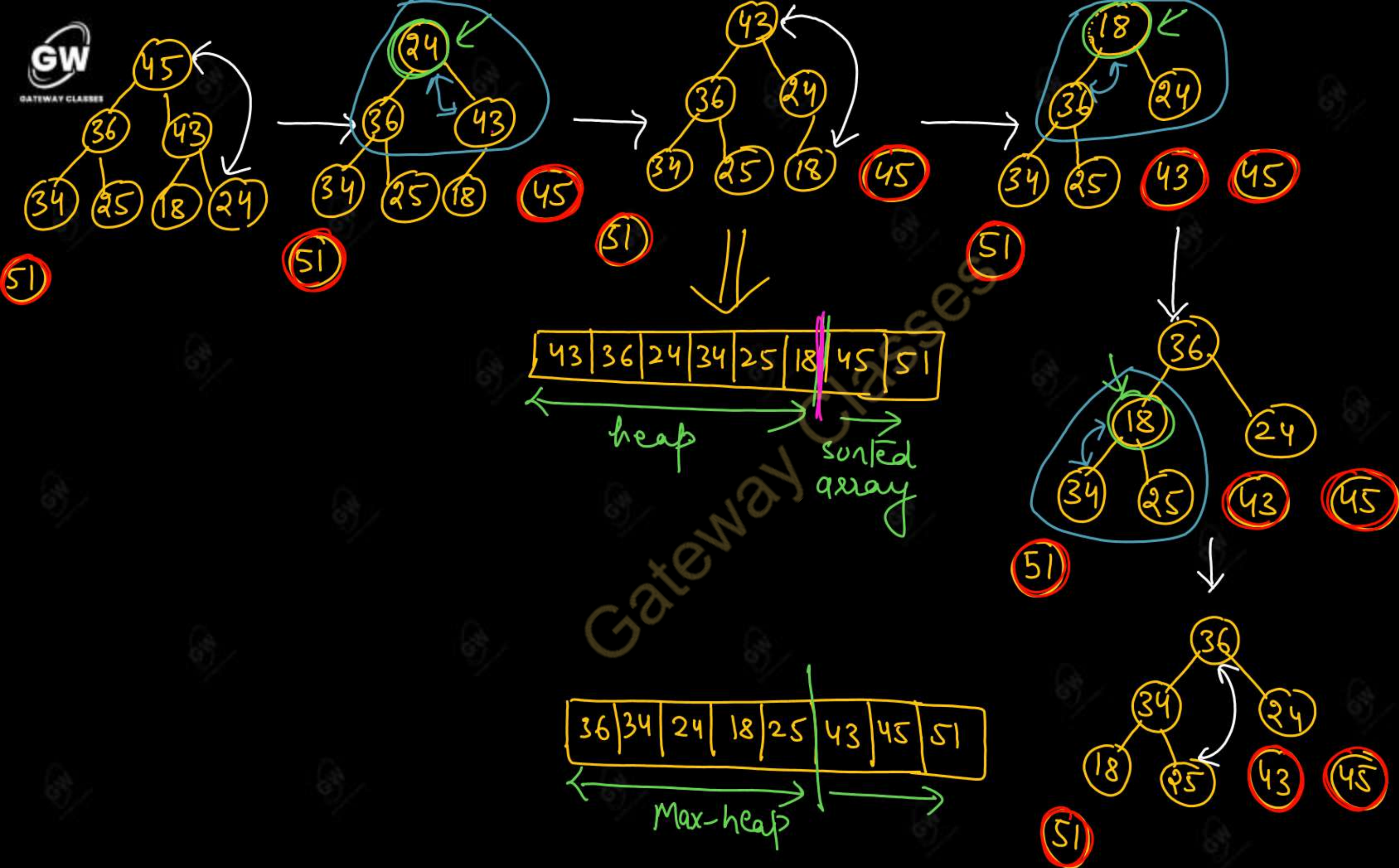


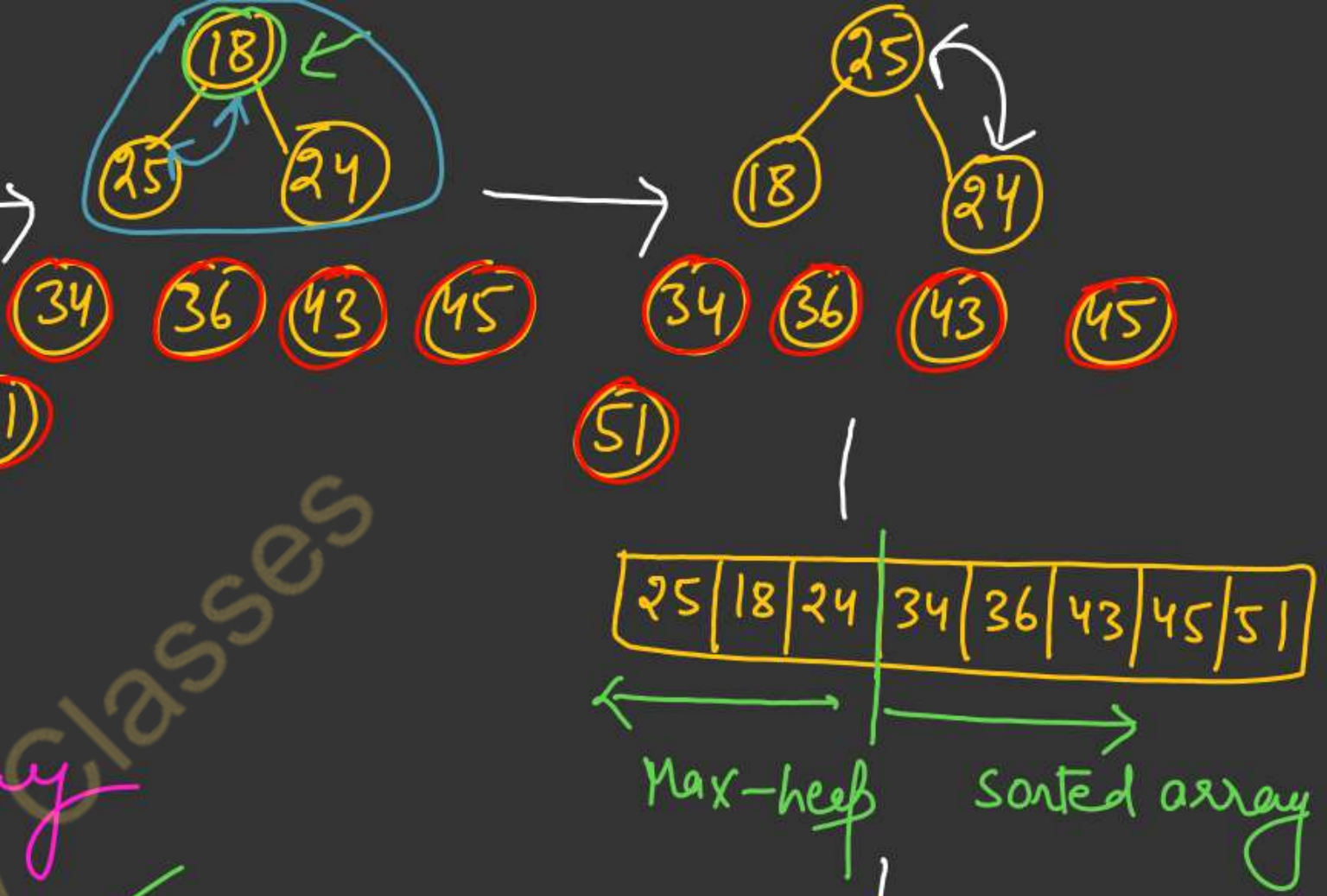
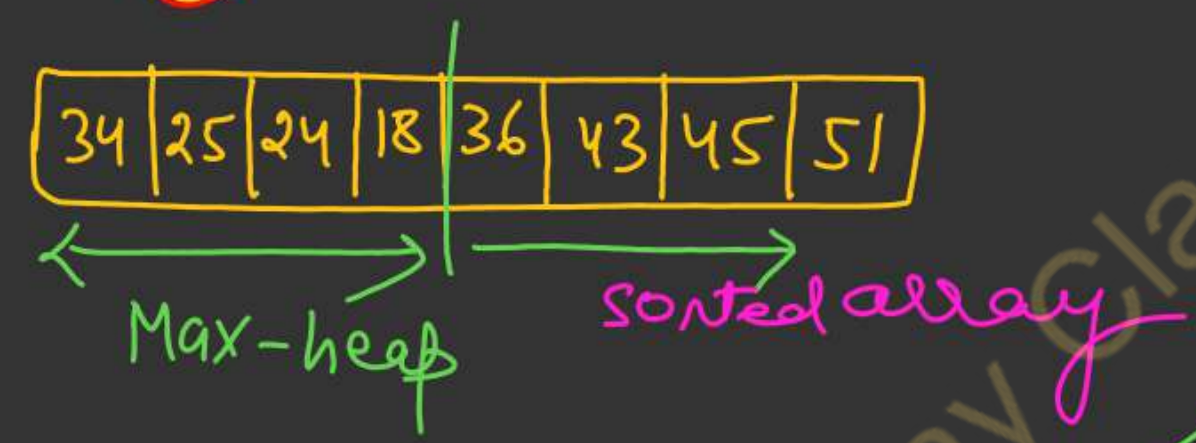
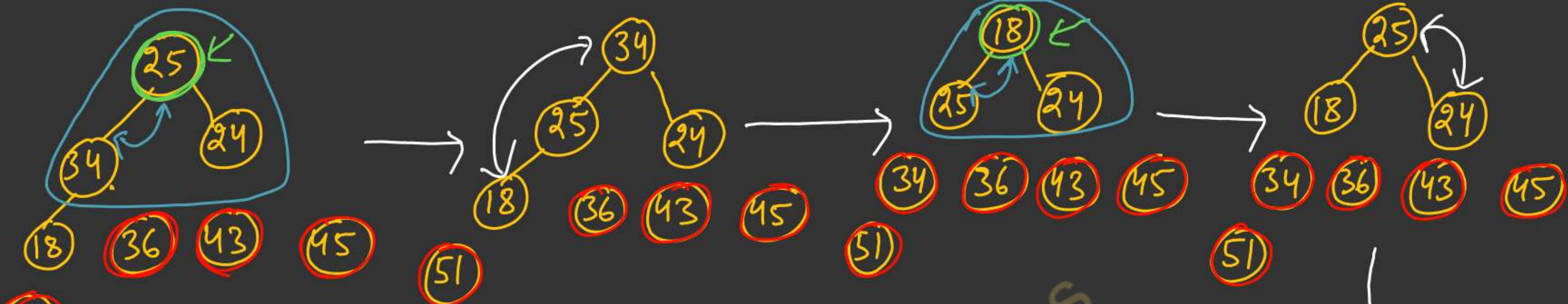
51	36	45	34	25	18	43	24
1	2	3	4	5	6	7	8

Deletion of 51

45	36	43	34	25	18	24	51
----	----	----	----	----	----	----	----

heap

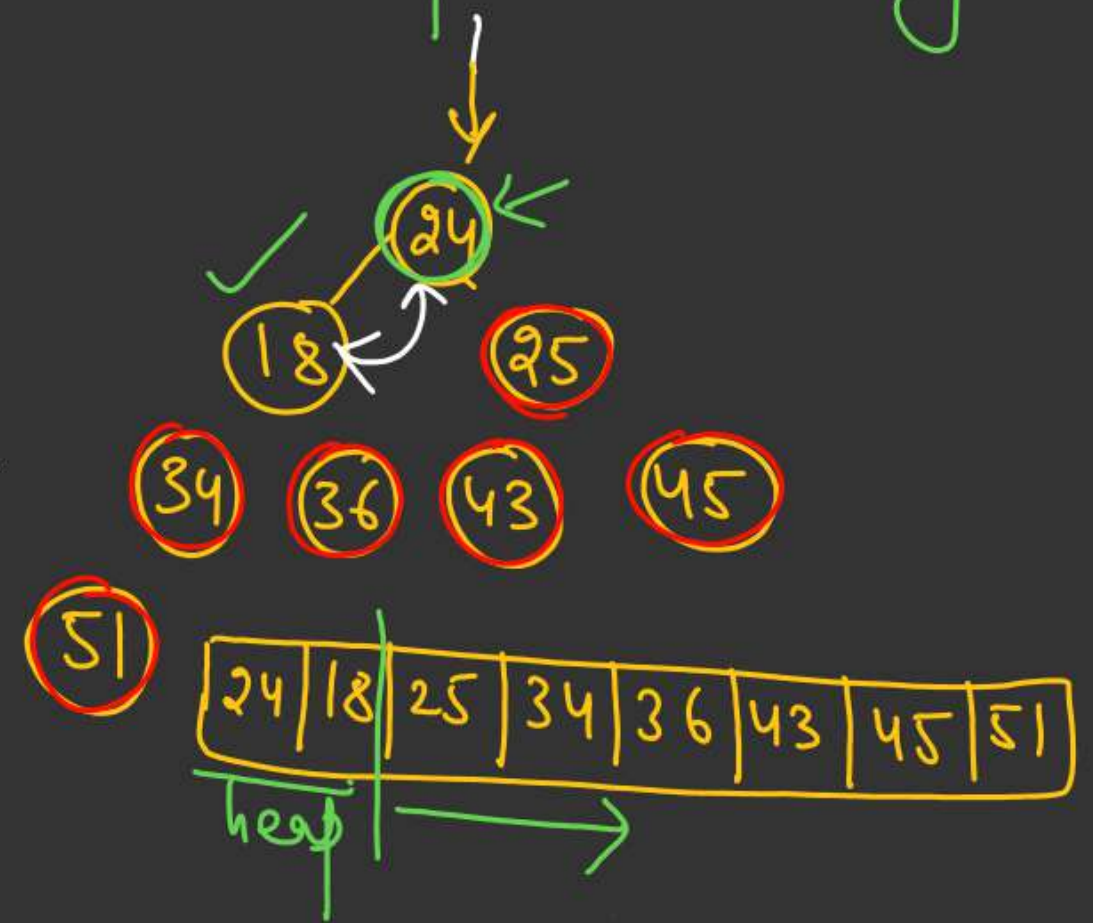




Ans ✓

18	24	25	34	36	43	45	51
----	----	----	----	----	----	----	----

 sorted-Array.



Radix Sort

- **Radix Sort** is a linear sorting algorithm $O(n)$
 - Sorts elements by processing them digit by digit.
 - Efficient sorting algorithm for integers.
 - Digit by digit sorting is performed that is started from the least significant digit to the most significant digit using any stable sorting algorithm.
 - It uses arrays to sort the input array so it is not an in-place sorting algorithm.
- Non-Comparison based sorting.

007, 217, 273

To sort d digits:

RADIX-SORT(A, d)

for i = 1 to d

use a stable sort to sort
array A on digit i

counting
sort

Time Complexity:

$O(n \cdot d)$

Space Complexity:

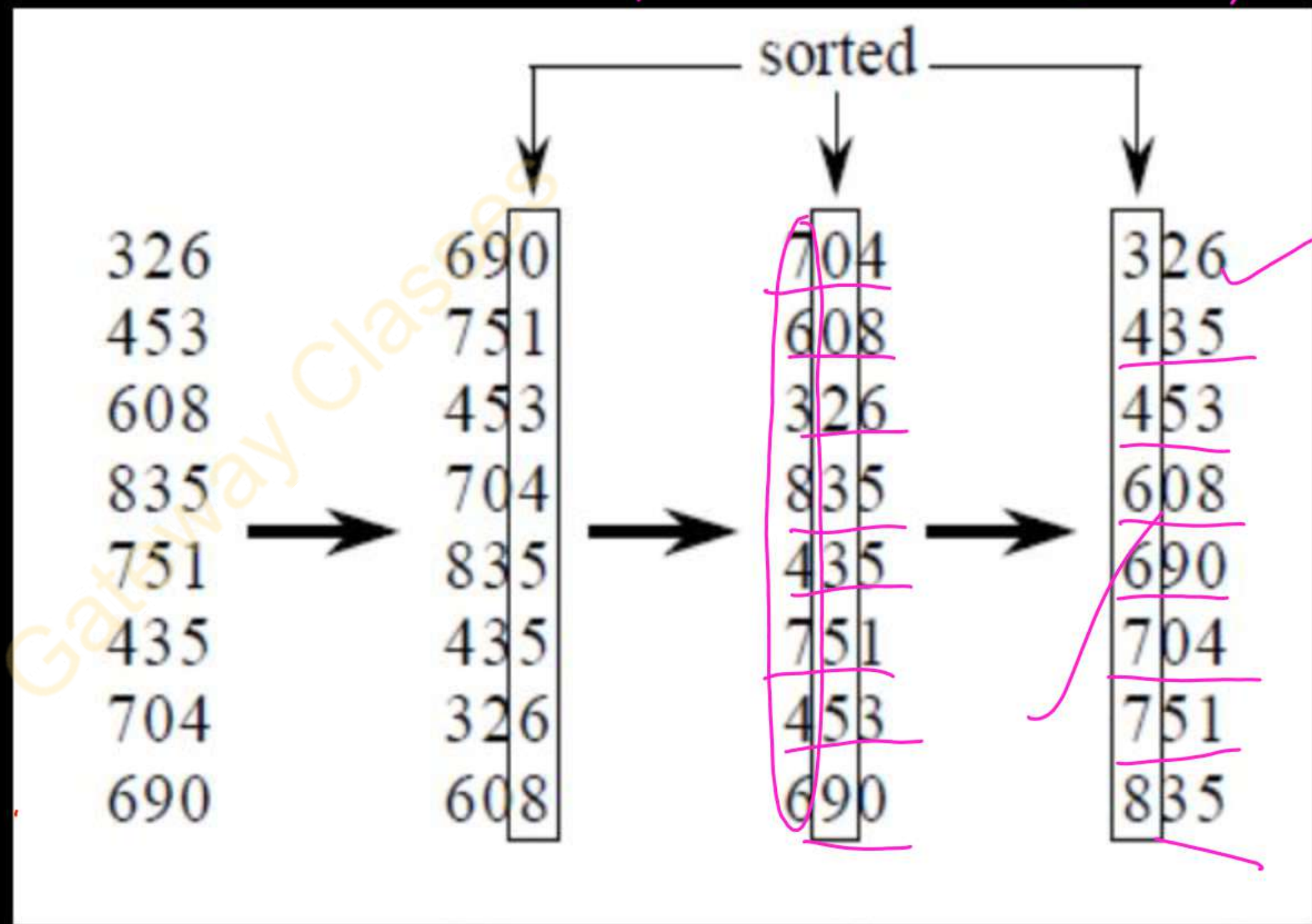
$O(n + k)$

71, 235, 5, 67
↓ ↓ ↓ ↓
071, 235, 005, 067

base

Radix Sort

In this example base = 10 (0-9)



Searching is a process of locating a particular element present in a given set of elements.

It is possible that the search for a particular element is unsuccessful if that element does not exist.

There are two basic techniques available for searching.

1) Linear Search (Sequential Search)

2) Binary Search

3) Index Sequential Search

Gateway Classes

Linear Search

- In Linear search, we search an element or value in a given array by traversing the array from the starting, till the desired element or value is found.
- It compares the element to be searched with all the elements present in the array.
- when the element is matched successfully, it returns the index of the element in the array, else it return -1.
- Linear Search is applied on unsorted or unordered lists, when there are fewer elements in a list.

$n=8$

Key = 18

A

8	6	1	7	3	9	18	4
0	1	2	3	4	5	6	7

Start loop

Step-1

$i=0$

8	6	1	7	3	9	18	4
0	1	2	3	4	5	6	7

$a[i]=8$, Key = 18, $a[i] \neq \text{Key}$
so $i++$

$i=1$

8	6	1	7	3	9	18	4
0	1	2	3	4	5	6	7

$a[i]=6$, Key = 18, $a[i] \neq \text{Key}$
so $i++$

$i=2$

8	6	1	7	3	9	18	4
0	1	2	3	4	5	6	7

$a[i]=2$, Key = 18
 $a[i] \neq \text{Key}$ so $i++$

$i=3$

8	6	1	7	3	9	18	4
0	1	2	3	4	5	6	7

$a[i]=7$, Key = 18
 $a[i] \neq \text{Key}$ so $i++$

$i=4$

8	6	1	7	3	9	18	4
0	1	2	3	4	5	6	7

$a[i]=3$, Key = 18
 $a[i] \neq \text{Key}$ so $i++$

$i=5$

8	6	1	7	3	9	18	4
0	1	2	3	4	5	6	7

$a[i]=9$, Key = 18
 $a[i] \neq \text{Key}$ so $i++$

$i=6$

8	6	1	7	3	9	18	4
0	1	2	3	4	5	6	7

$a[i]=18$, Key = 18

$a[i] = \text{Key}$ ✓

so come out of
loop and print
index position at
which Key = 18 is
found.

Algorithm for linear search

LINEAR_SEARCH(A, N, VAL)

↑ Array

→ Key to be searched.

↓ No of elements

assumes

Step 1: [INITIALIZE] SET Found = 0

Step 2: [INITIALIZE] SET I = 1

Step 3: Repeat Step 4 while I ≤ N

Step 4: IF A[I] == VAL
SET Found = 1
PRINT POS

Go to Step 6

[END OF IF]

SET I = I + 1

[END OF LOOP]

Step 5: IF Found = 0 PRINT "VALUE IS NOT PRESENT IN THE ARRAY"

Step 6: EXIT

It is a flag which initially (shows) that the value to be searched does not exist in the array. If in case during search the element is found in array, then we will set found = 1.

Program to search an element from an array using Linear Search

Time Complexity $\Rightarrow O(n)$

```
#include <stdio.h>
```

```
void SequentialSearch(int a[], int n, int x)
```

```
{
    int i, found = 0;
```

```
    for (i = 0; i <= n-1; i++)
```

```
    {
        if (a[i] == x)
```

```
        {
            found = 1;
```

```
            break;
```

```
        }
    }
```

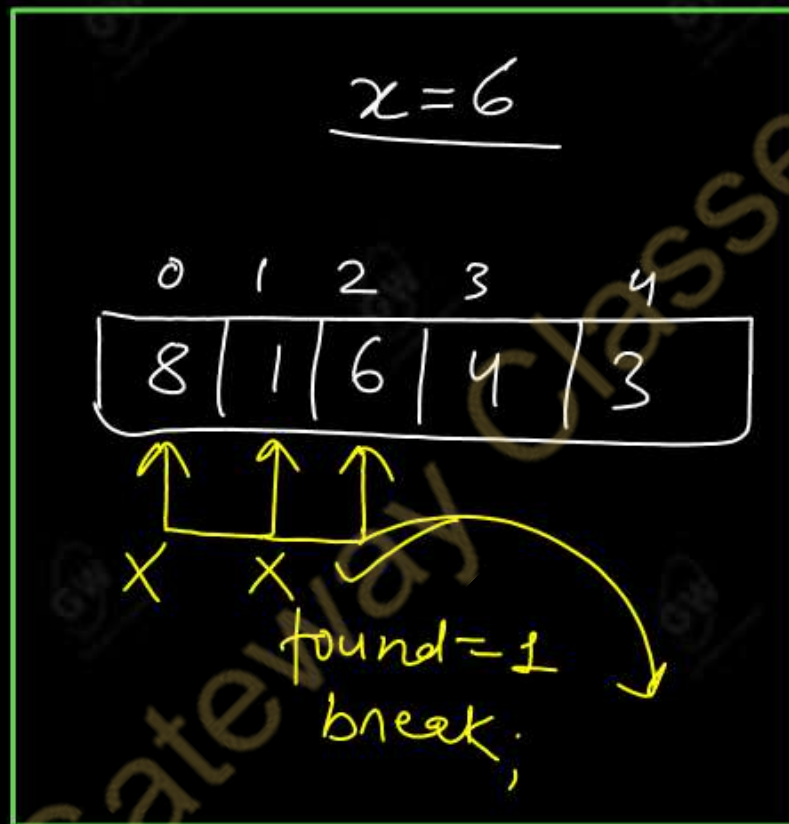
```
if (found == 1)
```

```
    printf("Element found at position %d", i+1);
```

```
else
```

```
    printf("Element not found");
```

```
}
```



```
int main()
```

```
{
    int a[100], i, n, Key;
```

```
    printf("Enter no. of elements");
```

```
    scanf("%d", &n);
```

```
    printf("Enter array");
```

```
    for (i = 0; i <= n-1; i++)
```

```
    {
        scanf("%d", &a[i]);
```

```
    }
```

```
    printf("Enter key to search");
```

```
    scanf("%d", &Key);
```

```
    SequentialSearch(a, n, Key);
```

```
    return 0;
```

```
}
```

1. Illustrate the concept of Sequential search.
2. Illustrate the concept of radix sort.

Gateway Classes



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-3 Lecture-6

Today's Target

- Index Sequential Search ✓
- Binary Search
- Difference between Linear Search and Binary Search
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

Linear Search

$x = 22$

-10	2	7	9	12	15	16	18	22	25
0	1	2	3	4	5	6	7	8	9
X	X	X	X	X	X	X	X	✓	

No. of comparisons $\Rightarrow$ 9 ✓✓

worst case Time complexity $\Rightarrow O(n)$

Binary Search

- Binary Search is useful when there are large number of elements in an array and they are sorted. In binary search, to find **key** into the array, we follow the following steps:
 - Divide the search space into two halves by finding the middle index “mid”
 - Compare the middle element of the search space with the **key**.
 - If the **key** is found at middle element, the process is terminated.
 - If the **key** is not found at middle element, choose which half will be used as the next search space.
 - If the **key** is smaller than the middle element, then the **left** side is used for next search.
 - If the **key** is larger than the middle element, then the **right** side is used for next search.
 - This process is continued until the **key** is found or the total search space is exhausted.

It has a time complexity of $O(\log n)$

Binary Search

$n = 10$

low = 0 ✓
high = n - 1 ✓

-10	2	7	9	12	15	16	18	22	25
-----	---	---	---	----	----	----	----	----	----

$x = 22$

To be searched

low → 0 1 2 3 4 mid 5 low 6 7 mid 8 high 9

(low ≤ high)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

No. of comparisons

3 ✓

low	high	mid	
0	9	$\frac{0+9}{2} = 4$	$a[4] < x$
5	9	$\frac{5+9}{2} = 7$	$a[7] < x$
8	9	$\frac{8+9}{2} = 8$	$a[8] == x$

1) if $a[mid] > x$
Then $high = mid - 1$
2) if $a[mid] < x$
Then $low = mid + 1$

→ position at which 22 exists.

Iterative Implementation of Binary Search

```
#include <stdio.h>

int Binary_Search(int a[], int low, int high, int x)
{
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (a[mid] == x)
            return (mid);
        else if (a[mid] > x)
            high = mid - 1;
        else
            low = mid + 1;
    }
    return (-1);
}
```

```
int main()
{
    int a[100], n, i, result, x;
    printf("Enter no. of elements");
    scanf("%d", &n);
    printf("Enter array");
    for (i = 0; i <= n - 1; i++)
    {
        scanf("%d", &a[i]);
    }
    printf("Enter the no. to be searched");
    scanf("%d", &x);
    result = Binary_Search(a, 0, n - 1, x);
    if (result == -1)
        printf("Unsuccessful Search");
    else
        printf("Element found at index %d", result);
    return 0;
}
```

Imp. Recursive Implementation of Binary Search

```
#include <stdio.h>

int Binary_Search(int a[], int low, int high, int x)
{
    if (low <= high)
    {
        mid = (low + high) / 2;
        if (a[mid] == x)
            return (mid);
        else if (a[mid] > x)
            return Binary_Search(a, low, mid - 1, x);
        else
            return Binary_Search(a, mid + 1, high, x);
    }
    else
        return -1;
}
```

```
int main()
{
    int a[100], n, i, result, x;
    printf("Enter no. of elements");
    scanf("%d", &n);
    printf("Enter array");
    for (i = 0; i <= n - 1; i++)
    {
        scanf("%d", &a[i]);
    }
    printf("Enter the no. to be searched");
    scanf("%d", &x);
    result = Binary_Search(a, 0, n - 1, x);
    if (result == -1)
        printf("Unsuccessful Search");
    else
        printf("Element found at index %d", result);
    return 0;
}
```

Recursive Binary Search Pseudocode / Algorithm

```

BinarySearch(A, low, high, x)
  if (low <= high)
    mid =  $\lfloor (low + high) / 2 \rfloor$ 

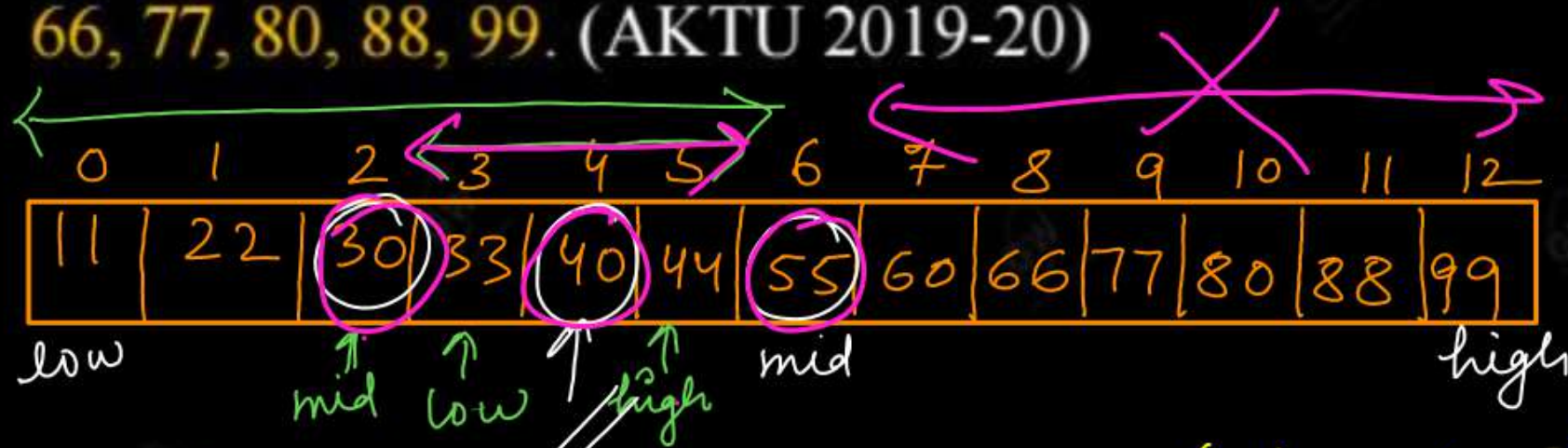
    if (A[mid] == x)
      return mid

    if (A[mid] > x)
      return BinarySearch(A, low, mid - 1)

    if (A[mid] < x)
      return BinarySearch(A, mid + 1, high)

  # endif
  return false // if not present in array
  
```

Q: Apply binary search to find item 40 in the sorted array: 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, 99. (AKTU 2019-20)



low = 0
high = 12

$x = 40$

low	high	mid	
0	12	$\lfloor \frac{0+12}{2} \rfloor = 6$	$a[6] = 55$ $a[6] > x$
0	5	$\lfloor \frac{0+5}{2} \rfloor = 2$	$a[2] = 30$ $a[2] < x$
3	5	$\lfloor \frac{3+5}{2} \rfloor = 4$	$a[4] = 40$ $a[4] = x$

(low ≤ high)

high = mid - 1

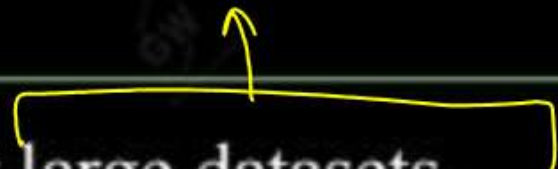
low = mid + 1

No. of Comparisons: 3

⇒ Element found.

Basis	Linear Search	Binary Search
Definition	Checks each element one by one until the desired element is found.	Repeatedly divides the sorted array in half to locate the target element.
Data Requirement	Works on both sorted and unsorted data.	Requires data to be sorted beforehand.
Approach	Sequential search through all elements.	Divide and conquer approach.
Time Complexity	$O(n)$ in worst case.	$O(\log n)$ in worst case.
Space Complexity	$O(1)$	$O(1)$ (iterative) or $O(\log n)$ (recursive).
Efficiency	Less efficient for large datasets.	More efficient for large datasets.

Large Datasets



IndexSequentialSearch(array, key)

{

Divide array into blocks *(Block size given)*

Create two arrays named elements[] and index[] storing first element for each block and storing starting index of each block *respectively.*

Find block where $key \leq first_element$

if block found:

Perform linear search within that block

if key found:

return position

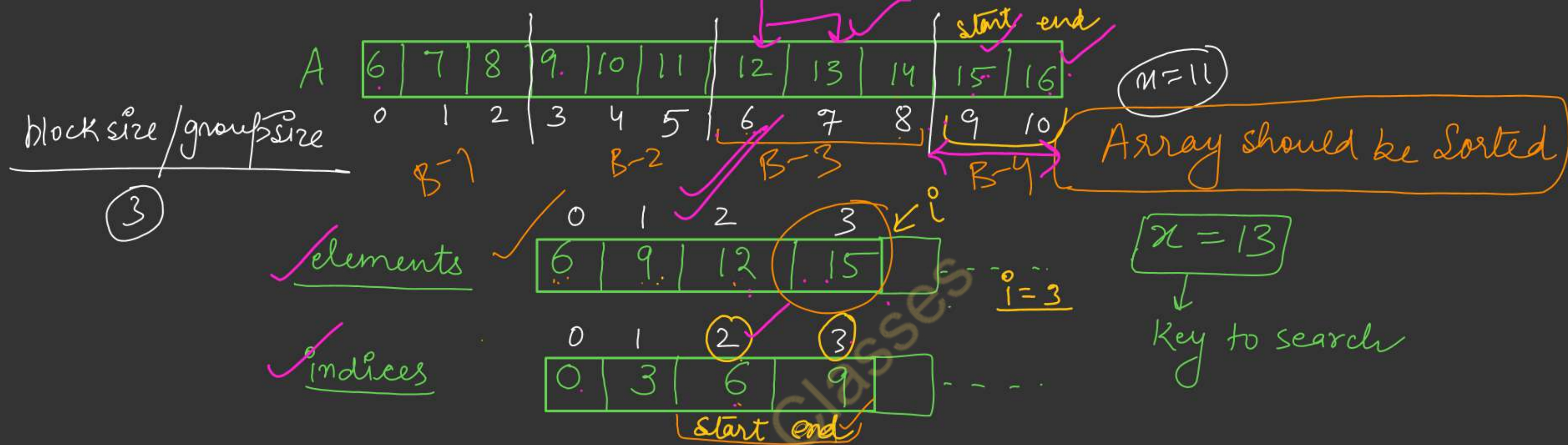
else:

return "Not Found"

else:

return "Not Found"

}



Case - 1. $x < \text{elements}[0] \Rightarrow x$ is not present in the array

B-1 X because $13 > 9$

B-2 X because $13 > 12$

B-3 (Maybe) because $13 < 15$

perform linear search
in Block-3

Index Sequential Search

```
#include <stdio.h>
#include <stdlib.h>
void indexedSequentialSearch(int arr[], int n, int k)
{
    int elements[20], indices[20], i, set = 0;
    int j = 0, x = 0, start, end;
    for (i = 0; i < n; i += 3) → bs = 3
    {
        elements[x] = arr[i];
        indices[x] = i;
        x++;
    }
    if (k < elements[0]) → case - 1
    {
        printf("Not found");
        exit(0);
    }
```

creating elements[] and indices[]

```
for (i = 1; i <= x; i++) // Traverse elements[]
{
    if (k <= elements[i])
    {
        start = indices[i - 1];
        end = indices[i];
        set = 1;
        break;
    }
    if (set == 0) ✓
    {
        start = indices[i - 1];
        end = n - 1;
    }
    for (i = start; i <= end; i++)
    {
        if (k == arr[i])
        {
            j = 1;
            break;
        }
    }
}
```

Index of the block

flag which shows that the block is identified

Linear Search

```
if (j == 1)
    printf("Found at index %d", i);
else
    printf("Not found");
}

void main()
{

    int arr[] = { 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int k;
    printf("what to search");
    scanf("%d", &k);
    indexedSequentialSearch(arr, n, k);
}
```

1. Illustrate the concept of Indexed Sequential search. (AKTU 2024-25)
2. Write a program in C language to implement binary search algorithm. Also discuss the average behaviour of the algorithm. (AKTU 2023-24)
3. Differentiate between linear and binary search algorithm. Write a recursive function to implement binary search. (AKTU 2021-22)
4. Differentiate linear sequential search and binary search. (AKTU 2020-21)
5. With the help of algorithm explain the binary search. (AKTU 2018-19)
6. Write a C program for Index Sequential Search. (AKTU 2020-21)
7. How binary search is different from linear search? Apply binary search to find item 40 in the sorted array: 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, 99. Also discuss the complexity of binary search. (AKTU 2019-20)



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-3 Lecture-7

Today's Target

- Hashing ✓
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

Hashing

➤ Hashing is one of the searching techniques that uses a constant time.

➤ The time complexity in hashing is $O(1)$.

$\frac{\text{Linear Search}}{O(n)}$	$\frac{\text{Binary Search}}{O(\log n)}$	$\rightarrow$ input size
-------------------------------------	--	--------------------------

➤ The main idea is to create (key/value) pairs. If the key is given, then algorithm computes the index at which the value would be stored.

➤ In Hashing, the hash table and hash function are used.

➤ Using hash function, we can calculate the address at which the value can be stored.

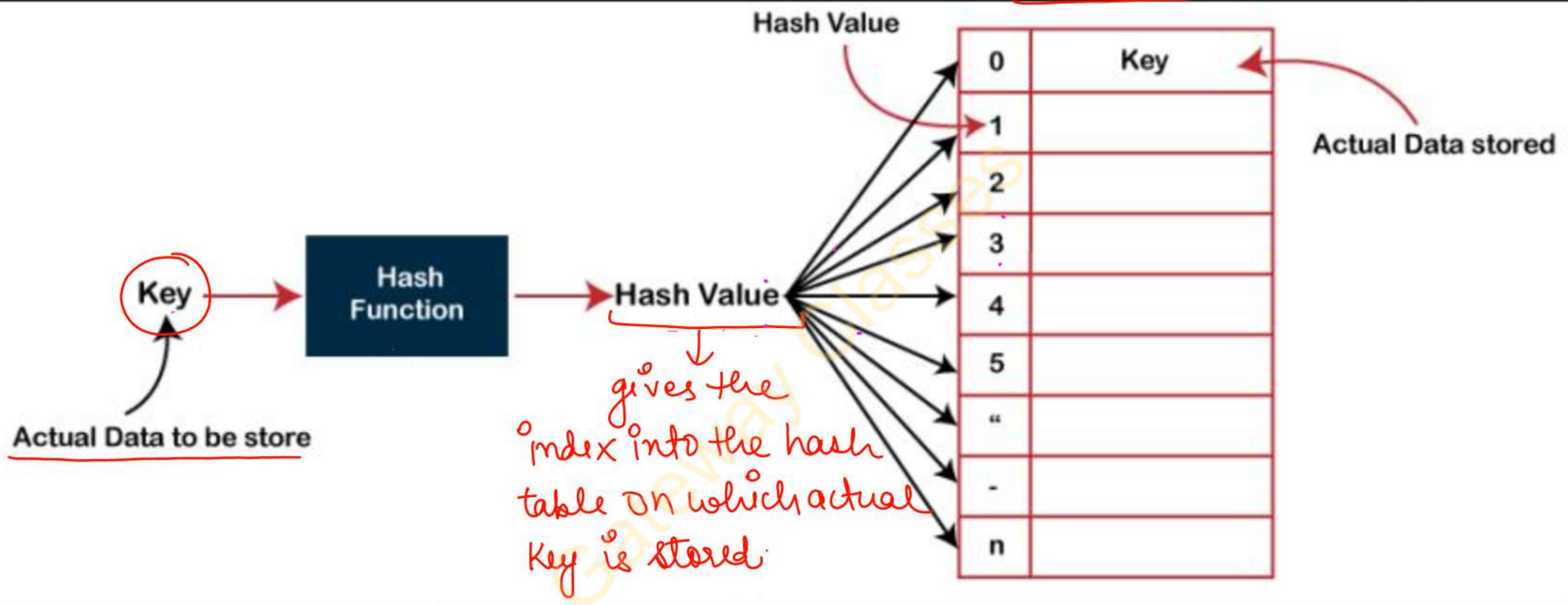
➤ It can be written as:

$$\text{Index} = \text{hash}(\text{key})$$

hash function

index

Hash Table



- Hash table is one of the most important data structures that uses a hash function that maps a given value with a key to access the elements faster.
- A Hash table stores information, and has basically two main components, key and value.
- The hash table can be implemented with the help of an associative array.
- The efficiency of mapping depends upon the efficiency of the hash function used for mapping.

For example, suppose the key value is John and the value is the phone number, so when we pass the key value in the hash function shown as below:

$$\text{Hash}(\text{key}) = \text{index};$$

What is a Hash function? (AKTU PYQ)

A hash function creates a mapping from an input key to an index in hash table.

A good hash function should have the following properties:

1. Efficient ✓
2. Should uniformly distribute the keys to each index of hash table.
3. Should minimize collisions.
4. Should have a low load factor (number of items in the table divided by the size of the table).

Index Key

0	
1	
2	
3	
4	
5	
6	
...	

X

There are three ways of calculating the hash function:

1. Division method
2. Folding method
3. Mid square method

1. Division Method: The hash function divides the value k by M and then uses the remainder obtained.

Formula:

$$h(K) = k \bmod M$$

↑ Key
size of hash table

Here, k is the key value, and M is the size of the hash table.

Example:

$$\begin{aligned}
 k &= 12345 & K \bmod M \\
 M &= 95 & \Rightarrow 12345 \bmod 95 = 90
 \end{aligned}$$

$$M = 11, K = 1276$$

$$h(1276) = 1276 \bmod 11 = 0$$

Pros:

1. Good for any value of M . $h(12345) = 12345 \bmod 95 = 90$
2. Very fast since it requires only a single division operation.

Cons:

1. Poor performance since consecutive keys map to consecutive hash values in the hash table.
2. Sometimes extra care should be taken to choose the value of M .

2. Mid Square Method:

The mid-square method involves two steps to compute the hash value-

1. Square the value of key k i.e. k^2
2. Extract the middle r digits as the hash value.

Formula: $h(K) = h(k \times k)$

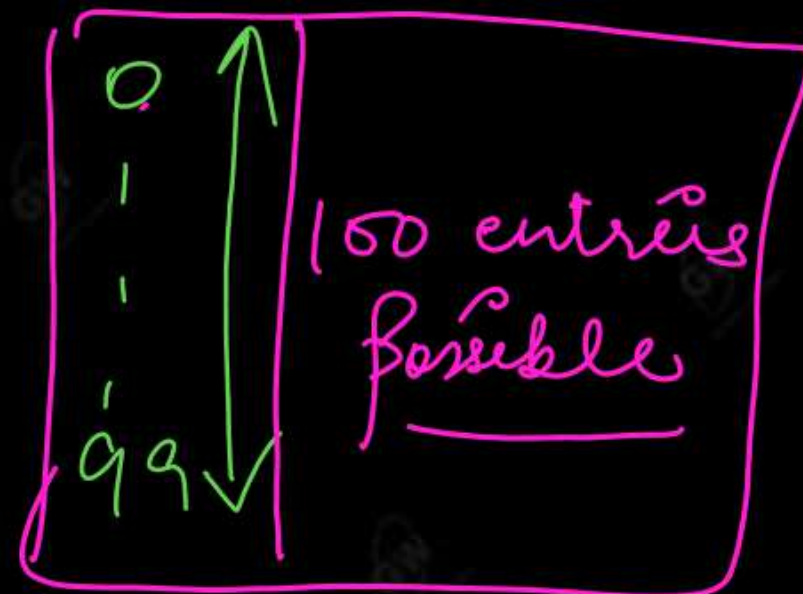
Here, k is the key value.

The value of r can be decided based on the size of the table.

Example: Suppose hash table has 100 memory locations. So, $r = 2$ because two digits are required to map the key to the memory location.

$$\begin{aligned}
 k &= 60 \\
 h(k) &= k \times k = 60 \times 60 = 3600 \\
 h(60) &= \underline{60}
 \end{aligned}$$

Hash table



3. Digit Folding Method:

This method involves two steps:

1. Divide the key-value k into a number of parts i.e. $k_1, k_2, k_3, \dots, k_n$, where each part has the same number of digits except for the last part that can have lesser digits than the other parts.
2. Add the individual parts. The hash value is obtained by ignoring the last carry if any.

Formula: $k = k_1, k_2, k_3, k_4, \dots, k_n$

$$s = k_1 + k_2 + k_3 + k_4 + \dots + k_n$$

$h(K) = s$ Here, s is obtained by adding the parts of the key k

Example: $k = 12345$

$$k_1 = 12, k_2 = 34, k_3 = 5$$

$$s = k_1 + k_2 + k_3$$

$$= 12 + 34 + 5$$

$$= 51$$

$$h(K) = 51$$

What is Collision in Hashing?

When two or more keys have the same hash value, a **collision** happens. This **collision** creates problem in searching, insertion, deletion, and updating of value.

For example:

Keys :- 10, 15, size of hash Table = 5

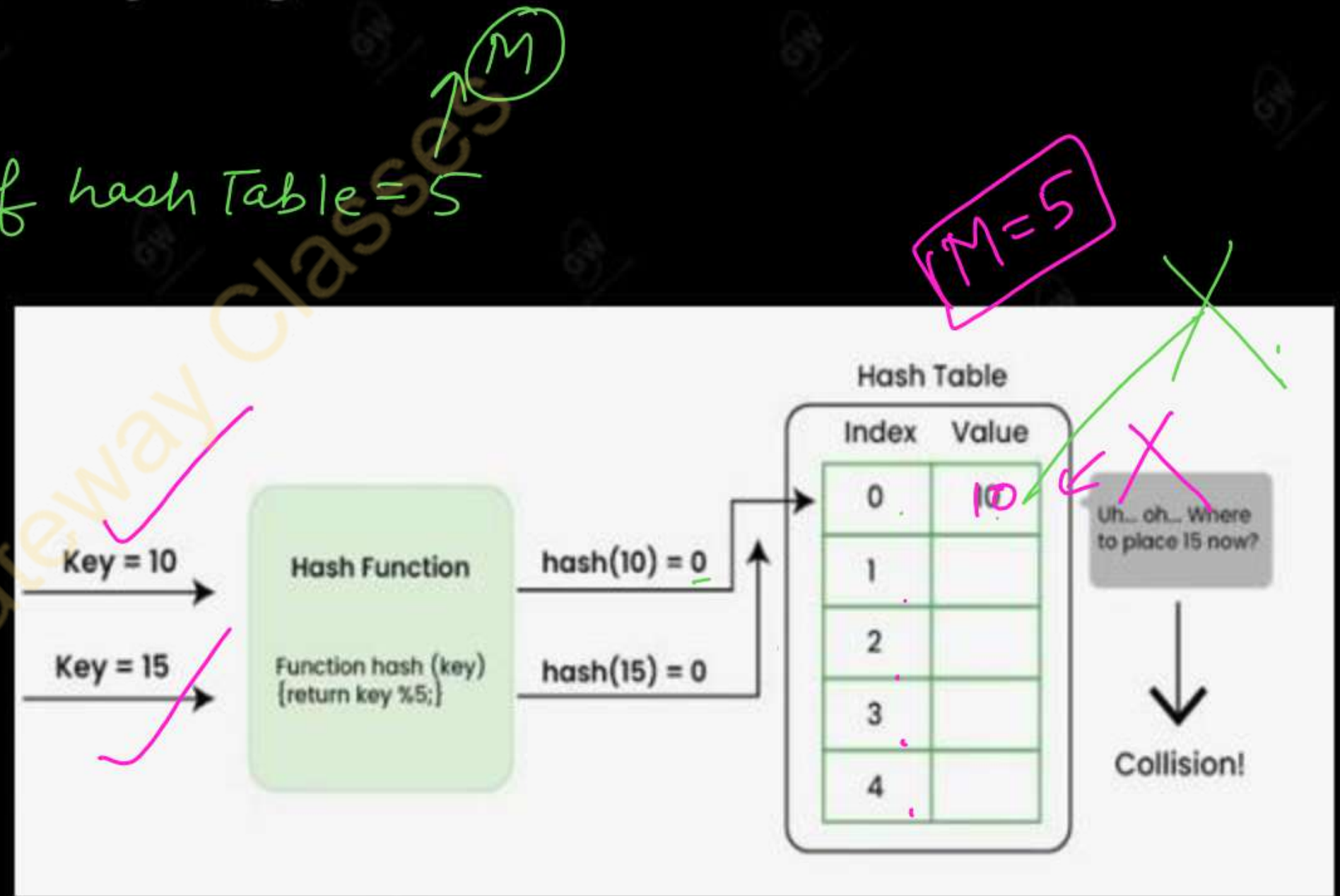
$$h(K) = K \bmod 5$$

Key = 10

$$h(10) = 10 \bmod 5 = 0$$

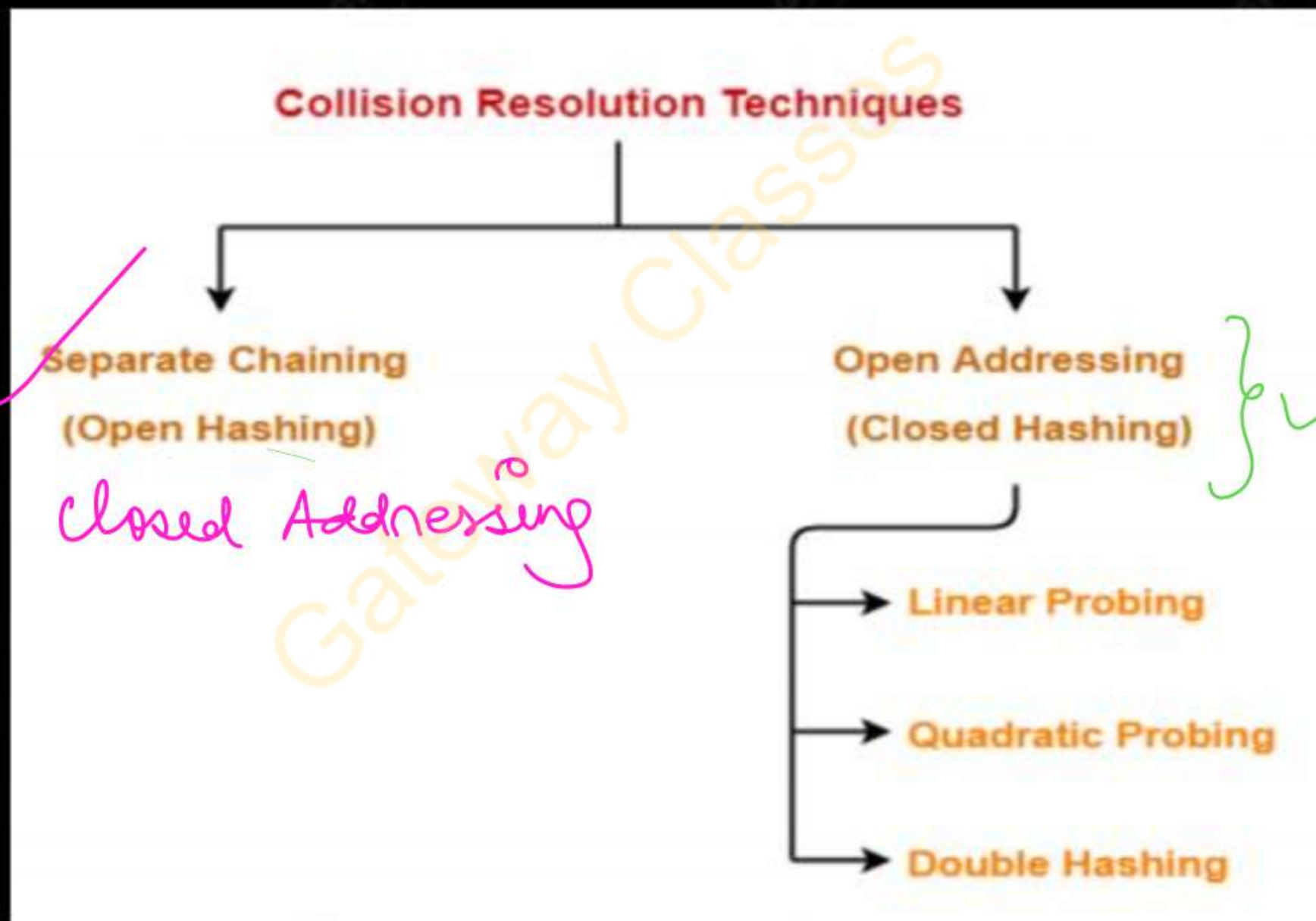
Key = 15

$$h(15) = 15 \bmod 5 = 0$$



To resolve these collisions, following are the collision handling techniques:

1. Open Addressing: It is also known as closed Hashing.
2. Separate Chaining: It is also known as open Hashing.



In open addressing, all the keys are stored inside the hash table and No key is stored outside the hash table.

a) Linear Probing:

- The simplest approach to resolve a collision is linear probing.
- In this technique, if a value is already stored at a location generated by $h(k)$, it means collision occurred then we do a sequential search to find the empty location.
- Here the idea is to place a value in the next available position. Because in this approach searches are performed sequentially so it's known as linear probing.
- Here array or hash table is considered circular because when the last slot reached and an empty location not found then the search proceeds to the first location of the array.

If the location is empty then store value otherwise find the next location.

- Following hash function is used to resolve the collision in:

$$h(k, i) = [h(k) + i] \bmod m$$

Where m = size of the hash table, $h(k) = (k \bmod m)$, i = probe number that varies from 0 to $m-1$.

- Probes is a count to find the free location for each value to store in the hash table.

Q: A hash function H defined as $H(\text{key}) = \text{key} \% 7$, with linear probing, is used to insert the key 37, 38, 72, 48, 98, 11, 66 into a table indexed from 0 to 6. what will be the location of key 11? Justify your answer, also count the total number of collisions in this probing. (AKTU 2020-21)

Solⁿ.

$H(\text{Key}) = \text{Key} \% 7$ ✓

Keys :- 37, 38, 72, 48, 98, 11, 66

Hash Table

Index	Key
0	
1	
2	37 ✓
3	38 ✓
4	72 ✓
5	
6	

hash Function $h(K)$

$h(37) = 37 \% 7 = 2$

$h(38) = 38 \% 7 = 3$

$h(72) = 72 \% 7 = 2 \Rightarrow \text{collision}$ $h(K) = 2$

$h(K, i) = [h(K) + i] \bmod m$

$i=0 \rightarrow$

$h(72, 0) = [2 + 0] \bmod 7 \Rightarrow 2 \bmod 7 \Rightarrow 2$

$i=1 \rightarrow$

$h(72, 1) = [2 + 1] \bmod 7 \Rightarrow 3 \bmod 7 = 3$ (collision)

$i=2 \rightarrow$

$h(72, 2) = [2 + 2] \bmod 7 = 4 \bmod 7 = 4$ ✓

$i = 0, 1, 2, 3, \dots$

Q: A hash function H defined as $H(\text{key}) = \text{key} \% 7$, with linear probing, is used to insert the key 37, 38, 72, 48, 98, 11, 66 into a table indexed from 0 to 6. what will be the location of key 11? Justify your answer, also count the total number of collisions in this probing. (AKTU 2020-21)

Solⁿ.

$$H(\text{Key}) = \text{Key} \% 7$$

Keys :- 37, 38, 72, 48, 98, 11, 66

Index	Key	Key
0	98	4) 48
1	66	5) 98
2	37	6) 11
3	38	
4	72	7) 66
5	11	
6	48	

hash Function $h(K)$

$$h(48) = 48 \bmod 7 = 6$$

$$h(98) = 98 \bmod 7 = 0$$

$$h(11) = 11 \bmod 7 = 4 \Rightarrow \text{collision}$$

$$\begin{aligned} i=1 \quad h(11, 1) &= [h(11) + 1] \bmod 7 \\ &= [4 + 1] \bmod 7 \Rightarrow 5 \checkmark \end{aligned}$$

$$h(66) = 66 \bmod 7 = 3 \Rightarrow \text{collision}$$

$$h(66, 1) = [3 + 1] \bmod 7 = 4 \text{ coll.}$$

$$h(66, 2) = [3 + 2] \bmod 7 = 5 \text{ coll.}$$

$$h(66, 3) = [3 + 3] \bmod 7 = 6 \text{ coll.}$$

$$h(66, 4) = [3 + 4] \bmod 7 = 0 \text{ Collision}$$

$$h(66, 5) = [3 + 5] \bmod 7 = 1 \checkmark$$

No of collisions = 8

Q: Construct a Hash table that contains 10 buckets and uses linear probing to resolve collisions. The key values are integers and the hash function used is key % 10. Insert values 43, 165, 62, 123, 142 in the table. Show all the steps and find the location of the key value 142? (AKTU 2024-25)

Index Key

0	
1	
2	62
3	43
4	123
5	165
6	142
7	
8	
9	

$$h(K) = K \bmod 10, \quad h(k, i) = [h(k) + i] \bmod m$$

Keys:- 43, 165, 62, 123, 142

No. of collisions = 5

Key
1) 43
2) 165
3) 62
4) 123
5) 142

hash value

$$h(43) = 43 \bmod 10 = 3$$

$$h(165) = 165 \bmod 10 = 5$$

$$h(62) = 62 \bmod 10 = 2$$

$$h(123) = 123 \bmod 10 = 3$$

$$h(142) = 142 \bmod 10 = 2$$

$$h(142, 1) = [2 + 1] \bmod 10 = 3$$

$$h(142, 2) = [2 + 2] \bmod 10 = 4$$

$$h(142, 3) = [2 + 3] \bmod 10 = 5$$

$$h(123, 1) = [3 + 1] \bmod 10 = 4$$

$$h(142, 4) = [2 + 4] \bmod 10 = 6$$

Q: The keys 12, 17, 13, 2, 5, 43, 5 and 15 are inserted into an initially empty hash table of length 15 using open addressing with hash function $h(k) = k \bmod 10$ and linear probing. Find resultant hash

table? (AKTU 2021-22)

Index Key

0	
1	
2	12
3	2
4	43
5	5
6	5
7	17
8	15
9	

Key

1) 12

2) 17

3) 2

4) 5

5) 43

6) 5

7) 15

hash value

$$h(12) = 12 \bmod 10 = 2$$

$$h(17) = 17 \bmod 10 = 7$$

$$h(2) = 2 \bmod 10 = 2 \text{ collision } h[2, 1] = [2 + 1] \bmod 10 = 3 \checkmark$$

$$h(5) = 5 \bmod 10 = 5$$

$$h(43) = 43 \bmod 10 = 3 \text{ collision } h[43, 1] = [3 + 1] \bmod 10 = 4 \checkmark$$

$$h(5) = 5 \bmod 10 = 5 \text{ collision } h[5, 1] = [5 + 1] \bmod 10 = 6 \checkmark$$

$$h(15) = 15 \bmod 10 = 5 \text{ collision } h[15, 1] = [5 + 1] \bmod 10 = 6$$

$$h[15, 2] = [5 + 2] \bmod 10 = 7$$

$$h[15, 3] = [5 + 3] \bmod 10 = 8 \checkmark$$

No. of collisions = 6

Disadvantages of Linear Probing

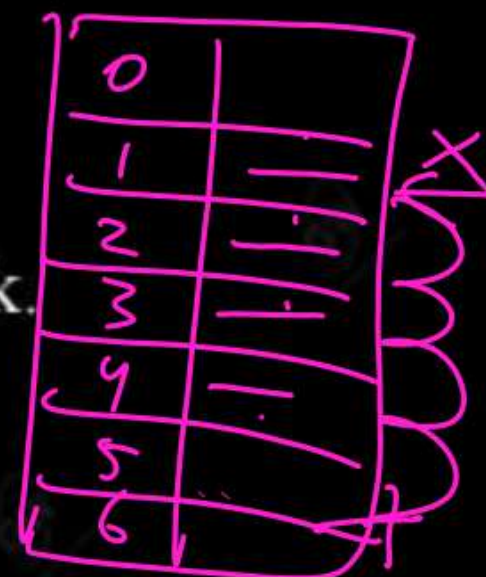
1. Primary Clustering

- When multiple keys hash to the same or nearby slots, they form a **cluster** of occupied cells.
- Future insertions that hash anywhere into that cluster must probe linearly past it, making the cluster grow even larger.
- As a result, **search, insertion, and deletion times increase**, especially when the table becomes moderately full.

Example:

If keys 12, 22, 32, 42 all hash to index 2, they will occupy 2, 3, 4, 5 forming a large block.

Any new key hashing near 2 will have to skip through this entire cluster.



Quadratic Probing

In this technique, if a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$h(k, i) = (h(k) + i^2) \bmod m$$

- where m is size of hash table, $h(k) = (k \bmod m)$, i is the probe number that varies from 0 to $m-1$,
- Quadratic probing solves the clustering problem which is in linear probing because instead of doing a linear search, it does a quadratic search.
- The disadvantage of quadratic probing is it does not search all locations of the list.

Q: A hash function H defined as $H(\text{key}) = \text{key} \% 7$, with quadratic probing, is used to insert the key 37, 38, 72, 48, 98, 11, 66 into a table indexed from 0 to 6. what will be the location of key 11? Justify your answer, also count the total number of collisions in this probing.

Index Key

0	48
1	98
2	37
3	38
4	11
5	
6	72

- 1) 37
- 2) 38
- 3) 72
- 4) 48
- 5) 98
- 6) 11

Key

Keys — 37, 38, 72, 48, 98, 11, 66

hash value

$$h(37) = 37 \% 7 = 2$$

$$h(38) = 38 \% 7 = 3$$

$$h(72) = 72 \% 7 = 2 \text{ collision}$$

$$h(48) = 48 \% 7 = 6 \text{ collision}$$

$$h(98) = 98 \% 7 = 0 \text{ collision}$$

$$h(11) = 11 \% 7 = 4$$

$$h(66) = 66 \% 7 = 3 \text{ collision}$$

$$h(11) = 11 \% 7 = 4$$

$$h[K, i] = [h(K) + i^2] \% m$$

$$h[72, 1] = [2 + 1 \times 1] \% 7 = 3 \text{ coll}$$

$$h[72, 2] = [2 + 2 \times 2] \% 7 = 6 \checkmark$$

0, 1, 2, ..., 6

Q: A hash function H defined as $H(\text{key}) = \text{key} \% 7$, with quadratic probing, is used to insert the key 37, 38, 72, 48, 98, 11, 66 into a table indexed from 0 to 6. what will be the location of key 11? Justify your answer, also count the total number of collisions in this probing.

Index Key

0	48
1	98
2	37
3	38
4	11
5	66
6	72

7) 66

Keys — 37, 38, 72, 48, 98, 11, 66

hash value

$$h(66) = 66 \bmod 7 = 3 \text{ collision}$$

$$h(66, 1) = [3 + 1 \times 1] \bmod 7 = 4 \text{ collision}$$

$$h(66, 2) = [3 + 2 \times 2] \bmod 7 = 0 \text{ collision}$$

$$h(66, 3) = [3 + 3 \times 3] \bmod 7 = 5 \checkmark$$

No. of collisions = 7



➤ Double Hashing

- Double hashing uses two hash functions to calculate an empty location.
- The hash function can be given as:

$$h(k, i) = [h_1(k) + ih_2(k)] \bmod m$$

where m is the size of the hash table, $h_1(k)$ and $h_2(k)$ are two hash functions

$$h_1(k) = k \bmod m,$$

$$h_2(k) = k \bmod m$$

i is the probe number that varies from 0 to $m-1$

- Double hashing requires more computational time because two hash functions need to be computed.

Q: A hash function H defined as $H(\text{key}) = \text{key} \% 7$, with double hashing, is used to insert the key 37, 38, 72, 48, 98, 11, 66 into a table indexed from 0 to 6. what will be the location of key 11? Justify your answer, also count the total number of collisions in this probing.

$$h_1(K) = K \bmod 7$$

$$h_2(K) = K \bmod 7$$

Index Key

0	
1	
2	37
3	38
4	72
5	
6	48

Key

1) 37

2) 38

3) 72

4) 48

Keys $\rightarrow$ 37, 38, 72, 48, 98, 11, 66

Hash value

$$h(37) = 37 \bmod 7 = 2$$

$$h(38) = 38 \bmod 7 = 3$$

$$h(72) = 72 \bmod 7 = 2 \text{ collision}$$

$$h(K, i) = [h_1(K) + i \times h_2(K)] \bmod m$$

$$h(72, 1) = [2 + 1 \times 2] \bmod 7 = 4 \checkmark$$

$$h(48) = 48 \bmod 7 = 6$$

$$h_2 K = 72 \bmod 7 = 2$$

Q: A hash function H defined as $H(\text{key}) = \text{key} \% 7$, with double hashing, is used to insert the key 37, 38, 72, 48, 98, 11, 66 into a table indexed from 0 to 6. what will be the location of key 11? Justify your answer, also count the total number of collisions in this probing.

Index Key

0	98
1	11
2	37
3	38
4	72
5	66
6	48

Key

5) 98

6) 11

7) 66

Keys $\rightarrow$ 37, 38, 72, 48, 98, 11, 66

Hash value

$$h(98) = 98 \bmod 7 = 0$$

$$h(11) = 11 \bmod 7 = 4 \rightarrow \text{collision}$$

$$h(11, 1) = [4 + 1 \times 4] \bmod 7 = 1$$

$$h(66) = 66 \bmod 7 = 3 \rightarrow \text{collision}$$

$$h(66, 1) = [3 + 1 \times 3] \bmod 7 = 6 \text{ coll.}$$

$$h(66, 2) = [3 + 2 \times 3] \bmod 7 = 2 \text{ coll.}$$

$$h(66, 3) = [3 + 3 \times 3] \bmod 7 = 5$$

No. of collisions = 4

$$h_1(K) = K \bmod 7$$

$$h_2(K) = K \bmod 7$$

$$h_1(K) = h_2(K) = 4$$

$$h_1(K) = h_2(K) = 3$$

Closed Addressing (Open Hashing)

- In closed addressing, all the keys are stored inside and outside the hash table.
- Each slot of the hash table is linked with linked list So if a collision occurs key stores in the linked list.

Separate Chaining :

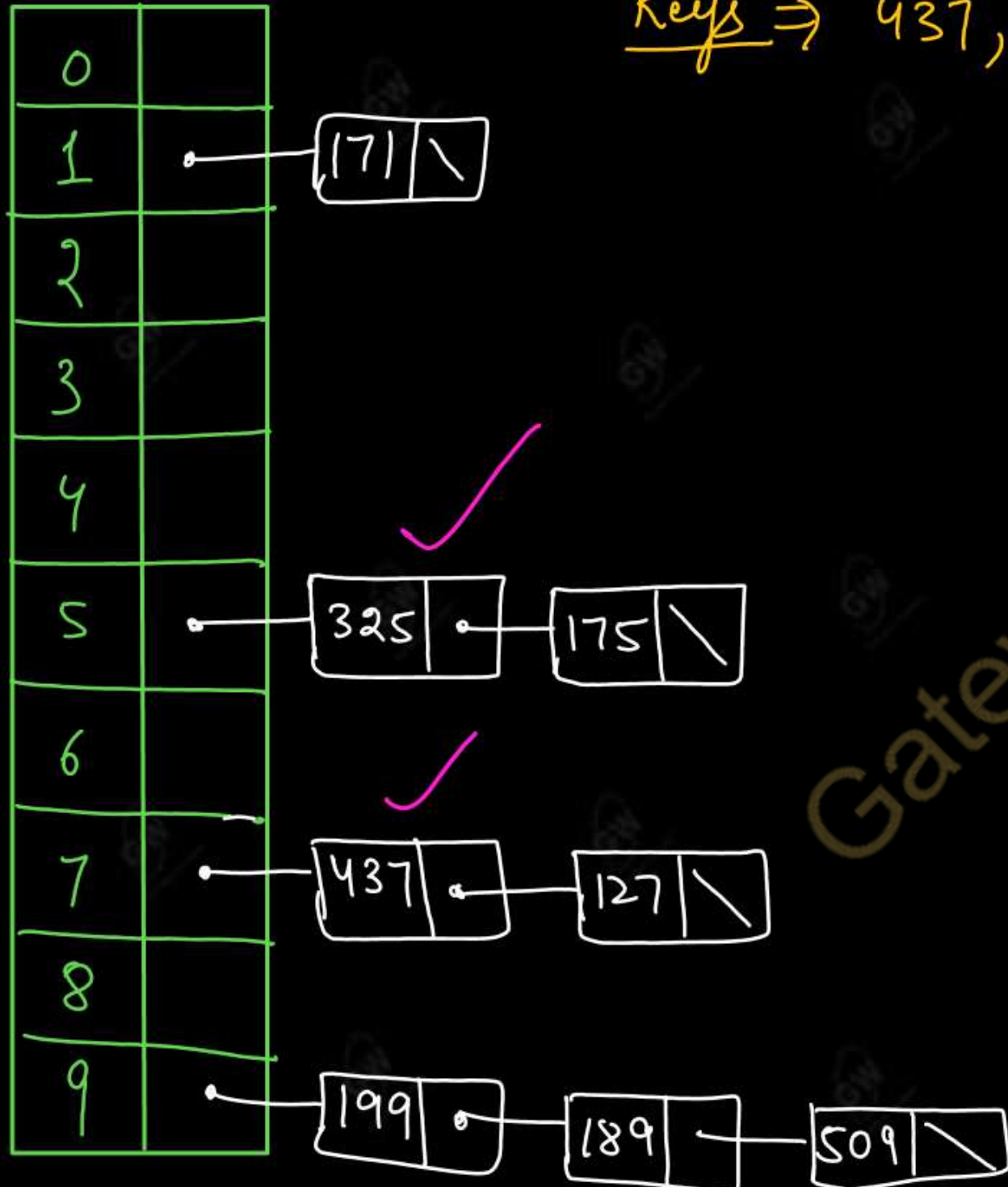
- In the chaining approach, the hash table is an array of linked lists. This means each index of the hash table has its own linked list.
- if Collision arises then a new value will be stored in the linked list of that index. And at index, this linked list appears like a chain that is why this technique is known as the chaining technique.

Time complexity of Chaining :

For Searching : In worst case all values are present in the linked list of same index. So, sequential search is performed to search the value. So, time complexity will be $O(n)$ for searching.

Q: Sequence of numbers { 437, 325, 175, 199, 171, 189, 127, 509} and a hash function $H(X) = X \bmod 10$. Show the results of separate chaining hash table.

Keys $\Rightarrow$ 437, 325, 175, 199, 171, 189, 127, 509



- Key
- 1) 437
 - 2) 325
 - 3) 175
 - 4) 199
 - 5) 171
 - 6) 189
 - 7) 127
 - 8) 509

- hash value
- $h(437) = 437 \bmod 10 = 7$
 - $h(325) = 325 \bmod 10 = 5$
 - $h(175) = 175 \bmod 10 = 5$
 - $h(199) = 199 \bmod 10 = 9$
 - $h(171) = 171 \bmod 10 = 1$
 - $h(189) = 189 \bmod 10 = 9$
 - $h(127) = 127 \bmod 10 = 7$
 - $h(509) = 509 \bmod 10 = 9$

1. Construct a Hash table that contains 10 buckets and uses linear probing to resolve collisions. The key values are integers and the hash function used is $\text{key} \% 10$. Insert values 43, 165, 62, 123, 142 in the table. Show all the steps and find the location of the key value 142? (AKTU 2024-25)
2. What are the 2 different forms of hashing? (AKTU 2023-24)
3. What are the disadvantages of linear probing in hashing? Discuss how quadratic probing can be used to solve some of these problems. (AKTU 2023-24)
4. What is Hashing? Explain division method to compute the hash function and also explain the collision resolution strategies used in hashing. (AKTU 2022-23)
5. The keys 12, 17, 13, 2, 5, 43, 5 and 15 are inserted into an initially empty hash table of length 15 using open addressing with hash function $h(k) = k \bmod 10$ and linear probing. Find resultant hash table? (AKTU 2021-22)
6. Differentiate between linear and quadratic probing techniques (AKTU 2021-22)

7. Explain any three commonly used hash function with the suitable example? A hash function H defined as $H(\text{key}) = \text{key} \% 7$, with linear probing, is used to insert the key 37, 38, 72, 48, 98, 11, 66 into a table indexed from 0 to 6. what will be the location of key 11? Justify your answer, also count the total number of collisions in this probing. (AKTU 2020-21)
8. What is hashing? Give the characteristics of hash function. Explain collision resolution technique in hashing. (AKTU 2019-20)
9. Classify the Hashing Functions based on the various methods by which the key value is found. What are the types of Collision Resolution Techniques and the methods used in each of the type? (AKTU 2018-19)
10. What do you mean by hashing and collision? Discuss the advantages and disadvantages of hashing over other searching techniques ? (AKTU 2017-18)



Gateway Classes



Empowering Learners, Transforming Futures

B.Tech Courses



GET IT ON
Google Play



Download on the
App Store

Helpline No- 7819 0058 53